

Alke Wallet

Documentación técnica del desarrollo

Módulo 7: Acceso a datos en aplicaciones Django

Full Stack Python Trainee · Talento Digital / Alkemy

Demostración en línea: <https://javiermll.github.io/alke-wallet-django/>

Publicada en Render, con la base de datos PostgreSQL en Neon.

Autor: Javier Muñoz Llanos

Fecha: 8-Oct-2026

Índice

1. Resumen del proyecto.....	6
Demostración en línea	6
Funciones.....	6
Tecnologías.....	7
Instalación y ejecución local.....	7
1. Clonar el repositorio	7
2. Crear y activar el entorno virtual.....	8
3. Instalar las dependencias	8
4. Configurar las variables de entorno	8
5. Aplicar las migraciones.....	8
6. Cargar datos de demostración (opcional)	8
7. Crear un superusuario (para el panel de administración).....	9
8. Ejecutar el servidor.....	9
2. Cumplimiento de la consigna	10
Estado por etapa	10
3. Arquitectura y modelo de datos.....	11
Estructura de carpetas	11
Modelo de datos	12
Reglas de negocio.....	13
Flujo de Git	13
4. Desarrollo por etapas	14
Etapa 0: entorno y proyecto base	15
Objetivo	15
Qué se hizo	15
Comandos.....	15
Decisiones y alternativas descartadas.....	15
Incidencias y solución	16
Resultado.....	16
Etapa 1: conexión a la base de datos	17
Objetivo	17
Qué se hizo	17
Comandos.....	18
Decisiones y alternativas descartadas.....	18

Incidencias y solución	18
Resultado.....	18
Etapa 2: app gestion y modelos	20
Objetivo	20
Qué se hizo	20
Comandos.....	21
Decisiones y alternativas descartadas.....	21
Resultado.....	22
Etapa 3: migraciones	23
Objetivo	23
Qué se hizo	23
Comandos.....	24
Resultado.....	25
Etapa 4: consultas personalizadas.....	29
Objetivo	29
Qué se hizo	29
Decisiones y alternativas descartadas.....	44
Incidencias y solución	45
Etapa 5: panel de administración.....	46
Objetivo	46
Qué se hizo	46
Comandos.....	53
Decisiones y alternativas descartadas.....	53
Limitaciones conocidas.....	53
Reflexiones	53
Etapa 6: pantallas con vistas basadas en clases.....	54
Objetivo	54
Qué se hizo	54
Decisiones y alternativas descartadas.....	74
Incidencias y solución	75
Limitaciones conocidas.....	75
Reflexiones	75
Etapa 7: autenticación y archivos estáticos	76
Objetivo	76
Qué se hizo	76

Comandos.....	90
Decisiones y alternativas descartadas.....	91
Limitaciones conocidas.....	91
Reflexiones	91
Etapa 8: pruebas automatizadas.....	92
Objetivo	92
Qué se hizo	92
Comandos.....	92
Decisiones y alternativas descartadas.....	93
Resultado.....	93
Limitaciones conocidas.....	93
Reflexiones	94
Etapa 9: mejora visual con Bootstrap	95
Objetivo	95
Qué se hizo	95
Decisiones y alternativas descartadas.....	101
Incidencias y solución	102
Resultado.....	102
Reflexiones	102
Etapa 10: preparación para el despliegue en Render	103
Objetivo	103
Qué se hizo	103
Comandos.....	110
Decisiones y alternativas descartadas.....	110
Incidencias y solución	111
Limitaciones conocidas.....	111
Reflexiones	111
Etapa 11: revisión final y publicación	112
Objetivo	112
Qué se hizo	112
Resultado.....	112
5. Pruebas	113
Subetapas de la etapa de pruebas	113
6. Reflexiones	114
Migraciones (Etapa 3).....	114

ORM y consultas personalizadas (Etapa 4)	114
7. Conclusiones y próximos pasos	115
Conclusiones.....	115
Próximos pasos.....	115
Autor.....	115
8. Anexo: índice de capturas	116

Después de pegar las capturas, haz clic derecho sobre el índice y elige «Actualizar campo» (o selecciona todo y pulsa F9) para renovar los números de página.

1. Resumen del proyecto

Billetera digital desarrollada con Django para el proyecto final del **Módulo 7: Acceso a datos en aplicaciones Django** (Full Stack Python Trainee, Talento Digital / Alkemy).

Alke Wallet permite crear y gestionar cuentas digitales, registrar depósitos, retiros y transferencias, guardar contactos frecuentes, consultar saldos y generar reportes. Usa el ORM de Django, migraciones y las aplicaciones preinstaladas del framework, y funciona con SQLite en desarrollo y PostgreSQL en producción.

La aplicación está publicada en Render, con la base de datos PostgreSQL en Neon. El enlace recomendado es <https://javiermll.github.io/alke-wallet-django/> (la aplicación corre en <https://alke-wallet.onrender.com>)

Demostración en línea

<https://javiermll.github.io/alke-wallet-django/> la aplicación corre en Render, plan gratuito, con la base de datos en Neon, en <https://alke-wallet.onrender.com>). Este es el enlace recomendado: abre una página de espera con la marca del proyecto y entra sola a la aplicación cuando el servidor despierta.

- **Primera visita:** el servidor gratuito se duerme tras 15 minutos sin visitas, así que la primera carga puede tardar cerca de un minuto. Con el enlace de arriba, una página de espera lo explica y redirige sola. Si se escribe directamente la dirección de Render, es Render quien muestra su propia pantalla de arranque, que no se puede personalizar en el plan gratuito. Después de despertar, la aplicación responde con normalidad, y al iniciar sesión o registrarse una ventana de espera explica cualquier demora.
- **Cómo probarla:** lo más directo es crear una cuenta propia desde "Crear cuenta": el registro es público, toma menos de un minuto y abre una primera cuenta con saldo cero. También existen usuarios de demostración (ana, luis, carla, diego y marta) con datos de ejemplo; su clave se define con la variable CLAVE_DEMO en el panel de Render y no está escrita en el repositorio, por lo que se entrega por separado a quien la necesite.
- **Panel de administración** (/admin/): privado, con clave solo del autor.

Funciones

- **Clientes y cuentas:** alta, ficha, edición y eliminación con vistas basadas en clases. Cada cuenta muestra su saldo, calculado desde los movimientos.
- **Movimientos:** depósitos, retiros y transferencias con validación de saldo, registro atómico y listado con filtros y paginación.
- **Contactos:** agenda de contactos por cliente, con atajo para transferir.
- **Reporte general:** movimientos por tipo, saldos por cliente y movimientos por mes, separados por moneda.
- **Roles y seguridad:** todo es privado salvo el login y el registro. El personal administra todo; cada cliente ve y toca solo lo suyo. Los formularios exigen token CSRF.
- **Registro y perfil:** una persona crea su usuario, su ficha y su primera cuenta, y puede cambiar su contraseña.
- **Panel de administración** de Django con columnas, búsqueda y filtros.

- **Interfaz moderna y adaptable:** Bootstrap 5, portada de presentación con el inicio de sesión y el registro en la misma pantalla, y diseño pensado para escritorio, tablet y celular.
- **Páginas de error propias** (403, 404 y 500) y una **ventana de espera** al iniciar sesión o registrarse, que explica la demora cuando el servidor gratuito estaba dormido.
- **Publicada en Render** con PostgreSQL en Neon: configuración por variables de entorno, archivos estáticos con WhiteNoise, HTTPS y script de construcción. Demostración en línea: <https://javiermll.github.io/alke-wallet-django/>.
- **245 pruebas automatizadas** y un informe de pruebas.

Tecnologías

Herramienta	Versión	Uso
Python	3.14.6	Lenguaje base
Django	6.1.1	Framework web
psycopg (psycopg[binary])	3.3.6	Adaptador entre Django y PostgreSQL (reemplaza a psycopg2-binary; ver incidencia de la etapa 4). La variante [binary] evita compilarlo al desplegar
python-dotenv	1.2.3	Lectura de credenciales desde el archivo .env
PostgreSQL	17	Base de datos de producción
SQLite	incluido en Python	Base de datos de desarrollo
Bootstrap	5.3.3 (CDN)	Sistema de diseño de la interfaz: rejilla, tarjetas, tablas, formularios y menú
Bootstrap Icons	1.11.3 (CDN)	Íconos de la interfaz
Inter (Google Fonts)	-	Tipografía
gunicorn	26.2.0	Servidor que ejecuta la aplicación en producción (solo en Linux; no corre en Windows)
WhiteNoise	6.12.0	Sirve los archivos estáticos desde la propia aplicación
dj-database-url	3.1.2	Convierte la variable DATABASE_URL en la configuración de base de datos de Django
Render	plan gratuito	Alojamiento de la aplicación en línea
Neon	plan gratuito	PostgreSQL en la nube para producción
Github Pages	Gratuito	Uso: Aloja la página de espera (docs/index.html) que despierta la demostración
Visual Studio Code	última versión	Editor
Git y GitHub	-	Control de versiones

Instalación y ejecución local

Los comandos están escritos para **PowerShell en Windows**.

1. Clonar el repositorio

```
# Descarga el proyecto y entra a su carpeta
git clone https://github.com/Javiermll/alke-wallet-django.git
cd alke-wallet-django
```

2. Crear y activar el entorno virtual

```
# Crea el entorno virtual en la carpeta "venv"
python -m venv venv

# Lo activa; el prompt debe empezar con (venv)
.\venv\Scripts\activate
```

Cada terminal nueva necesita la activación antes de trabajar. Sin `(venv)` en el prompt, Python usa la instalación global y Django no se encuentra.

3. Instalar las dependencias

```
# Instala las librerías listadas en requirements.txt
python -m pip install -r requirements.txt
```

4. Configurar las variables de entorno

Copiar la plantilla `.env.example` como `.env` y completar los valores:

```
# Crea el archivo .env a partir de la plantilla
Copy-Item .env.example .env
```

Contenido del `.env`:

```
# Base de datos: sqlite (desarrollo y pruebas) o postgres (producción)
DB_ENGINE=sqlite

# Datos de conexión a PostgreSQL (solo se usan si DB_ENGINE=postgres)
DB_NAME=alke_wallet
DB_USER=postgres
DB_PASSWORD=escribe_aqui_tu_contraseña
DB_HOST=localhost
DB_PORT=5432

# Clave para los usuarios de demostración (opcional; si falta, poblar_datos
genera una al azar)
CLAVE_DEMO=
```

Para usar PostgreSQL, crear antes la base de datos vacía (`CREATE DATABASE alke_wallet;`) y cambiar `DB_ENGINE=postgres`.

En producción (Render) no se usa este archivo: las variables `DEBUG`, `SECRET_KEY`, `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`, `DATABASE_URL` y `DJANGO_SUPERUSER_*` se cargan en el panel del servicio. Están descritas en la etapa 10. Si no existen, el proyecto usa los valores de desarrollo y funciona igual en local.

5. Aplicar las migraciones

```
# Crea todas las tablas en la base configurada (las internas de Django y las de
gestion)
python manage.py migrate
```

6. Cargar datos de demostración (opcional)

```
# Carga clientes, cuentas, contactos y movimientos de ejemplo (se puede repetir
sin duplicar)
python manage.py poblar_datos
```

```
# Ejecuta todas las consultas del proyecto y muestra el resultado
python manage.py demo_consultas
```

7. Crear un superusuario (para el panel de administración)

```
# Crea la cuenta con la que se entra al panel (pide usuario, correo y contraseña)
python manage.py createsuperuser
```

8. Ejecutar el servidor

```
# Levanta el servidor de desarrollo en http://127.0.0.1:8000/
python manage.py runserver
```

Las pantallas de la aplicación están en `http://127.0.0.1:8000/` (inicio), `/clientes/`, `/cuentas/`, `/transacciones/`, `/reporte/` y `/perfil/`. El panel de administración está en `http://127.0.0.1:8000/admin/`.

Todas las pantallas piden iniciar sesión, salvo el login (`/acceso/login/`) y el registro (`/registro/`). Para probar los distintos roles:

Rol	Con qué usuario	Qué ve
Personal	El superusuario del paso 7	Todas las pantallas, el reporte y el panel de administración
Cliente	ana, luis, carla, diego o marta (los crea <code>poblar_datos</code>)	Solo sus propias cuentas, movimientos y agenda
Sin cliente	Un usuario creado en el panel sin ficha de cliente	Solo el perfil y un aviso en el inicio

Al crear los usuarios de ejemplo, `poblar_datos` genera una clave aleatoria y la muestra una sola vez en pantalla. Si se prefiere una clave propia para la demostración, se define en el archivo `.env` con la variable `CLAVE_DEMO` (ese archivo no se sube al repositorio). El código no contiene ninguna clave escrita. En la demostración en línea la clave se define con la variable `CLAVE_DEMO` en el panel de Render.

Con `DEBUG=True`, las tres páginas de error se pueden previsualizar en `http://127.0.0.1:8000/vista-403/`, `/vista-404/` y `/vista-500/`. Esas rutas solo existen en desarrollo; con `DEBUG=False` Django usa directamente `403.html`, `404.html` y `500.html`.

Para verificar los accesos y la protección CSRF en cualquier momento:

```
# Prueba roles, alcance por usuario y CSRF con datos temporales (no deja nada en la base de datos)
python manage.py verificar_accesos
```

2. Cumplimiento de la consigna

Requisito de la consigna	Dónde se cumple (etapa)	Estado
CRUD con vistas basadas en clases	Etapa 6	Completada
Protección CSRF en los formularios	Etapas 6 y 7; comprobada en las pruebas de seguridad	Completada
Modelos con relaciones 1:1, N:1 y N:M	Modelo de datos y Etapa 2	Completada
Migraciones	Etapa 3	Completada
Consultas personalizadas (<code>filter</code> , <code>annotate</code> , <code>raw()</code> , cursores)	Etapa 4	Completada
SQLite en desarrollo y PostgreSQL en producción	Etapa 1 y Etapa 3	Completada
Ramas de Git por funcionalidad	Flujo de Git	Completada (ramas fusionadas con main)
Pruebas e informe de pruebas	Pruebas y docs/informe_pruebas.docx	Completada
Diseño responsive con Bootstrap	Etapa 9	Completada
Preparación para publicar en Render y Neon	Etapa 10	Completada
README	Este documento	Compleda

Estado por etapa

Tabla «Estado del proyecto» tal como figura en el README.

Etapa	Descripción	Estado
0	Entorno virtual, instalación de Django y repositorio Git	Completada
1	Conexión a base de datos (SQLite y PostgreSQL)	Completada
2	App <code>gestion</code> y modelos	Completada
3	Migraciones en SQLite y PostgreSQL	Completada
4	Consultas personalizadas (<code>filter</code> , <code>exclude</code> , <code>annotate</code> , <code>raw()</code> , cursores)	Completada
5	Panel de administración	Completada
6	Vistas CRUD basadas en clases y templates	Completada
7	Autenticación, archivos estáticos, roles, alcance por usuario, registro y perfil	Completada
8	Pruebas automatizadas (239 pruebas en esa etapa; hoy son 245) e informe de pruebas	Completada
9	Mejora visual con Bootstrap: portada, diseño responsive, componentes y páginas de error	Completada
10	Preparación y despliegue en Render y Neon	Completada
11	Revisión final, README final y publicación	Completada

3. Arquitectura y modelo de datos

Estructura de carpetas

```

alke_wallet/
├── core/                                # Paquete de configuración del proyecto
│   ├── settings.py                    # Configuración general; en producción lee su
valor desde variables de entorno
│   ├── urls.py                       # Rutas principales (login, registro, salud y
vistas previas de errores en desarrollo)
│   ├── formats/es_CL/                # Formato de números chileno: punto como separador
de miles y coma decimal
│   ├── asgi.py
│   └── wsgi.py
├── gestion/                            # App principal con la lógica del negocio
│   ├── management/
│   │   └── commands/
│   │       ├── poblar_datos.py        # Carga datos de demostración sin duplicar
│   │       ├── demo_consultas.py     # Ejecuta y muestra todas las consultas
│   │       └── verificar_accesos.py   # Prueba roles, alcance por usuario y CSRF
(no deja datos)
│   │   └── crear_superuser.py        # Crea el administrador desde variables de
entorno (despliegue)
│   ├── migrations/
│   │   ├── 0001_initial.py           # Migración inicial: crea las 5 tablas de la app
│   │   └── 0002_alter_cliente_options_alter_contacto_options_and_more.py #
Nombres legibles (sin cambios en las tablas)
│   ├── templatetags/
│   │   └── ui.py                     # Filtros que aplican las clases de Bootstrap a
los campos de un formulario
│   ├── models.py                    # Modelos: Moneda, Cliente, Contacto, Cuenta,
Transaccion
│   ├── consultas.py                 # Consultas reutilizables: ORM, raw() y cursor (12
funciones)
│   ├── servicios.py                 # registrar_transaccion() atómica con bloqueo;
crear_cliente_con_cuenta() para el registro
│   ├── mixins.py                    # Roles: SoloPersonalMixin y PersonalOClienteMixin
│   ├── alcance.py                   # Filtros por dueño: cada cliente ve solo lo suyo
│   └── forms.py                      # Formularios de clientes, cuentas, transacciones,
contactos y registro
│   ├── admin.py                     # Panel de administración: columnas, búsqueda,
filtros y tablas anidadas
│   ├── views.py                     # Vistas basadas en clases: CRUD, inicio, reporte,
registro, perfil y salud
│   ├── urls.py                      # Rutas de la app, con nombre
│   └── tests/                        # Pruebas automatizadas, un archivo por bloque
(245 en total)
├── templates/
│   ├── base.html                    # Plantilla común: menú, mensajes y pie de página
│   └── inicio.html                  # Página de inicio (distinta para personal y
clientes)
│   ├── reporte.html                 # Reporte general con barras de proporción
│   └── perfil.html                  # Datos de acceso y de cliente de la persona con
sesión
│   ├── 403.html, 404.html           # Páginas de error que heredan de base.html
│   └── 500.html                     # Error del servidor: documento independiente, sin
depender de la sesión ni del menú

```

```

|   ├── componentes/           # Piezas reutilizables: campos de formulario,
etiquetas, avisos y lista de movimientos para celular
|   ├── registration/          # landing (portada), login, registro y
cambiar_clave
|   ├── clientes/              # lista, detalle, formulario y confirmar_eliminar
|   ├── cuentas/               # lista, detalle, formulario y confirmar_eliminar
|   ├── transacciones/         # lista, detalle y formulario
|   ├── contactos/             # formulario y confirmar_eliminar
|   ├── static/
|   │   ├── css/estilos.css    # Capa de marca sobre Bootstrap: colores,
tarjetas, portada, pie y animaciones
|   │   ├── js/app.js          # Ver contraseña, evitar doble envío, avisos que
se cierran y ventana de espera
|   │   └── img/logo.svg      # Logo
|   ├── docs/
|   │   ├── capturas/          # Capturas de pantalla usadas en este README
|   │   └── vista_previa/      # Capturas del nuevo diseño en escritorio, tablet
y celular
|   └── informe_pruebas.docx   # Informe de pruebas con casos, resultados y
evidencias
|   ├── build.sh               # Script de construcción para Render (instala,
migra, crea el administrador y carga datos)
|   ├── render.yaml            # Plano del servicio en Render
|   ├── .gitattributes         # Fuerza saltos de línea LF en los scripts (Linux
falla con los de Windows)
|   ├── manage.py              # Utilidad de línea de comandos de Django
|   ├── requirements.txt        # Dependencias del proyecto
|   ├── .env                   # Credenciales locales (NO se sube a GitHub)
|   ├── .env.example           # Plantilla pública del .env
|   └── .gitignore              # Archivos excluidos del repositorio

```

Modelo de datos

Diagrama entidad-relación del README (Mermaid), transcrito como tabla de relaciones:

Entidad A	Cardinalidad en A	Cardinalidad en B	Entidad B	Relación
USER	exactamente 1	exactamente 1	CLIENTE	1:1
CLIENTE	exactamente 1	0 o muchos	CUENTA	tiene
MONEDA	exactamente 1	0 o muchos	CUENTA	moneda de
CUENTA	0 o 1	0 o muchos	TRANSACCION	origen
CUENTA	0 o 1	0 o muchos	TRANSACCION	destino
CLIENTE	exactamente 1	0 o muchos	CONTACTO	propietario
CLIENTE	exactamente 1	0 o muchos	CONTACTO	agendado

Modelo	Tabla	Descripción	Relaciones
Moneda	gestion_moneda	Monedas disponibles (código, nombre, símbolo)	1 moneda → N cuentas
Cliente	gestion_cliente	Datos de negocio de cada persona	1:1 con User de Django
Contacto	gestion_contacto	Ficha de agenda: quién guarda a quién, con apodo y fecha	Tabla intermedia del N:M Cliente ↔ Cliente
Cuenta	gestion_cuenta	Cuenta digital de un cliente en una moneda	N:1 con Cliente y N:1 con Moneda

Modelo	Tabla	Descripción	Relaciones
Transaccion	gestion_transaccion	Depósito, retiro o transferencia	N:1 con Cuenta origen y N:1 con Cuenta destino

Reglas de negocio

Tipo	Cuenta origen	Cuenta destino
Depósito	vacía	obligatoria
Retiro	obligatoria	vacía
Transferencia	obligatoria	obligatoria, distinta de la origen y de la misma moneda

Otras reglas: el monto debe ser mayor que cero, un cliente no puede agendarse a sí mismo y no se puede agendar dos veces a la misma persona.

Flujo de Git

Rama	Propósito	Estado
main	Código estable	Activa
feature/modelos	Definición de modelos y migraciones	Fusionada con main (fast-forward)
feature/consultas	Datos de demostración y consultas personalizadas	Fusionada con main
feature/admin	Idioma, nombres legibles y panel de administración	Fusionada con main
feature/crud	Vistas, formularios, servicio de movimientos, contactos y reporte	Fusionada con main
feature/auth	Login, estáticos, roles, alcance por usuario, registro, perfil y verificación de accesos	Fusionada con main
feature/tests	Pruebas automatizadas (modelos, servicios, consultas, formularios, vistas y seguridad) e informe	Fusionada con main
fix/clave-demo	Clave de los usuarios de demostración al azar o desde .env	Fusionada con main
feature/bootstrap	Rediseño con Bootstrap: portada, componentes, diseño responsive, páginas de error y formato de montos	Fusionada con main (7 commits por tema, fast-forward)
feature/espera	Endpoint /salud/ y ventana de espera al iniciar sesión o registrarse	Fusionada con main
feature/render	Configuración de producción, crear_superusuario, build.sh, render.yaml y actualización del README	Fusionada con main
feature/puente	Página de espera en GitHub Pages y permiso CORS en /salud/	Fusionada con main

4. Desarrollo por etapas

Cada etapa del README se presenta con los mismos apartados. Los comandos de consola se reúnen en «Comandos» y las incidencias de cada apartado en «Incidencias y solución», con su origen indicado. Las reflexiones de las etapas 3 y 4 están en la sección 6.

Etapa 0: entorno y proyecto base

Objetivo

Dejar un entorno aislado y reproducible con el proyecto Django funcionando y bajo control de versiones.

Qué se hizo

Pasos realizados:

Comandos: ver «Comandos», apartado «Pasos realizados».

Comandos

Pasos realizados

```
# 1. Crear la carpeta raíz y abrirla en VS Code
mkdir alke_wallet
cd alke_wallet
code .

# 2. Crear y activar el entorno virtual
python -m venv venv
.\venv\Scripts\activate

# 3. Instalar las librerías
python -m pip install django psycopg2-binary python-dotenv

# 4. Crear el proyecto con el paquete interno "core" (el punto evita una carpeta
anidada)
python -m django startproject core .

# 5. Guardar la lista de librerías (utf8 evita que PowerShell genere UTF-16)
python -m pip freeze | Out-File -Encoding utf8 requirements.txt

# 6. Probar el servidor
python manage.py runserver

# 7. Iniciar el repositorio y guardar el primer commit
git init
git branch -M main
git add .
git commit -m "Configuración inicial del proyecto Django"
```

Decisiones y alternativas descartadas

- **Nombre core:** el paquete interno se llama `core` para evitar la estructura `alke_wallet/alke_wallet/`, que resulta confusa.
- **.gitignore:** excluye `venv/`, `__pycache__/`, `db.sqlite3`, `.env` y `.vscode/`. El entorno virtual y las credenciales nunca se versionan.
- **requirements.txt:** permite reconstruir el entorno en otro equipo con un solo comando.
- **Adaptador de PostgreSQL:** al inicio se instaló `psycopg2-binary`. En la etapa 4 se reemplazó por `psycopg` (ver la incidencia de esa etapa).

Incidencias y solución

Incidencia resuelta: en este equipo, una política de control de aplicaciones de Windows bloquea `pip.exe` dentro del entorno virtual.

Error al ejecutar el programa '`pip.exe`': Una directiva de Control de aplicaciones bloqueó este archivo

Solución: ejecutar `pip` a través del intérprete de Python, que sí está permitido.

```
# En vez de: pip install ...
python -m pip install django psycopg2-binary python-dotenv

# En vez de: django-admin startproject ...
python -m django startproject core .
```

Resultado

Verificación: al abrir `http://127.0.0.1:8000/` aparece la página de bienvenida de Django (el cohete).

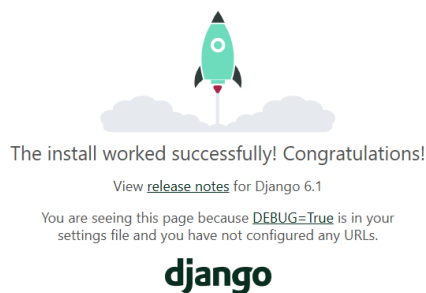


Figura 1. Servidor de Django funcionando

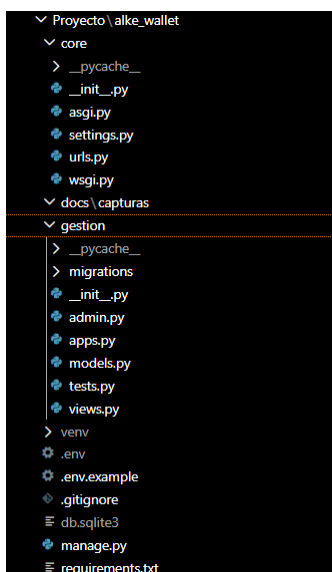


Figura 2. Estructura del proyecto en VS Code

Etapa 1: conexión a la base de datos

Objetivo

Configurar Django para conectarse a **SQLite** (desarrollo) y **PostgreSQL** (producción), alternando entre ambas sin modificar el código.

Qué se hizo

Creación de la base de datos en PostgreSQL:

Comandos: ver «Comandos», apartado «Creación de la base de datos en PostgreSQL».

Comandos: ver «Comandos», apartado «Creación de la base de datos en PostgreSQL».

Configuración en `core/settings.py`:

```
# "os" permite leer variables de entorno del sistema
import os

# load_dotenv lee el archivo .env y carga sus valores como variables de entorno
from dotenv import load_dotenv

# Carga las variables del archivo .env que está en la raíz del proyecto
load_dotenv(BASE_DIR / '.env')

# Lee del .env qué motor usar; si la variable no existe, usa "sqlite" por defecto
DB_ENGINE = os.getenv('DB_ENGINE', 'sqlite')

# Si en el .env se indicó "postgres", se configura la conexión a PostgreSQL
if DB_ENGINE == 'postgres':
    DATABASES = {
        'default': {
            # Adaptador de PostgreSQL (psycopg)
            'ENGINE': 'django.db.backends.postgresql',
            # Nombre de la base de datos, usuario y contraseña
            'NAME': os.getenv('DB_NAME'),
            'USER': os.getenv('DB_USER'),
            'PASSWORD': os.getenv('DB_PASSWORD'),
            # Dónde corre PostgreSQL y en qué puerto
            'HOST': os.getenv('DB_HOST', 'localhost'),
            'PORT': os.getenv('DB_PORT', '5432'),
        }
    }
# En cualquier otro caso se usa SQLite (archivo db.sqlite3 en la raíz)
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
```

Seguridad: las credenciales viven únicamente en el `.env`, que está en el `.gitignore`. El repositorio incluye `.env.example`, una plantilla sin la contraseña real.

Comandos

Creación de la base de datos en PostgreSQL

```
# Entra a PostgreSQL con el usuario postgres
psql -U postgres
```

Creación de la base de datos en PostgreSQL

```
-- Crea la base de datos vacía del proyecto
CREATE DATABASE alke_wallet;

-- Sale de psql
\q
```

Decisiones y alternativas descartadas

Opción	Descripción	Resultado
A. Variable DB_ENGINE en .env	Un solo settings.py; se cambia una línea del .env	Elegida: cumple la consigna y demuestra ambas conexiones
B. Dos archivos de settings	settings_dev.py y settings_prod.py	Descartada: agrega complejidad innecesaria para este alcance
C. Solo SQLite con PostgreSQL comentado	Mínimo exigido	Descartada: la conexión a PostgreSQL quedaría sin demostrar

Incidencias y solución

Error	Causa probable
No module named 'django'	El entorno virtual no está activo
password authentication failed	La contraseña del .env no coincide con la de PostgreSQL
database "alke_wallet" does not exist	Falta crear la base de datos
connection refused	El servicio de PostgreSQL no está corriendo

Resultado

```
# Lista las migraciones; si la conexión funciona, Django las muestra todas sin
aplicar ([ ])
python manage.py showmigrations

# Confirma qué motor y qué base está usando Django en este momento
python manage.py shell -c "from django.db import connection;
print(connection.vendor, connection.settings_dict['NAME'])"
```

Resultados obtenidos:

DB_ENGINE	Salida del comando de verificación
sqlite	sqlite <ruta>\db.sqlite3
postgres	postgresql alke_wallet

En ambos casos `showmigrations` lista las 18 migraciones internas de Django (`admin`, `auth`, `contenttypes` y `sessions`) sin aplicar. Es lo esperado: se aplican en la etapa 3.

```

82 # Lee del .env qué motor usar; si la variable no existe, usa "sqlite" por defecto
83 DB_ENGINE = os.getenv('DB_ENGINE', 'sqlite')
84
85 # Si en el .env pusimos "postgres", configuramos la conexión a PostgreSQL
86 if DB_ENGINE == 'postgres':
87     DATABASES = {
88         'default': {
89             # Le dice a Django que use el adaptador de PostgreSQL (psycopg2)
90             'ENGINE': 'django.db.backends.postgresql',
91             # Nombre de la base de datos que creamos en el paso 1
92             'NAME': os.getenv('DB_NAME'),
93             # Usuario de PostgreSQL
94             'USER': os.getenv('DB_USER'),
95             # Contraseña de ese usuario
96             'PASSWORD': os.getenv('DB_PASSWORD'),
97             # Dónde corre PostgreSQL (localhost = tu propio equipo)
98             'HOST': os.getenv('DB_HOST', 'localhost'),
99             # Puerto de PostgreSQL (5432 es el estándar)
100             'PORT': os.getenv('DB_PORT', '5432'),
101         }
102     }
103 # En cualquier otro caso, usamos SQLite (un archivo db.sqlite3 en la raíz)
104 else:
105     DATABASES = {
106         'default': {
107             # Le dice a Django que use el adaptador de SQLite (viene incluido)
108             'ENGINE': 'django.db.backends.sqlite3',
109             # Ruta del archivo donde SQLite guarda los datos
110             'NAME': BASE_DIR / 'db.sqlite3',
111         }
112     }
113

```

Figura 3. Bloque DATABASES en settings.py

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Muestra qué motor y qué base está usando Django en este momento
>> python manage.py shell -c "from django.db import connection; print(connection.vendor, connection.settings_dict['NAME'])"
12 objects imported automatically (use -v 2 for details).

sqlite D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet\db.sqlite3

```

Figura 4. Verificación con SQLite

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Muestra qué motor y qué base está usando Django en este momento
>> python manage.py shell -c "from django.db import connection; print(connection.vendor, connection.settings_dict['NAME'])"
12 objects imported automatically (use -v 2 for details).

postgresql alke_wallet

```

Figura 5. Verificación con PostgreSQL

Etapa 2: app gestion y modelos

Objetivo

Crear la app principal y definir el modelo de datos con el ORM de Django, con campos adecuados, validaciones y relaciones Uno a Uno, Muchos a Uno y Muchos a Muchos.

Qué se hizo

Pasos realizados:

Comandos: ver «Comandos», apartado «Pasos realizados».

Registro de la app en `core/settings.py`:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Nuestra app; sin esta línea Django no reconoce sus modelos
    'gestion',
]
```

Fragmentos clave de `gestion/models.py` (el archivo completo está en el repositorio):

Relación N:M de Cliente consigo mismo, a través de la tabla intermedia `Contacto`:

```
# 'self' significa "el mismo modelo"
# through='Contacto': la tabla intermedia la definimos nosotros
# through_fields indica cuál campo de Contacto es "quien agenda" y cuál "a quién agenda"
# symmetrical=False: si Ana agenda a Luis, Luis NO queda agendado por Ana automáticamente
contactos = models.ManyToManyField(
    'self',
    through='Contacto',
    through_fields=('propietario', 'agendado'),
    symmetrical=False,
    related_name='agendado_por',
)
```

Restricciones que la propia base de datos hace cumplir en `Contacto`:

```
constraints = [
    # No se puede agendar dos veces a la misma persona
    models.UniqueConstraint(fields=['propietario', 'agendado'],
    name='contacto_unico'),
    # Nadie puede agendarse a sí mismo
    models.CheckConstraint(
        condition=~models.Q(propietario=models.F('agendado')),
        name='no_agendarse_a_si_mismo',
    ),
]
```

Saldo calculado en `Cuenta`, sin columna en la tabla:

```
# @property permite usarlo como un dato más: cuenta.saldo (sin paréntesis)
@property
```

```
def saldo(self):
    # Suma todo el dinero que ha llegado a esta cuenta (None si no hay
    movimientos)
    entradas =
self.transacciones_recibidas.aggregate(total=Sum('monto'))['total'] or
Decimal('0.00')
    # Suma todo el dinero que ha salido de esta cuenta
    salidas = self.transacciones_enviadas.aggregate(total=Sum('monto'))['total']
    or Decimal('0.00')
    # El saldo es lo que entró menos lo que salió
    return entradas - salidas
```

Doble protección de datos:

Nivel	Herramienta	Dónde actúa
Python	MinValueValidator y clean()	Formularios y panel de administración, con mensajes claros
Base de datos	UniqueConstraint y CheckConstraint	Toda escritura, incluso desde la shell

Notas técnicas:

- on_delete=PROTECT en Cuenta.moneda y en las cuentas de Transaccion impide borrar monedas o cuentas que tengan datos asociados. CASCADE se usa donde los datos dependientes no tienen sentido sin su padre (cuentas de un cliente, fichas de contacto).
- Django 6 usa condition= en CheckConstraint. Los ejemplos antiguos usan check=, que ya no existe.
- La regla "no retirar más de lo que hay" no vive en clean(). Se aplicará en las vistas, dentro de una operación atómica, para no rechazar la edición de movimientos ya registrados.

Comandos

Pasos realizados

```
# Crea la rama de trabajo que pide la consigna y se cambia a ella
git checkout -b feature/modelos

# Crea la app "gestion"
python manage.py startapp gestion
```

Decisiones y alternativas descartadas

Tema	Elegida	Alternativa descartada	Motivo
Relación N:M	Contactos frecuentes entre clientes (Cliente ↔ Cliente)	Categorías de transacción	Se acerca más al funcionamiento de una billetera real
Contactos	Modelo intermedio Contacto con apodo y fecha	ManyToManyField simple	Permite guardar datos propios de la relación
Saldo	Calculado sumando transacciones	Columna saldo guardada y actualizada	Nunca queda desincronizado con los movimientos
Monedas	Transferencias solo entre cuentas de la misma moneda	Conversión con tasas de cambio	La conversión queda fuera del alcance

Resultado

```
# Revisa que los modelos no tengan errores (no toca la base de datos)
python manage.py check
```

Resultado: System check identified no issues (0 silenced).

```
35 INSTALLED_APPS = [
36     'django.contrib.admin',
37     'django.contrib.auth',
38     'django.contrib.contenttypes',
39     'django.contrib.sessions',
40     'django.contrib.messages',
41     'django.contrib.staticfiles',
42     'gestion', # Nuestra app; sin esta línea Django no reconoce sus modelos
43 ]
```

Figura 6. INSTALLED_APPS con la app gestion

```
29 # -----
30 # CLIENTE: los datos de negocio de cada persona
31 # -----
32 class Cliente(models.Model):
33     # Relación 1:1 con User: un usuario de login tiene un solo cliente, y al revés
34     # on_delete=CASCADE: si se borra el usuario, se borra también su cliente
35     # related_name='cliente' permite escribir usuario.cliente para llegar a él
36     usuario = models.OneToOneField(User, on_delete=models.CASCADE, related_name='cliente')
37     nombre = models.CharField(max_length=100) # Nombre de hasta 100 caracteres
38     email = models.EmailField(unique=True) # Correo con formato validado; unique=True impide dos clientes con el mismo email
39     telefono = models.CharField(max_length=20, blank=True, null=True) # Teléfono opcional: blank=True permite dejarlo vacío en formularios
40     fecha_registro = models.DateTimeField(auto_now_add=True) # Fecha y hora que se llenan solas cuando se crea el cliente
41
42     # Relación N:M de Cliente con Cliente, pasando por la ficha "Contacto"
43     # 'self' significa "el mismo modelo"
44     # through='Contacto' le dice a Django que la tabla intermedia la definimos nosotros
45     # through_fields indica cuál campo de Contacto es "quien agenda" y cuál "a quién agenda"
46     # symmetrical=False: si Ana agenda a Luis, Luis NO queda agendado por Ana automáticamente
47     # related_name='agendado_por' permite preguntar quién tiene agendado a un cliente
48     contactos = models.ManyToManyField(
49         'self',
50         through='Contacto',
51         through_fields=('propietario', 'agendado'),
52         symmetrical=False,
53         related_name='agendado_por',
54     )
55
56     # Texto que se muestra cuando Django imprime un cliente (admin, shell, listas)
57     def __str__(self):
58         return self.nombre
59
```

Figura 7. Archivo models.py en VS Code

Etapa 3: migraciones

Objetivo

Aplicar las migraciones para reflejar los modelos en el esquema de la base de datos de forma controlada y con historial versionado, en SQLite y en PostgreSQL.

Idea central: `makemigrations` escribe el plan (un archivo Python que describe qué crear) y `migrate` lo ejecuta sobre la base de datos. Cada cambio futuro en los modelos generará una migración nueva en vez de modificar la anterior.

Qué se hizo

Pasos realizados:

Comandos: ver «Comandos», apartado «Pasos realizados».

Resultado de `makemigrations`: se creó `gestion/migrations/0001_initial.py` con:

```
+ Create model Moneda
+ Create model Cliente
+ Create model Contacto
+ Add field contactos to cliente
+ Create model Cuenta
+ Create model Transaccion
+ Create constraint contacto_unico on model contacto
+ Create constraint no_agendarse_a_si_mismo on model contacto
+ Create constraint monto_positivo on model transaccion
```

Resultado de `migrate`: se aplicaron 19 migraciones: las 18 internas de Django (`admin`, `auth`, `contenttypes`, `sessions`) más `gestion.0001_initial`.

Tablas creadas (idénticas en ambos motores):

Origen	Tablas
App <code>gestion</code>	<code>gestion_moneda</code> , <code>gestion_cliente</code> , <code>gestion_contacto</code> , <code>gestion_cuenta</code> , <code>gestion_transaccion</code>
Autenticación (<code>auth</code>)	<code>auth_user</code> , <code>auth_group</code> , <code>auth_permission</code> , <code>auth_group_permissions</code> , <code>auth_user_groups</code> , <code>auth_user_user_permissions</code>
Administración y sesiones	<code>django_admin_log</code> , <code>django_content_type</code> , <code>django_session</code>
Control de migraciones	<code>django_migrations</code>

En PostgreSQL, la verificación se repitió con `psql`:

Comandos: ver «Comandos», apartado «Tablas creadas (idénticas en ambos motores)».

Resultado: 15 tablas, todas en el esquema `public`.

Prueba funcional en la shell (`python manage.py shell`), repetida en ambos motores:

```
# Importa el usuario de login y nuestros modelos
from django.contrib.auth.models import User
from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion

# Crea la moneda
clp = Moneda.objects.create(codigo='CLP', nombre='Peso chileno', simbolo='$')

# Crea dos usuarios de login; create_user guarda la contraseña cifrada
```

```

u1 = User.objects.create_user(username='ana', password='<contraseña_de_prueba>')
u2 = User.objects.create_user(username='luis', password='<contraseña_de_prueba>')

# Crea un cliente por cada usuario (relación 1:1)
ana = Cliente.objects.create(usuario=u1, nombre='Ana García',
email='ana@example.com', telefono='123456789')
luis = Cliente.objects.create(usuario=u2, nombre='Luis Pérez',
email='luis@example.com')

# Crea una cuenta para cada cliente, ambas en pesos
c_ana = Cuenta.objects.create(cliente=ana, moneda=clp, numero='0001')
c_luis = Cuenta.objects.create(cliente=luis, moneda=clp, numero='0002')

# Un depósito de 100000 en la cuenta de Ana (solo tiene destino)
Transaccion.objects.create(tipo='deposito', cuenta_destino=c_ana, monto=100000)

# Una transferencia de 30000 de Ana a Luis
Transaccion.objects.create(tipo='transferencia', cuenta_origen=c_ana,
cuenta_destino=c_luis, monto=30000)

# Ana agenda a Luis como contacto (se crea la ficha intermedia)
Contacto.objects.create(proprietario=ana, agendado=luis, apodo='Lucho')

# Saldos calculados, contactos de Ana y agendas donde aparece Luis
print(c_ana.saldo, c_luis.saldo)
print(ana.contactos.all())
print(luis.agendado_por.all())

# La base de datos debe rechazar un monto 0 (restricción monto_positivo)
from django.db import IntegrityError
try:
    Transaccion.objects.create(tipo='deposito', cuenta_destino=c_ana, monto=0)
except IntegrityError:
    print('La base rechazó el monto 0')

```

Resultados obtenidos:

Comprobación	SQLite	PostgreSQL
Saldo de Ana (100000 – 30000)	70000	70000.00
Saldo de Luis	30000.00	30000.00
Contactos de Ana	Luis Pérez	Luis Pérez
Agendas donde aparece Luis	Ana García	Ana García
Monto 0	Rechazado por la base	Rechazado por la base

Estos datos de prueba se conservan en ambas bases para las consultas de la etapa 4.

Comandos

Pasos realizados

```

# 1. Genera el plan de migración a partir de los modelos
python manage.py makemigrations gestion

# 2. Muestra el SQL exacto que se ejecutará (no modifica la base)
python manage.py sqlmigrate gestion 0001

# 3. Aplica todas las migraciones pendientes

```

```
python manage.py migrate
```

```
# 4. Confirma que todas aparecen aplicadas [X]
```

```
python manage.py showmigrations
```

```
# 5. Lista las tablas creadas en la base que Django está usando
```

```
python manage.py shell -c "from django.db import connection;
print(sorted(connection.introspection.table_names()))"
```

Tablas creadas (idénticas en ambos motores)

```
# Lista las tablas de la base alke_wallet en PostgreSQL
```

```
psql -U postgres -d alke_wallet -c "\dt"
```

Resultado

```
1 #gestion/migrations/0001_initial.py
2 # Generated by Django 6.1.1 on 2026-09-29 16:01
3
4 import django.core.validators
5 import django.db.models.deletion
6 from decimal import Decimal
7 from django.conf import settings
8 from django.db import migrations, models
9
10
11 class Migration(migrations.Migration):
12
13     initial = True
14
15     dependencies = [
16         migrations.swappable_dependency(settings.AUTH_USER_MODEL),
17     ]
18
19     operations = [
20         migrations.CreateModel(
21             name='Moneda',
22             fields=[
23                 ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
24                 ('codigo', models.CharField(max_length=3, unique=True)),
25                 ('nombre', models.CharField(max_length=50)),
26                 ('simbolo', models.CharField(max_length=5)),
27             ],
28         ),
29         migrations.CreateModel(
30             name='Cliente',
31             fields=[
32                 ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
33                 ('nombre', models.CharField(max_length=100)),
34             ],
35         ),
36     ]
```

Figura 8. Migración 0001_initial.py en VS Code

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet>
# Muestra el SQL exacto que Django ejecutará (no cambia nada en la base)
>>> python manage.py sqlmigrate gestion 0001
BEGIN;
--
-- Create model Moneda
--
CREATE TABLE "gestion_moneda" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "codigo" varchar(3) NOT NULL UNIQUE, "nombre" varchar(50) NOT NULL, "simbolo" varchar(5) NOT NULL);
--
-- Create model Cliente
--
CREATE TABLE "gestion_cliente" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "nombre" varchar(100) NOT NULL, "email" varchar(254) NOT NULL UNIQUE, "telefono" varchar(20) NULL, "fecha_registro" datetime NOT NULL, "usuario_id" integer NOT NULL UNIQUE REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED);
--
-- Create model Contacto
--
CREATE TABLE "gestion_contacto" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "apodo" varchar(50) NOT NULL, "fecha_agregado" datetime NOT NULL, "agendado_id" bigint NOT NULL REFERENCES "gestion_cliente" ("id") DEFERRABLE INITIALLY DEFERRED, "propietario_id" bigint NOT NULL REFERENCES "gestion_cliente" ("id") DEFERRABLE INITIALLY DEFERRED);
--
-- Add field contactos to cliente
--
CREATE TABLE "new_gestion_cliente" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "nombre" varchar(100) NOT NULL, "email" varchar(254) NOT NULL UNIQUE, "telefono" varchar(20) NULL, "fecha_registro" datetime NOT NULL, "usuario_id" integer NOT NULL UNIQUE REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED);
INSERT INTO "new_gestion_cliente" ("id", "nombre", "email", "telefono", "fecha_registro", "usuario_id") SELECT "id", "nombre", "email", "telefono", "fecha_registro", "usuario_id" FROM "gestion_cliente";
DROP TABLE "gestion_cliente";
ALTER TABLE "new_gestion_cliente" RENAME TO "gestion_cliente";
```

Figura 9. Salida de sqlmigrate

```
# Ejecuta todas las migraciones pendientes: las 18 internas de Django y la nuestra
>> python manage.py migrate\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet>

Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying gestion.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Figura 10. Migrate y showmigrations en SQLite

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet>
# Lista las tablas que existen en la base que Django está usando
>> python manage.py shell -c "from django.db import connection; print(sorted(connection.intr
ospection.table_names()))"
17 objects imported automatically (use -v 2 for details).

['auth_group', 'auth_group_permissions', 'auth_permission', 'auth_user', 'auth_user_groups',
'auth_user_user_permissions', 'django_admin_log', 'django_content_type', 'django_migrations',
'django_session', 'gestion_cliente', 'gestion_contacto', 'gestion_cuenta', 'gestion_moneda',
'gestion_transaccion']
```

Figura 11. Tablas en SQLite

```
>> python manage.py migrate\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet>
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying gestion.0001_initial... OK
  Applying sessions.0001_initial... OK
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Repite la lista de la etapa 1; ahora todo debe aparecer con [X]
>> python manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
gestion
[X] 0001_initial
sessions
[X] 0001_initial
```

Figura 12. Migrate y showmigrations en PostgreSQL

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proye
>> psql -U postgres -d alke_wallet -c "\dt"
Password for user postgres:

      List of relations
Schema | Name                | Type  | Owner
-----+-----+-----+-----
public | auth_group           | table | postgres
public | auth_group_permissions | table | postgres
public | auth_permission      | table | postgres
public | auth_user            | table | postgres
public | auth_user_groups     | table | postgres
public | auth_user_user_permissions | table | postgres
public | django_admin_log     | table | postgres
public | django_content_type  | table | postgres
public | django_migrations   | table | postgres
public | django_session       | table | postgres
public | gestion_cliente      | table | postgres
public | gestion_contacto     | table | postgres
public | gestion_cuenta       | table | postgres
public | gestion_moneda       | table | postgres
public | gestion_transaccion  | table | postgres
(15 rows)
```

Figura 13. Tablas en PostgreSQL con psql

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Importa el usuario de login y nuestros modelos
>>> from django.contrib.auth.models import User
>>> from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion
>>>
>>> # Crea la moneda
>>> clp = Moneda.objects.create(codigo='CLP', nombre='Peso chileno', simbolo='$')
>>>
>>> # Crea dos usuarios de login; create_user guarda la contraseña cifrada
>>> u1 = User.objects.create_user(username='ana', password='Clave12345!')
>>> u2 = User.objects.create_user(username='luis', password='Clave12345!')
>>>
>>> # Crea un cliente por cada usuario (relación 1:1)
>>> ana = Cliente.objects.create(usuario=u1, nombre='Ana García', email='ana@example.com', telefono='123456789')
>>> luis = Cliente.objects.create(usuario=u2, nombre='Luis Pérez', email='luis@example.com')
>>>
>>> # Crea una cuenta para cada cliente, ambas en pesos
>>> c_ana = Cuenta.objects.create(cliente=ana, moneda=clp, numero='0001')
>>> c_luis = Cuenta.objects.create(cliente=luis, moneda=clp, numero='0002')
>>>
>>> # Un depósito de 100000 en la cuenta de Ana (solo tiene destino)
>>> Transaccion.objects.create(tipo='deposito', cuenta_destino=c_ana, monto=100000)
<Transaccion: Depósito de 100000>
>>>
>>> # Una transferencia de 30000 de Ana a Luis
>>> Transaccion.objects.create(tipo='transferencia', cuenta_origen=c_ana, cuenta_destino=c_luis, monto=30000)
<Transaccion: Transferencia de 30000>
>>>
>>> # Saldos calculados: Ana debe tener 70000 y Luis 30000
>>> print(c_ana.saldo, c_luis.saldo)
70000 30000.00
>>>
>>> # Ana agenda a Luis como contacto (se crea la ficha intermedia)
>>> Contacto.objects.create(propietario=ana, agendado=luis, apodo='Lucho')
<Contacto: Ana García -> Luis Pérez>
>>>
```

Figura 14. Prueba en la shell con SQLite

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\W7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Importa el usuario de login y nuestros modelos
>>> from django.contrib.auth.models import User
>>> from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion
>>>
>>> # Crea la moneda
>>> clp = Moneda.objects.create(codigo='CLP', nombre='Peso chileno', simbolo='$')
>>>
>>> # Crea dos usuarios de login; create_user guarda la contraseña cifrada
>>> u1 = User.objects.create_user(username='ana', password='Clave12345!')
>>> u2 = User.objects.create_user(username='luis', password='Clave12345!')
>>>
>>> # Crea un cliente por cada usuario (relación 1:1)
>>> ana = Cliente.objects.create(usuario=u1, nombre='Ana García', email='ana@example.com', telefono='123456789')
>>> luis = Cliente.objects.create(usuario=u2, nombre='Luis Pérez', email='luis@example.com')
>>>
>>> # Crea una cuenta para cada cliente, ambas en pesos
>>> c_ana = Cuenta.objects.create(cliente=ana, moneda=clp, numero='0001')
>>> c_luis = Cuenta.objects.create(cliente=luis, moneda=clp, numero='0002')
>>>
>>> # Un depósito de 100000 en la cuenta de Ana (solo tiene destino)
>>> Transaccion.objects.create(tipo='deposito', cuenta_destino=c_ana, monto=100000)
<Transaccion: Depósito de 100000>
>>>
>>> # Una transferencia de 30000 de Ana a Luis
>>> Transaccion.objects.create(tipo='transferencia', cuenta_origen=c_ana, cuenta_destino=c_luis, monto=30000)
<Transaccion: Transferencia de 30000>
>>>
>>> # Saldos calculados: Ana debe tener 70000 y Luis 30000
>>> print(c_ana.saldo, c_luis.saldo)
70000.00 30000.00
>>>
>>> # Ana agenda a Luis como contacto (se crea la ficha intermedia)
>>> Contacto.objects.create(propietario=ana, agendado=luis, apodo='Lucho')
<Contacto: Ana García -> Luis Pérez>
>>>
>>> # Contactos de Ana y en qué agendas aparece Luis
>>> print(ana.contactos.all())
<QuerySet [(<Cliente: Luis Pérez>)]>
>>> print(luis.agendado_por.all())
<QuerySet [(<Cliente: Ana García>)]>
>>>
>>> # Prueba de la restricción de la base de datos: un monto 0 debe ser rechazado
>>> from django.db import IntegrityError
>>> try:
...     Transaccion.objects.create(tipo='deposito', cuenta_destino=c_ana, monto=0)
... except IntegrityError:
...     print('La base rechazó el monto 0')

```

Figura 15. Prueba en la shell con PostgreSQL

Etapa 4: consultas personalizadas

Objetivo

Acceder y manipular datos con el ORM de Django y con SQL propio: CRUD desde la shell, filtros avanzados, anotaciones y agregaciones, `raw()` y cursores.

Rama de trabajo: `feature/consultas`. No se modificó ningún modelo, por lo que esta etapa no genera migraciones nuevas.

Qué se hizo

4.1 Datos de demostración: poblar_datos

El comando `python manage.py poblar_datos` carga clientes, cuentas, contactos y movimientos de ejemplo. Usa `get_or_create` con un campo único (código, número de cuenta, descripción o usuario), por lo que **se puede ejecutar varias veces sin duplicar datos**. Las fechas de los movimientos se reparten en los últimos 60 días, para poder probar filtros por fecha.

Tabla	Total en la base
Monedas	2 (CLP y USD)
Clientes	5
Cuentas	7
Contactos	5
Transacciones	18

Saldos resultantes:

Cuenta	Dueño	Moneda	Saldo
0001	Ana García	CLP	115000
0002	Luis Pérez	CLP	60000
0003	Ana García	USD	400
0004	Carla Soto	CLP	140000
0005	Diego Rojas	CLP	125000
0006	Diego Rojas	USD	400
0007	Marta Vega	CLP	95000

Dos datos se dejaron a propósito para probar más adelante: Luis y Diego tienen teléfono `NULL`, y Marta lo tiene como texto vacío (`' '`).

La segunda ejecución informa `0` registros nuevos, lo que demuestra que no duplica.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Ejecuta el comando; el nombre es el del archivo, sin .py
>> python manage.py poblar_datos
Registros nuevos en esta ejecución:
monedas: 1
clientes: 3
cuentas: 5
contactos: 4
transacciones: 16
Totales en la base de datos:
monedas: 2
clientes: 5
cuentas: 7
contactos: 5
transacciones: 18
SalDOS por cuenta:
0001 (CLP) Ana García: 115000
0002 (CLP) Luis Pérez: 60000.00
0003 (USD) Ana García: 400.00
0004 (CLP) Carla Soto: 140000
0005 (CLP) Diego Rojas: 125000
0006 (USD) Diego Rojas: 400
0007 (CLP) Marta Vega: 95000
```

Figura 16. Primera ejecución de poblar_datos

```
>> python manage.py poblar_datos
Registros nuevos en esta ejecución:
monedas: 0
clientes: 0
cuentas: 0
contactos: 0
transacciones: 0
Totales en la base de datos:
monedas: 2
clientes: 5
cuentas: 7
contactos: 5
transacciones: 18
SalDOS por cuenta:
0001 (CLP) Ana García: 115000
0002 (CLP) Luis Pérez: 60000.00
0003 (USD) Ana García: 400.00
0004 (CLP) Carla Soto: 140000
0005 (CLP) Diego Rojas: 125000
0006 (USD) Diego Rojas: 400
0007 (CLP) Marta Vega: 95000
```

Figura 17. Segunda ejecución: sin registros nuevos

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Mismo comando, ahora sobre la base alke_wallet de PostgreSQL
>> python manage.py poblar_datos
Registros nuevos en esta ejecución:
monedas: 1
clientes: 3
cuentas: 5
contactos: 4
transacciones: 16
Totales en la base de datos:
monedas: 2
clientes: 5
cuentas: 7
contactos: 5
transacciones: 18
SalDOS por cuenta:
0001 (CLP) Ana García: 115000.00
0002 (CLP) Luis Pérez: 60000.00
0003 (USD) Ana García: 400.00
0004 (CLP) Carla Soto: 140000.00
0005 (CLP) Diego Rojas: 125000.00
0006 (USD) Diego Rojas: 400.00
0007 (CLP) Marta Vega: 95000.00
```

Figura 18. poblar_datos en PostgreSQL

4.2 CRUD desde la shell

Se hizo sobre un cliente temporal, **Pedro Prueba**, que se crea y se borra, para dejar intactos los datos de demostración.

```
# CREATE: create() guarda de inmediato; Modelo(...) más save() lo hace en dos
pasos
usuario = User.objects.create_user(username='pedro',
password='<contraseña_de_prueba>')
```

```

pedro = Cliente(usuario=usuario, nombre='Pedro Prueba',
email='pedro@example.com')
pedro.save()

# READ: get() devuelve un registro; filter() devuelve una lista; exists()
devuelve True o False
Cliente.objects.get(email='pedro@example.com')
Cliente.objects.filter(email='pedro@example.com').exists()

# UPDATE: save() guarda un objeto; update() modifica directo en la base y
devuelve las filas cambiadas
pedro.telefono = '555123456'
pedro.save()
Cuenta.objects.filter(numero='0008').update(activa=False)

# DELETE: primero se borra lo que protege (movimiento, cuenta) y luego el usuario
deposito.delete()
cuenta.delete()
usuario.delete()

```

Operación	Qué se comprobó
Create	El objeto no tiene id hasta que se guarda; el saldo de un depósito de 10000 es 10000
Read	get() falla con DoesNotExist si no hay registro; las relaciones se recorren en ambos sentidos, incluida la 1:1
Update	update() no actualiza el objeto que ya está en memoria hasta usar refresh_from_db()
Validación	full_clean() detecta Un depósito no debe tener cuenta origen. y el monto mínimo
Delete	PROTECT impide borrar una moneda con cuentas y una cuenta con movimientos; CASCADE borra el cliente al borrar su usuario

Al terminar, la base quedó como antes: 5 clientes, 7 cuentas y 18 movimientos.

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Importa el usuario de login y nuestros modelos
>>> from django.contrib.auth.models import User
>>> from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion
>>>
>>> # CREATE con create(): crea el objeto y lo guarda en la base en un solo paso
>>> usuario = User.objects.create_user(username='pedro', password='Demo12345!')
>>>
>>> # CREATE en dos pasos: primero se arma el objeto solo en memoria...
>>> pedro = Cliente(usuario=usuario, nombre='Pedro Prueba', email='pedro@example.com')
>>>
>>> # ...y todavía no tiene id, porque no existe en la base
>>> print(pedro.pk)
None
>>>
>>> # save() lo guarda en la base de datos
>>> pedro.save()
>>>
>>> # Ahora sí tiene id
>>> print(pedro.pk)
7
>>>
>>> # Busca la moneda CLP que ya existe (get devuelve un solo objeto)
>>> clp = Moneda.objects.get(codigo='CLP')
>>>
>>> # Crea una cuenta para Pedro en pesos
>>> cuenta = Cuenta.objects.create(cliente=pedro, moneda=clp, numero='0008')
>>>
>>> # Crea un depósito de 10000 en esa cuenta (solo tiene destino)
>>> deposito = Transaccion.objects.create(
...     tipo='deposito',
...     cuenta_destino=cuenta,
...     monto=10000,
...     descripcion='Depósito de prueba CRUD',
... )
>>>
>>> # El saldo se calcula sumando movimientos: debe ser 10000
>>> print(cuenta.saldo)
10000.00

```

Figura 19. CRUD: crear

```

>>> # Todos los clientes (ahora son 6: los 5 de la demostración más Pedro)
>>> print(Cliente.objects.all())
<QuerySet [ <Cliente: Ana García>, <Cliente: Luis Pérez>, <Cliente: Carla Soto>, <Cliente: Diego Rojas>, <Cliente: Marta Vega>, <Cliente: Pedro Prueba>]>
>>>
>>> # Cuántos hay
>>> print(Cliente.objects.count())
6
>>>
>>> # get() devuelve UN registro; falla si no hay ninguno o si hay más de uno
>>> c = Cliente.objects.get(email='pedro@example.com')
>>> print(c.nombre, c.email)
Pedro Prueba pedro@example.com
>>>
>>> # filter() devuelve una lista (QuerySet), aunque tenga un solo elemento
>>> print(Cliente.objects.filter(email='pedro@example.com'))
<QuerySet [ <Cliente: Pedro Prueba>]>
>>>
>>> # Ordena por nombre y trae solo esa columna, como lista simple
>>> print(Cliente.objects.order_by('nombre').values_list('nombre', flat=True))
<QuerySet [ 'Ana García', 'Carla Soto', 'Diego Rojas', 'Luis Pérez', 'Marta Vega', 'Pedro Prueba' ]>
>>>
>>> # exists() responde True o False sin traer los datos
>>> print(Cliente.objects.filter(email='nadie@example.com').exists())
False
>>>
>>> # get() con un registro inexistente lanza DoesNotExist; lo capturamos para verlo
>>> try:
...     Cliente.objects.get(email='nadie@example.com')
... except Cliente.DoesNotExist:
...     print('No existe ese cliente')
...
No existe ese cliente
>>> # Navegar relaciones, hacia atrás: las cuentas de Pedro
>>> print(pedro.cuentas.all())
<QuerySet [ <Cuenta: 0008 (CLP)>]>
>>>
>>> # Hacia adelante: el cliente dueño de la cuenta
>>> print(cuenta.cliente.nombre)
Pedro Prueba
>>>
>>> # Movimientos que recibió la cuenta
>>> print(cuenta.transacciones_recibidas.all())
<QuerySet [ <Transaccion: Depósito de 10000.00>]>
>>>
>>> # Relación 1:1 en los dos sentidos
>>> print(pedro.usuario.username)
pedro
>>> print(usuario.cliente.nombre)
Pedro Prueba

```

Figura 20. CRUD: leer

```

>>> # UPDATE con save(): se cambia el atributo en memoria y se guarda
>>> pedro.telefono = '555123456'
>>> pedro.save()
>>>
>>> # refresh_from_db() vuelve a leer el registro desde la base, para confirmar el cambio
>>> pedro.refresh_from_db()
>>> print(pedro.telefono)
555123456
>>>
>>> # UPDATE masivo con update(): modifica directo en la base, sin cargar objetos
>>> # Devuelve cuántas filas cambió
>>> filas = Cuenta.objects.filter(numero='0008').update(activa=False)
>>> print(filas)
1
>>>
>>> # El objeto "cuenta" en memoria sigue con el valor viejo hasta que lo releemos
>>> print(cuenta.activa)
True
>>> cuenta.refresh_from_db()
>>> print(cuenta.activa)
False

```

Figura 21. CRUD: actualizar

```

>>> # Importa el error de validación
>>> from django.core.exceptions import ValidationError
>>>
>>> # Arma en memoria (sin guardar) un depósito inválido: los depósitos no llevan cuenta origen
>>> mala = Transaccion(tipo='deposito', cuenta_origen=cuenta, cuenta_destino=cuenta, monto=100)
>>>
>>> # full_clean() ejecuta las validaciones de campos y el método clean() del modelo
>>> try:
...     mala.full_clean()
... except ValidationError as e:
...     print(e.messages)
...
['Un depósito no debe tener cuenta origen.']
>>> # Un monto 0 incumple el validador de monto mínimo
>>> otra = Transaccion(tipo='deposito', cuenta_destino=cuenta, monto=0)
>>> try:
...     otra.full_clean()
... except ValidationError as e:
...     print(e.message_dict)
...
{'monto': ['Ensure this value is greater than or equal to 0.01.']}

```

Figura 22. Validación con full_clean()

```

>>> # Importa el error que lanza PROTECT
>>> from django.db.models import ProtectedError
>>>
>>> # Intento 1: borrar la moneda CLP, que tiene cuentas asociadas
>>> try:
...     clp.delete()
... except ProtectedError as e:
...     print('Bloqueado:', e.args[0])
...
Bloqueado: Cannot delete some instances of model 'Moneda' because they are referenced through protected foreign keys: 'Cuenta.moneda'.
>>> # Intento 2: borrar la cuenta de Pedro, que tiene un movimiento
>>> try:
...     cuenta.delete()
... except ProtectedError as e:
...     print('Bloqueado:', e.args[0])
...
Bloqueado: Cannot delete some instances of model 'Cuenta' because they are referenced through protected foreign keys: 'Transaccion.cuenta_destino'.
>>> # Orden correcto: primero se borra quien protege (el movimiento), luego la cuenta
>>> deposito.delete()
(1, {'gestion.Transaccion': 1})
>>> cuenta.delete()
(1, {'gestion.Cuenta': 1})
>>>
>>> # Borrar el usuario borra en cascada a su cliente (CASCADE)
>>> # Devuelve el total de objetos borrados y el detalle por modelo
>>> print(usuario.delete())
(2, {'gestion.Cliente': 1, 'auth.User': 1})
>>>
>>> # Verificación final: la base debe quedar como antes del ejercicio
>>> print(Cliente.objects.count(), Cuenta.objects.count(), Transaccion.objects.count())
5 7 18
>>> print(User.objects.filter(username='pedro').exists())
False
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...

```

Figura 23. CRUD: borrar, con PROTECT y CASCADE

4.3 Filtros avanzados

filter() y exclude() encadenan condiciones; la coma equivale a **Y**, y para **O** y **NO** se usa Q con | y ~. El doble guion bajo cumple dos funciones: campo__operador compara, y relacion__campo cruza tablas.

Consulta	Resultado
monto__gt=50000	4 movimientos
monto__gte=50000	6
monto__range=(10000, 30000)	7
descripcion__icontains='retiro'	3
nombre__istartswith='a' (clientes)	Ana García
fecha__gte hace 14 días	8 (depende de la fecha en que se ejecute)

Consulta	Resultado
tipo='transferencia', monto__gte=20000	3
exclude(tipo='deposito')	10
Q(tipo='retiro') Q(monto__gte=100000)	6
~Q(tipo='deposito') & Q(monto__lt=20000)	5
cuenta_destino_cliente_nombre='Ana García'	5
Cuentas con moneda__codigo='USD'	0003 y 0006
Cientes con cuentas__moneda__codigo='USD'	Ana García y Diego Rojas
Cientes con fichas_de_agenda__isnull=True	Marta Vega

NULL frente a texto vacío:

Filtro	Cientes
telefono__isnull=True	Luis Pérez y Diego Rojas
telefono=''	Marta Vega
telefono__isnull=False	Ana García, Carla Soto y Marta Vega
telefono__isnull=False con exclude(telefono='')	Ana García y Carla Soto

`isnull=False` cuenta como "con teléfono" a Marta, que en realidad no tiene. Para detectar un teléfono escrito hay que descartar también el texto vacío.

Los querysets son perezosos: no consultan la base hasta que se usan, y `.query` muestra el SQL que se ejecutaría.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> # Q permite combinar condiciones con O, Y y NO
>>> from django.db.models import Q
>>>
>>> # timezone y timedelta sirven para calcular fechas relativas a "hoy"
>>> from django.utils import timezone
>>> from datetime import timedelta
>>>
>>> # Importa nuestros modelos
>>> from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion
>>>
>>> # Movimientos con monto mayor que 50000 (gt = greater than)
>>> print(Transaccion.objects.filter(monto__gt=50000).count())
4
>>>
>>> # Mayor o igual que 50000 (gte): suma los movimientos de exactamente 50000
>>> print(Transaccion.objects.filter(monto__gte=50000).count())
6
>>>
>>> # Entre 10000 y 30000, ambos extremos incluidos
>>> print(Transaccion.objects.filter(monto__range=(10000, 30000)).count())
7
>>>
>>> # Descripción que contiene "retiro", sin distinguir mayúsculas (la i de iconains)
>>> print(Transaccion.objects.filter(descripcion__icontains='retiro').count())
3
>>>
>>> # Clientes cuyo nombre empieza por "a"
>>> print(Cliente.objects.filter(nombre__startswith='a'))
<QuerySet [ <Cliente: Ana García > ]>
>>>
>>> # Movimientos de los últimos 14 días: se calcula la fecha límite y se compara
>>> hace_14_dias = timezone.now() - timedelta(days=14)
>>> print(Transaccion.objects.filter(fecha__gte=hace_14_dias).count())
8
```

Figura 24. Operadores de campo

```

>>> # La coma funciona como Y: transferencias Y de 20000 o más
>>> print(Transaccion.objects.filter(tipo='transferencia', monto__gte=20000).count())
3
>>>
>>> # exclude: todos los movimientos menos los depósitos
>>> print(Transaccion.objects.exclude(tipo='deposito').count())
10
>>>
>>> # O con Q: retiros O montos de 100000 o más
>>> print(Transaccion.objects.filter(Q(tipo='retiro') | Q(monto__gte=100000)).count())
6
>>>
>>> # NO con ~: que no sean depósitos Y con monto menor que 20000
>>> print(Transaccion.objects.filter(~Q(tipo='deposito') & Q(monto__lt=20000)).count())
5

```

Figura 25. exclude y Q

```

>>> # Movimientos que llegaron a cuentas de Ana
>>> # Camino: transacción -> cuenta destino -> cliente -> nombre
>>> print(Transaccion.objects.filter(cuenta_destino__cliente__nombre='Ana García').count())
5
>>>
>>> # Cuentas en dólares: cruza la clave foránea hacia Moneda
>>> print(Cuenta.objects.filter(moneda_codigo='USD'))
<QuerySet [Cuenta: 0003 (USD), <Cuenta: 0006 (USD)>]>
>>>
>>> # Clientes con alguna cuenta en dólares (relación inversa: el related_name "cuentas")
>>> print(Cliente.objects.filter(cuentas_moneda_codigo='USD'))
<QuerySet [Cuenta: Ana García, <Cuenta: Diego Rojas]>]
>>>
>>> # Clientes que NO han agendado a nadie (no tienen fichas como propietario)
>>> print(Cliente.objects.filter(fichas_de_agenda_isnull=True))
<QuerySet [Cuenta: Marta Vega]>]
>>>
>>> # Apodo y nombre real de cada contacto de Ana
>>> print(Contacto.objects.filter(propietario_nombre='Ana García').values_list('apodo', 'agendado_nombre'))
<QuerySet [(<'Lucho', 'Luis Pérez'), (<'Carli', 'Carla Soto')>]>
>>>
>>>

```

Figura 26. Filtros a través de relaciones

```

>>> # Clientes con teléfono NULL (nunca se registró)
>>> print(Cliente.objects.filter(telefono_isnull=True))
<QuerySet [Cuenta: Luis Pérez, <Cuenta: Diego Rojas]>]
>>>
>>> # Clientes con teléfono guardado como texto vacío
>>> print(Cliente.objects.filter(telefono=''))
<QuerySet [Cuenta: Marta Vega]>]
>>>
>>> # Clientes que NO tienen NULL: incluye al del texto vacío
>>> print(Cliente.objects.filter(telefono_isnull=False))
<QuerySet [Cuenta: Ana García, <Cuenta: Carla Soto, <Cuenta: Marta Vega]>]
>>>
>>> # Clientes con un teléfono realmente escrito: ni NULL ni vacío
>>> print(Cliente.objects.filter(telefono_isnull=False).exclude(telefono=''))
<QuerySet [Cuenta: Ana García, <Cuenta: Carla Soto]>]
>>>
>>>

```

Figura 27. NULL frente a texto vacío

```

>>> # Los 3 movimientos de mayor monto: order_by con "-" ordena de mayor a menor
>>> # [:3] limita a tres resultados (equivale a LIMIT 3 en SQL)
>>> print(Transaccion.objects.order_by('-monto')[:3].values_list('descripcion', 'monto'))
<QuerySet [(<Depósito inicial Carla', Decimal('200000.00')), (<Depósito inicial Diego', Decimal('150000.00')), (<Depósito inicial Diego', Decimal('100000.00'))]>
>>>
>>> # Un QuerySet es "perezoso": no toca la base hasta que lo usas
>>> # .query muestra el SQL que Django ejecutaría
>>> consulta = Transaccion.objects.filter(tipo='retiro', monto__gt=10000).order_by('-monto')
>>> print(consulta.query)
SELECT "gestion_transaccion"."id", "gestion_transaccion"."tipo", "gestion_transaccion"."cuenta_origen_id", "gestion_transaccion"."cuenta_destino_id", "gestion_transaccion"."monto", "gestion_transaccion"."descripcion", "gestion_transaccion"."fecha" FROM "gestion_transaccion" WHERE ("gestion_transaccion"."monto" > 10000 AND "gestion_transaccion"."tipo" = retiro) ORDER BY "gestion_transaccion"."monto" DESC
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...

```

Figura 28. Orden, límite y SQL generado

4.4 Anotaciones y agregaciones

`aggregate()` devuelve **un** resultado para toda la consulta. `annotate()` agrega un cálculo **por fila**, y combinado con `values()` lo hace **por grupo** (equivale a GROUP BY). Los montos en pesos y en dólares no se suman entre sí, por lo que los totales se filtran por moneda.

Resumen global de los movimientos en pesos:

Cálculo	Resultado
Cantidad	15
Total	815000
Promedio	≈ 54333,33
Mayor y menor	200000 y 5000
Depósitos, transferencias y retiros	610000, 130000 y 75000 (con <code>Sum(..., filter=Q(...))</code>)

Por grupo y por fila:

Consulta	Resultado
Movimientos por tipo (todas las monedas)	Depósito 8, retiro 3, transferencia 7
Cuentas por cliente	Ana 2, Diego 2, Carla 1, Luis 1, Marta 1
Contactos agendados por cliente	Ana 2, Carla 1, Diego 1, Luis 1, Marta 0
Cientes con más de una cuenta (<code>num_cuentas__gt=1</code>)	Ana García y Diego Rojas

La trampa del saldo. Calcular las entradas y las salidas con dos `Sum` en un mismo `annotate` da resultados inflados. Django une las dos relaciones y cada movimiento se repite tantas veces como filas tiene el otro lado. La cuenta 0001 tiene 3 entradas y 2 salidas: cada entrada se cuenta 2 veces y cada salida 3 veces.

Cuenta	Entradas calculadas	Salidas calculadas	Entradas reales	Salidas reales
0001	320000	135000	160000	45000
0004	675000	255000	225000	85000
0005	350000	100000	175000	50000
0007	240000	50000	120000	25000

Las cuentas con pocos movimientos salen bien, por lo que el error pasa desapercibido hasta revisar una cuenta con entradas y salidas múltiples.

`Sum(distinct=True)` tampoco sirve: la cuenta 0002 recibió dos veces 30000 y contaría uno solo.

Solución: dos subconsultas (`Subquery`), una para lo que entró y otra para lo que salió, y una resta. Se calcula en una sola consulta, y el resultado coincide con la propiedad `Cuenta.saldo` en las siete cuentas.

```
>>> # Funciones de cálculo y herramientas para combinar condiciones
>>> from django.db.models import Count, Sum, Avg, Max, Min, Q, F
>>>
>>> # Herramientas para el bloque de subconsultas
>>> from django.db.models import OuterRef, Subquery, DecimalField
>>>
>>> # Coalesce reemplaza un valor vacío (NULL) por otro, por ejemplo 0
>>> from django.db.models.functions import Coalesce
>>>
>>> # Decimal para escribir montos exactos
>>> from decimal import Decimal
>>>
>>> # Nuestros modelos
>>> from gestion.models import Moneda, Cliente, Contacto, Cuenta, Transaccion
>>>
>>>
>>> # Solo movimientos en pesos: tienen origen o destino en una cuenta CLP
>>> solo_clp = Transaccion.objects.filter(
...     Q(cuenta_origen_moneda_codigo='CLP') | Q(cuenta_destino_moneda_codigo='CLP')
... )
>>>
>>> # aggregate devuelve un diccionario con un valor por cada cálculo pedido
>>> print(solo_clp.aggregate(
...     cantidad=Count('id'),
...     total=Sum('monto'),
...     promedio=Avg('monto'),
...     mayor=Max('monto'),
...     menor=Min('monto'),
... ))
{'cantidad': 15, 'total': Decimal('815000'), 'promedio': Decimal('54333.33333333333'), 'mayor': Decimal('200000'), 'menor': Decimal('5000')}
>>>
>>> # Suma condicional: filter dentro de Sum calcula tres totales en una sola consulta
>>> print(solo_clp.aggregate(
...     depositos=Sum('monto', filter=Q(tipo='deposito')),
...     transferencias=Sum('monto', filter=Q(tipo='transferencia')),
...     retiros=Sum('monto', filter=Q(tipo='retiro')),
... ))
{'depositos': Decimal('610000'), 'transferencias': Decimal('130000'), 'retiros': Decimal('75000')}
```

Figura 29. Aggregate

```
>>> # order_by() vacío quita el orden por defecto del modelo, para que no interfiera en la agrupación
>>> # values('tipo') agrupa por tipo; annotate calcula un resultado por cada grupo
>>> print(list(
...     Transaccion.objects.order_by().values('tipo')
...     .annotate(cantidad=Count('id'))
...     .order_by('tipo')
... ))
[{'tipo': 'deposito', 'cantidad': 8}, {'tipo': 'retiro', 'cantidad': 3}, {'tipo': 'transferencia', 'cantidad': 7}]
>>>
>>> # Lo mismo, pero solo en pesos y sumando los montos
>>> print(list(
...     solo_clp.order_by().values('tipo')
...     .annotate(cantidad=Count('id'), total=Sum('monto'))
...     .order_by('tipo')
... ))
[{'tipo': 'deposito', 'cantidad': 6, 'total': Decimal('610000')}, {'tipo': 'retiro', 'cantidad': 3, 'total': Decimal('75000')}, {'tipo': 'transferencia', 'cantidad': 6, 'total': Decimal('130000')}]
>>>
>>>
```

Figura 30. Agrupación por tipo

```
>>> # Cuenta las cuentas de cada cliente y las ordena de mayor a menor (y por nombre si empatan)
>>> print(list(
...     Cliente.objects.annotate(num_cuentas=Count('cuentas'))
...     .order_by('-num_cuentas', 'nombre')
...     .values_list('nombre', 'num_cuentas')
... ))
[('Ana García', 2), ('Diego Rojas', 2), ('Carla Soto', 1), ('Luis Pérez', 1), ('Marta Vega', 1)]
>>>
>>> # Se puede filtrar por la columna calculada: clientes con más de una cuenta
>>> print(list(
...     Cliente.objects.annotate(num_cuentas=Count('cuentas'))
...     .filter(num_cuentas__gt=1)
...     .order_by('nombre')
... ))
[<Cliente: Ana García>, <Cliente: Diego Rojas>]
>>>
>>> # Cuántos contactos ha agendado cada cliente (usa la ficha intermedia Contacto)
>>> print(list(
...     Cliente.objects.annotate(num_contactos=Count('fichas_de_agenda'))
...     .order_by('-num_contactos', 'nombre')
...     .values_list('nombre', 'num_contactos')
... ))
[('Ana García', 2), ('Carla Soto', 1), ('Diego Rojas', 1), ('Luis Pérez', 1), ('Marta Vega', 0)]
>>>
```

Figura 31. annotate sobre clients

```
>>> # Dos Sum sobre dos relaciones distintas en un mismo annotate: Django une ambas tablas
>>> # y cada movimiento se repite una vez por cada fila de la otra relación
>>> mal = Cuenta.objects.annotate(
...     entradas=Sum('transacciones_recibidas_monto'),
...     salidas=Sum('transacciones_enviadas_monto'),
... ).order_by('numero')
>>>
>>> # Imprime número de cuenta, entradas y salidas calculadas
>>> for c in mal:
...     print(c.numero, c.entradas, c.salidas)
...
0001 320000 135000
0002 60000 None
0003 400 None
0004 675000 255000
0005 350000 100000
0006 500 100
0007 240000 50000
```

Figura 32. La trampa del saldo

```

>>> # Tipo de campo decimal para los montos (14 dígitos, 2 decimales), igual que en el modelo
>>> campo_dinero = DecimalField(max_digits=14, decimal_places=2)
>>>
>>> # Subconsulta 1: para cada cuenta (OuterRef('pk')) es "la cuenta de la fila actual",
>>> # suma lo que recibió, agrupando por cuenta destino
>>> entradas_sq = (
...     Transaccion.objects
...     .filter(cuenta_destino=OuterRef('pk'))
...     .order_by()
...     .values('cuenta_destino')
...     .annotate(total=Sum('monto'))
...     .values('total')
... )
>>>
>>> # Subconsulta 2: lo mismo con lo que envió, agrupando por cuenta origen
>>> salidas_sq = (
...     Transaccion.objects
...     .filter(cuenta_origen=OuterRef('pk'))
...     .order_by()
...     .values('cuenta_origen')
...     .annotate(total=Sum('monto'))
...     .values('total')
... )
>>>
>>> # Cada cuenta recibe dos columnas calculadas; Coalesce pone 0 si no hubo movimientos
>>> # Luego se resta: saldo_calc = entradas - salidas
>>> cuentas_con_saldo = Cuenta.objects.annotate(
...     entradas=Coalesce(Subquery(entradas_sq, output_field=campo_dinero), Decimal('0.00')), output_field=campo_dinero,
...     salidas=Coalesce(Subquery(salidas_sq, output_field=campo_dinero), Decimal('0.00')), output_field=campo_dinero,
... ).annotate(saldo_calc=F('entradas') - F('salidas')).order_by('numero')
>>>
>>> # Imprime el cálculo y, al final, el saldo de la propiedad del modelo para compararlos
>>> for c in cuentas_con_saldo:
...     print(c.numero, c.entradas, c.salidas, c.saldo_calc, c.saldo)
...
0001 160000 45000 115000 115000
0002 60000 0 60000 60000.00
0003 400 0 400 400.00
0004 225000 85000 140000 140000
0005 175000 50000 125000 125000
0006 500 100 400 400
0007 120000 25000 95000 95000
>>> # También se puede filtrar por la columna calculada: cuentas en pesos con saldo sobre 100000
>>> print([c.numero for c in cuentas_con_saldo.filter(moneda_codigo='CLP', saldo_calc__gt=100000)])
['0001', '0004', '0005']
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...
(venv) PS D:\VsCode_Externo\Bootcamp_Python\W7_Acceso_datos_app_Django\Proyecto\alke_wallet>

```

Figura 33. Saldo correcto con subconsultas

4.5 SQL propio: raw() y cursores

Herramienta	Devuelve	Cuándo conviene
<code>raw()</code>	Objetos del modelo (con <code>.nombre</code> , <code>.email</code> , etc.)	Cuando se quiere seguir usando el modelo
Cursor	Filas planas (tuplas)	Reportes, conteos y modificaciones directas

```

# raw(): devuelve objetos Cliente; los valores van siempre como parámetros (%)
consulta = "SELECT * FROM gestion_cliente WHERE nombre LIKE %s ORDER BY nombre"
Cliente.objects.raw(consulta, ['%a%'])

# Cursor: devuelve filas planas; "with" lo cierra al terminar
with connection.cursor() as cursor:
    cursor.execute("SELECT COUNT(*) FROM gestion_cliente")
    total = cursor.fetchone()[0]

```

Resultados:

Prueba	Resultado
<code>telefono IS NOT NULL</code> (ejemplo de la consigna)	Ana, Carla y Marta (teléfono vacío)
Con AND <code>telefono <> ''</code>	Ana y Carla
Inyección SQL con texto pegado en el SQL	5 clientes (devuelve todos)
El mismo texto como parámetro %s	0 clientes

Prueba	Resultado
Columna extra COUNT(...) AS num_cuentas	Disponible como cliente.num_cuentas
SELECT sin la clave primaria	FieldDoesNotExist: Raw query must include the primary key
Saldo por cuenta con cursor	Igual al del ORM en las siete cuentas
UPDATE ... WHERE numero = %s con cursor	rowcount = 1; se revirtió después

Los parámetros `%s` tratan el valor como dato y nunca como código SQL. Pegar el texto dentro de la consulta permite que un valor malicioso cambie su condición, como ocurrió con el ejemplo de inyección.

El cursor y `raw()` no pasan por `clean()` ni por la lógica del modelo, por lo que las modificaciones de datos de la aplicación irán por el ORM.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\W7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # connection permite abrir un cursor y ejecutar SQL directo
>>> from django.db import connection
>>>
>>> # Nuestros modelos
>>> from gestion.models import Cliente, Cuenta
>>>
>>>
>>> # raw() ejecuta SQL y devuelve objetos Cliente; este es el ejemplo de la consigna
>>> consulta = "SELECT * FROM gestion_cliente WHERE telefono IS NOT NULL"
>>> for cliente in Cliente.objects.raw(consulta):
...     # repr() muestra las comillas, para distinguir un texto vacío de un teléfono real
...     print(cliente.nombre, repr(cliente.telefono))
...
Ana García '123456789'
Carla Soto '987654321'
Marta Vega ''
>>> # Versión corregida: además descarta los teléfonos guardados como texto vacío
>>> consulta = "SELECT * FROM gestion_cliente WHERE telefono IS NOT NULL AND telefono <> ''"
>>> for cliente in Cliente.objects.raw(consulta):
...     print(cliente.nombre, repr(cliente.telefono))
...
Ana García '123456789'
Carla Soto '987654321'
```

Figura 34. `raw()` básico

```
>>> # Con parámetros: el valor va aparte, en una lista, y reemplaza al %s de forma segura
>>> # LIKE '%a%' busca nombres que contengan una "a"
>>> consulta = "SELECT * FROM gestion_cliente WHERE nombre LIKE %s ORDER BY nombre"
>>> for cliente in Cliente.objects.raw(consulta, ['%a%']):
...     print(cliente.nombre)
...
Ana García
Carla Soto
Diego Rojas
Marta Vega
>>> # Un texto malicioso: intenta cambiar la condición para que siempre sea verdadera
>>> texto = "x' OR '1'='1"
>>>
>>> # FORMA INSEGURA (no usar): el texto se pega dentro del SQL
>>> # La consulta resultante es: ... WHERE nombre = 'x' OR '1'='1'
>>> inseguro = Cliente.objects.raw(f"SELECT * FROM gestion_cliente WHERE nombre = '{texto}'")
>>> print(len(list(inseguro)))
5
>>>
>>> # FORMA SEGURA: el mismo texto va como parámetro y se busca como nombre literal
>>> seguro = Cliente.objects.raw("SELECT * FROM gestion_cliente WHERE nombre = %s", [texto])
>>> print(len(list(seguro)))
0
>>>
```

Figura 35. Parámetros e inyección SQL

```

>>> # raw() convierte las columnas extra del SELECT en atributos del objeto
>>> # Aquí num_cuentas no existe en el modelo, pero queda disponible como cliente.num_cuentas
>>> consulta = """
... SELECT c.*, COUNT(cu.id) AS num_cuentas
... FROM gestion_cliente c
... LEFT JOIN gestion_cuenta cu ON cu.cliente_id = c.id
... GROUP BY c.id
... ORDER BY num_cuentas DESC, c.nombre
... """
>>> for cliente in Cliente.objects.raw(consulta):
...     print(cliente.nombre, cliente.num_cuentas)
...
Ana García 2
Diego Rojas 2
Carla Soto 1
Luis Pérez 1
Marta Vega 1
>>> # Regla de raw(): el SELECT debe incluir la clave primaria (id), o falla
>>> try:
...     list(Cliente.objects.raw("SELECT nombre FROM gestion_cliente"))
... except Exception as e:
...     print(type(e).__name__, e)
...
FieldDoesNotExist Raw query must include the primary key
>>>

```

Figura 36. Columnas extra y clave primaria

```

>>> # Un cursor ejecuta SQL y devuelve filas planas; "with" lo cierra al terminar
>>> with connection.cursor() as cursor:
...     # Ejemplo simple: cuenta los clientes
...     cursor.execute("SELECT COUNT(*) FROM gestion_cliente")
...     # fetchone() trae la primera fila (una tupla); [0] toma su primer valor
...     print(cursor.fetchone()[0])
...
<django.db.backends.sqlite3.base.SQLiteCursorWrapper object at 0x0000023A2A67D0D0>
5
>>> # Consulta de saldos: dos subconsultas por cuenta (entradas y salidas) y una resta
>>> # COALESCE reemplaza "nada" (NULL) por 0 cuando la cuenta no tiene movimientos
>>> consulta = """
... SELECT c.numero, m.codigo,
...        COALESCE((SELECT SUM(t.monto) FROM gestion_transaccion t WHERE t.cuenta_destino_id = c.id), 0)
...        - COALESCE((SELECT SUM(t.monto) FROM gestion_transaccion t WHERE t.cuenta_origen_id = c.id), 0)
...        AS saldo
... FROM gestion_cuenta c
... JOIN gestion_moneda m ON m.id = c.moneda_id
... ORDER BY c.numero
... """
>>> with connection.cursor() as cursor:
...     cursor.execute(consulta)
...     # description guarda los nombres de las columnas; el primer elemento de cada una es su nombre
...     print([columna[0] for columna in cursor.description])
...     # fetchall() trae todas las filas
...     for fila in cursor.fetchall():
...         print(fila)
...
<django.db.backends.sqlite3.base.SQLiteCursorWrapper object at 0x0000023A2A67D230>
['numero', 'codigo', 'saldo']
('0001', 'CLP', 115000)
('0002', 'CLP', 60000)
('0003', 'USD', 400)
('0004', 'CLP', 140000)
('0005', 'CLP', 125000)
('0006', 'USD', 400)
('0007', 'CLP', 95000)
>>> # Cursor con parámetro, y filas convertidas a diccionarios para leerlas mejor
>>> with connection.cursor() as cursor:
...     cursor.execute("SELECT numero, activa FROM gestion_cuenta WHERE numero = %s", ['0004'])
...     nombres = [columna[0] for columna in cursor.description]
...     # zip une cada nombre de columna con su valor
...     print([dict(zip(nombres, fila)) for fila in cursor.fetchall()])
...
<django.db.backends.sqlite3.base.SQLiteCursorWrapper object at 0x0000023A2A5F74D0>
[{'numero': '0004', 'activa': True}]
>>>

```

Figura 37. Saldos con cursor

```

>>> # UPDATE con parámetros: desactiva la cuenta 0004
>>> with connection.cursor() as cursor:
...     cursor.execute("UPDATE gestion_cuenta SET activa = %s WHERE numero = %s", [False, '0004'])
...     # rowcount indica cuántas filas cambió
...     print(cursor.rowcount)
...
<django.db.backends.sqlite3.base.SQLiteCursorWrapper object at 0x0000023A2A5F7620>
1
>>> # Comprueba el cambio leyendo con el ORM
>>> print(Cuenta.objects.get(numero='0004').activa)
False
>>>
>>> # Deja la cuenta como estaba
>>> with connection.cursor() as cursor:
...     cursor.execute("UPDATE gestion_cuenta SET activa = %s WHERE numero = %s", [True, '0004'])
...     print(cursor.rowcount)
...
<django.db.backends.sqlite3.base.SQLiteCursorWrapper object at 0x0000023A2A5FFC50>
1
>>> # Comprueba que volvió a su estado original
>>> print(Cuenta.objects.get(numero='0004').activa)
True
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...

```

Figura 38. UPDATE con cursor

4.6 Consultas reutilizables y comando `demo_consultas`

Las consultas quedaron como funciones en `gestion/consultas.py`, que se reutilizarán en las vistas de la etapa 6.

Función	Técnica
<code>movimientos_recientes(dias)</code>	<code>filter</code> con fecha
<code>movimientos_de_cliente(cliente)</code>	Q con O sobre origen y destino
<code>clientes_con_telefono()</code>	excluye de NULL y de texto vacío
<code>clientes_sin_contactos()</code>	<code>isnull</code> sobre una relación inversa
<code>clientes_con_numero_de_cuentas()</code>	<code>annotate</code> con <code>Count</code>
<code>resumen_por_tipo(moneda)</code>	values más <code>annotate</code> , filtrado por moneda
<code>cuentas_con_saldo()</code>	<code>annotate</code> con <code>Subquery</code>
<code>clientes_con_telefono_sql()</code>	<code>raw()</code>
<code>buscar_clientes_sql(texto)</code>	<code>raw()</code> con parámetros
<code>saldos_sql()</code>	Cursor con SQL puro

El comando `python manage.py demo_consultas` ejecuta las diez consultas de esta etapa (en la etapa 6 se sumaron dos funciones más para el reporte, hasta las 12 que tiene hoy `consultas.py`) y termina con un control que compara el saldo del ORM con el del SQL puro (coinciden todas las cuentas: `True`).

Resultados, idénticos en SQLite y PostgreSQL:

Consulta	Resultado
Movimientos de los últimos 14 días	8 (depende de la fecha)
Movimientos de Ana García	7 (tres depósitos y cuatro transferencias)
Cientes con teléfono escrito	Ana García y Carla Soto
Cientes sin contactos	Marta Vega
Resumen en CLP	Depósito 6 (610000), retiro 3 (75000), transferencia 6 (130000)

Consulta	Resultado
Resumen en USD	Depósito 2 (800), transferencia 1 (100)
Nombres que contienen "ar" (raw)	Ana García, Carla Soto y Marta Vega
SalDOS por cuenta	Los de la tabla de la sección 4.1

La única diferencia entre motores es el formato de los decimales: PostgreSQL muestra los montos con dos decimales (115000.00) y SQLite los muestra sin ellos (115000). El valor es el mismo.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\W7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py demo_consultas

1. Movimientos de los últimos 14 días
cantidad: 8

2. Movimientos de Ana García (como origen o destino)
transferencia: 30000.00
deposito: 100000.00
deposito: 50000.00 Sueldo Ana
transferencia: 100.00 Envío USD a Ana
transferencia: 10000.00 Pago de cuenta
transferencia: 15000.00 Devolución a Carla
deposito: 300.00 Depósito USD Ana

3. Clientes con teléfono escrito
[<Cliente: Ana García>, <Cliente: Carla Soto>]

4. Clientes sin contactos
[<Cliente: Marta Vega>]

5. Cantidad de cuentas por cliente
Ana García: 2
Diego Rojas: 2
Carla Soto: 1
Luis Pérez: 1
Marta Vega: 1

6. Resumen por tipo de movimiento
CLP:
deposito: 6 movimientos, total 610000
retiro: 3 movimientos, total 75000
transferencia: 6 movimientos, total 130000
USD:
deposito: 2 movimientos, total 800
transferencia: 1 movimientos, total 100

7. Saldo por cuenta (ORM con subconsultas)
0001 (CLP): 115000
0002 (CLP): 60000
0003 (USD): 400
0004 (CLP): 140000
0005 (CLP): 125000
0006 (USD): 400
0007 (CLP): 95000

8. Clientes con teléfono escrito (raw)
[<Cliente: Ana García>, <Cliente: Carla Soto>]

9. Clientes cuyo nombre contiene 'ar' (raw con parámetros)
[<Cliente: Ana García>, <Cliente: Carla Soto>, <Cliente: Marta Vega>]

10. Saldo por cuenta (cursor con SQL puro)
0001 (CLP) Ana García: 115000
0002 (CLP) Luis Pérez: 60000
0003 (USD) Ana García: 400
0004 (CLP) Carla Soto: 140000
0005 (CLP) Diego Rojas: 125000
0006 (USD) Diego Rojas: 400
0007 (CLP) Marta Vega: 95000
```

Figura 39. demo_consultas en SQLite

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py demo_consultas

1. Movimientos de los últimos 14 días
   cantidad: 8

2. Movimientos de Ana García (como origen o destino)
   transferencia: 30000.00
   deposito: 100000.00
   deposito: 50000.00 Sueldo Ana
   transferencia: 100.00 Envío USD a Ana
   transferencia: 10000.00 Pago de cuenta
   transferencia: 15000.00 Devolución a Carla
   deposito: 300.00 Depósito USD Ana

3. Clientes con teléfono escrito
   [<Cliente: Ana García>, <Cliente: Carla Soto>]

4. Clientes sin contactos
   [<Cliente: Marta Vega>]

5. Cantidad de cuentas por cliente
   Ana García: 2
   Diego Rojas: 2
   Carla Soto: 1
   Luis Pérez: 1
   Marta Vega: 1

6. Resumen por tipo de movimiento
   CLP:
     deposito: 6 movimientos, total 610000.00
     retiro: 3 movimientos, total 75000.00
     transferencia: 6 movimientos, total 130000.00
   USD:
     deposito: 2 movimientos, total 800.00
     transferencia: 1 movimientos, total 100.00

7. Saldo por cuenta (ORM con subconsultas)
   0001 (CLP): 115000.00
   0002 (CLP): 60000.00
   0003 (USD): 400.00
   0004 (CLP): 140000.00
   0005 (CLP): 125000.00
   0006 (USD): 400.00
   0007 (CLP): 95000.00

8. Clientes con teléfono escrito (raw)
   [<Cliente: Ana García>, <Cliente: Carla Soto>]

9. Clientes cuyo nombre contiene 'ar' (raw con parámetros)
   [<Cliente: Ana García>, <Cliente: Carla Soto>, <Cliente: Marta Vega>]

10. Saldo por cuenta (cursor con SQL puro)
   0001 (CLP) Ana García: 115000.00
   0002 (CLP) Luis Pérez: 60000.00
   0003 (USD) Ana García: 400.00
   0004 (CLP) Carla Soto: 140000.00
   0005 (CLP) Diego Rojas: 125000.00
   0006 (USD) Diego Rojas: 400.00
   0007 (CLP) Marta Vega: 95000.00

Control: saldo ORM frente a saldo SQL
```

Figura 40. demo_consultas en PostgreSQL

Decisiones y alternativas descartadas

Tema	Elegida	Alternativa descartada	Motivo
Datos de prueba	Comando propio poblar_datos, repetible sin duplicar	Cargarlos a mano en la shell o con un fixture JSON	Con pocos datos, los filtros y las anotaciones no muestran nada. El comando funciona igual en SQLite y PostgreSQL
Dónde viven las consultas	Funciones en gestion/consultas.py	Manager o QuerySet propio en models.py	Más simple, y se reutilizan después en las vistas
Valores en el SQL propio	Siempre parámetros %s	Pegar el texto dentro del SQL	Evita la inyección SQL
Prueba en ambos motores	Comando demo_consultas	Solo pruebas unitarias	Sirve como demostración funcional y compara SQLite con PostgreSQL con un solo comando

Incidencias y solución

Origen: 4.6 Consultas reutilizables y comando `demo_consultas`

Incidencia resuelta: adaptador de PostgreSQL bloqueado. Al probar `demo_consultas` en PostgreSQL apareció este error:

```
ImportError: DLL load failed while importing _psycopg: Una directiva de Control de aplicaciones bloqueó este archivo.
django.core.exceptions.ImproperlyConfigured: Error loading psycopg2 or psycopg module
```

Es la misma política de Windows que bloqueaba `pip.exe` en la etapa 0: impide cargar el archivo compilado de `psycopg2`. Django admite también `psycopg` (versión 3), que se instala sin archivos compilados propios y usa el `libpq.dll` que ya trae PostgreSQL.

```
# Instala el adaptador de PostgreSQL en Python puro (versión 3)
python -m pip install psycopg

# Verifica que carga; debe imprimir la versión y la palabra "python"
python -c "import psycopg; print(psycopg.__version__, psycopg.pq.__impl__)"

# Retira el adaptador bloqueado y actualiza la lista de dependencias
python -m pip uninstall psycopg2-binary -y
python -m pip freeze | Out-File -Encoding utf8 requirements.txt
```

No fue necesario cambiar `settings.py`: el motor `django.db.backends.postgresql` prueba primero `psycopg` y recurre a `psycopg2` solo si no lo encuentra. La consigna menciona `psycopg2` como ejemplo de adaptador; `psycopg` cumple el mismo papel.

Etapa 5: panel de administración

Objetivo

Aprovechar `django.contrib.admin` para administrar los datos sin programar pantallas: configurar el idioma, registrar los modelos con columnas, búsqueda y filtros, y crear un superusuario.

Rama de trabajo: `feature/admin`. Esta etapa agrega una segunda migración, la `0002`, que no modifica ninguna tabla.

Qué se hizo

5.1 Idioma y zona horaria

```
# Idioma del sitio: español de Chile; afecta al panel de administración y a los
# mensajes de validación
LANGUAGE_CODE = 'es-cl'

# Zona horaria para mostrar las fechas; la base de datos sigue guardando en UTC
# porque USE_TZ es True
TIME_ZONE = 'America/Santiago'
```

Comprobación	Resultado
<code>python manage.py check</code>	Sin problemas
<code>timezone.localtime()</code>	La hora actual, terminada en -03:00 (horario de verano de Chile)
<code>gettext('This field is required.')</code>	Este campo es obligatorio.

La zona horaria solo cambia cómo se **muestran** las fechas: la base de datos sigue guardándolas en UTC, por lo que las consultas de la etapa 4 no se ven afectadas. Django no trae una traducción propia para Chile y usa la del español general.

```
138 # Idioma del sitio: español de Chile; afecta al panel de administración y a los mensajes de validación
139 LANGUAGE_CODE = 'es-cl'
140
141 # Zona horaria para mostrar las fechas; la base de datos sigue guardando en UTC porque USE_TZ es True
142 TIME_ZONE = 'America/Santiago'
143
```

Figura 41. Idioma y zona horaria en `settings.py`

Figura 42. Pantalla de inicio de sesión del panel, en español

5.2 Nombres legibles y migración 0002

Cada modelo recibió su nombre en singular y plural dentro de `class Meta`:

```
# Nombres legibles que se muestran en el panel de administración
class Meta:
    verbose_name = 'transacción'
    verbose_name_plural = 'transacciones'
    ordering = ['-fecha']
```

El nombre de la app se definió en `gestion/apps.py`:

```
class GestionConfig(AppConfig):
    name = 'gestion'
    # Nombre que se muestra en el menú del panel de administración
    verbose_name = 'Gestión'
```

Comando	Resultado
<code>makemigrations gestion</code>	Crea <code>0002_alter_cliente_options_alter_contacto_options_and_more.py</code> , con cinco líneas ~ <code>Change Meta options on ...</code>
<code>sqlmigrate gestion 0002</code>	Solo comentarios -- (no-op): ningún cambio en las tablas
<code>migrate</code>	Applying <code>gestion.0002_...</code> OK (en SQLite y en PostgreSQL)
<code>showmigrations gestion</code>	[X] <code>0001_initial</code> y [X] <code>0002_...</code>
<code>makemigrations --check --dry-run</code>	No changes detected

Django guarda los nombres legibles dentro de las migraciones, por eso cambiarlos genera una migración nueva, aunque no ejecute SQL. Es un ejemplo del historial versionado del esquema.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Genera el plan: Django detecta que cambiaron las opciones de los cinco modelos
>> python manage.py makemigrations gestion
Migrations for 'gestion':
  gestion\migrations\0002_alter_cliente_options_alter_contacto_options_and_more.py
    ~ Change Meta options on cliente
    ~ Change Meta options on contacto
    ~ Change Meta options on cuenta
    ~ Change Meta options on moneda
    ~ Change Meta options on transaccion
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Muestra el SQL que ejecutará; debe estar vacío salvo comentarios
>> python manage.py sqlmigrate gestion 0002
BEGIN;
--
-- Change Meta options on cliente
--
-- (no-op)
--
-- Change Meta options on contacto
--
-- (no-op)
--
-- Change Meta options on cuenta
--
-- (no-op)
--
-- Change Meta options on moneda
--
-- (no-op)
--
-- Change Meta options on transaccion
--
-- (no-op)
COMMIT;
```

Figura 43. Migración 0002 generada y su SQL vacío

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Aplica la migración
>> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion, sessions
Running migrations:
  Applying gestion.0002_alter_cliente_options_alter_contacto_options_and_more... OK
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Debe mostrar [X] en las dos migraciones de gestion
>> python manage.py showmigrations gestion
gestion
[X] 0001_initial
[X] 0002_alter_cliente_options_alter_contacto_options_and_more
```

Figura 44. Migración 0002 aplicada

5.3 Registro de los modelos: *gestion/admin.py*

Modelo	Columnas	Búsqueda	Filtros	Extra
Moneda	código, nombre, símbolo	código, nombre		
Cliente	nombre, email, teléfono, cantidad de cuentas	nombre, email, usuario de login		Sus cuentas y contactos aparecen dentro de su ficha
Cuenta	número, cliente, moneda, saldo, activa, fecha	número, nombre y email del dueño	moneda, activa	Saldo calculado en una sola consulta
Transaccion	id, tipo, origen, destino, monto, descripción, fecha	descripción, números de cuenta	tipo	Navegación por año, mes y día
Contacto	propietario, agendado, apodo, fecha	los dos nombres y el apodo		

Tres piezas nuevas:

Pieza	Para qué sirve
Inline (TabularInline)	Muestra una tabla relacionada dentro de otra ficha: las cuentas y los contactos de cada cliente
get_queryset	Cambia qué registros trae el listado, para reutilizar las consultas de la etapa 4
@admin.display	Convierte un método en una columna del listado, ordenable con un clic

```
# Los contactos del cliente, en una tabla dentro de su ficha
class ContactoInline(admin.TabularInline):
    model = Contacto
    # Contacto tiene dos claves hacia Cliente; fk_name dice cuál une con esta
    # ficha
    fk_name = 'propietario'
    fields = ('agendado', 'apodo')
    extra = 0

# En CuentaAdmin: el listado reutiliza la consulta de la etapa 4, que calcula
# todos los saldos de una vez
def get_queryset(self, request):
    return consultas.cuentas_con_saldo().select_related('cliente', 'moneda')

# Columna del saldo, ordenable con un clic en su título
@admin.display(description='Saldo', ordering='saldo_calculado')
def saldo_actual(self, obj):
    # Una cuenta nueva todavía no tiene el saldo calculado, así que se muestra un
    # guion
    return getattr(obj, 'saldo_calculado', '-')
```

Contacto tiene dos claves hacia Cliente (propietario y agendado), por eso el inline necesita `fk_name`: sin él, Django no sabría cuál usar.

```

40 # Registra Cliente en el panel
41 @admin.register(Cliente)
42 class ClienteAdmin(admin.ModelAdmin):
43     list_display = ('nombre', 'email', 'telefono', 'cantidad_de_cuentas') # Columnas del listado; cantidad_de_cuentas es un método definido más abajo
44     search_fields = ('nombre', 'email', 'usuario__username') # La búsqueda también mira el nombre de usuario de login (relación 1:1)
45     readonly_fields = ('fecha_registro',) # La fecha de registro se llena sola, así que solo se muestra y no se edita
46     inlines = [CuentaInline, ContactoInline] # Dentro de la ficha del cliente aparecen sus cuentas y sus contactos
47
48     # Define qué registros trae el listado
49     def get_queryset(self, request):
50         queryset = super().get_queryset(request) # Parte de la consulta normal del panel
51         return queryset.annotate(num_cuentas=Count('cuentas')) # Agrega a cada cliente la columna calculada num_cuentas
52
53     # Columna calculada del listado; ordering permite ordenar haciendo clic en su título
54     @admin.display(description='Cuentas', ordering='num_cuentas')
55     def cantidad_de_cuentas(self, obj):
56         return obj.num_cuentas # Devuelve el valor calculado en get_queryset

```

Figura 45. Configuración de admin.py

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py shell -c "from djan
go.contrib import admin; print(sorted(m.__name__ for m in admin.site._registry))"
17 objects imported automatically (use -v 2 for details).

['Cliente', 'Contacto', 'Cuenta', 'Group', 'Moneda', 'Transaccion', 'User']

```

Figura 46. Modelos registrados en el panel

5.4 Superusuario y recorrido del panel

Comandos: ver «Comandos», apartado «5.4 Superusuario y recorrido del panel».

Los clientes de demostración (Ana, Luis, etc.) son usuarios comunes y no pueden entrar al panel: solo lo hace el personal. El superusuario se creó en SQLite y en PostgreSQL, porque cada base guarda sus propios usuarios.

Recorrido del panel:

Pantalla	Resultado
Inicio	Dos grupos: Autenticación y autorización, y Gestión (Clientes, Contactos, Cuentas, Monedas, Transacciones)
Clientes	5 clientes, del más nuevo al más antiguo, con su cantidad de cuentas (ordenable)
Ficha de Ana García	Tablas con sus cuentas 0001 (CLP) y 0003 (USD) y sus contactos Lucho y Carli
Cuentas	Saldos iguales a los de la etapa 4; filtro por moneda; orden por saldo
Transacciones	18 movimientos; filtro por tipo (3 retiros); navegación por fecha; búsqueda por número de cuenta
Contactos	5 fichas

Ejercicio de CRUD en el panel, con un cliente temporal (usuario tomas, cuenta 0009):

Operación	Resultado
Crear	Usuario, cliente con su cuenta (tabla anidada) y un depósito de 25000; el saldo de la 0009 pasa a 25000
Editar	Cambio de teléfono del cliente y desactivación de la cuenta
Validar	Los casos inválidos se rechazan con su mensaje (tabla siguiente)
Borrar	Borrar una cuenta con movimientos muestra una pantalla de protección; al borrar primero el depósito y luego el usuario, el cliente y su cuenta se borran en cascada. <i>Desde la etapa 6.1 el panel ya no permite borrar movimientos; para repetir este ejercicio, el depósito de prueba se borra desde la shell</i>

Al terminar, la base volvió a tener 5 clientes, 7 cuentas y 18 movimientos.

Validaciones de `clean()` en los formularios del panel:

Caso	Mensaje
Depósito con cuenta origen	Un depósito no debe tener cuenta origen.
Retiro sin cuenta origen	Un retiro necesita una cuenta origen.
Transferencia entre monedas distintas	Las cuentas deben tener la misma moneda.
Transferencia a la misma cuenta	La cuenta origen y la destino deben ser distintas.
Monto cero	Asegúrese de que este valor es mayor o igual a 0.01.
Contacto a sí mismo	Un cliente no puede agendarse a sí mismo.
Contacto duplicado	Contacto con este Propietario y Agendado ya existe.

Esto confirma lo que quedó pendiente de la etapa 3: las reglas de `clean()` no actúan al crear desde la shell, pero sí en los formularios.

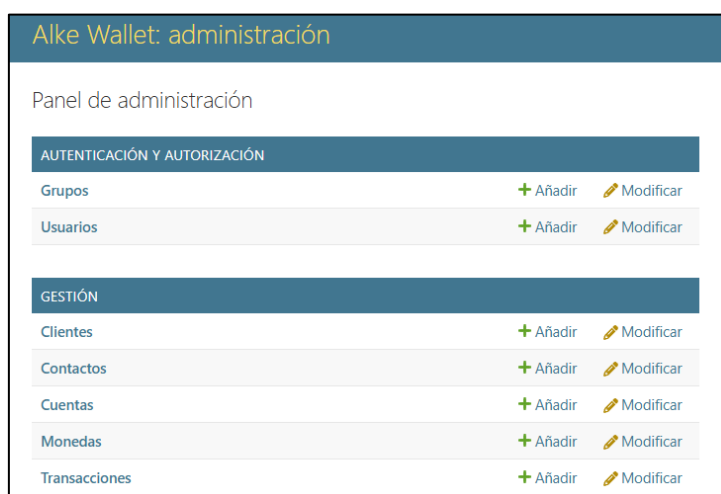


Figura 47. Inicio del panel

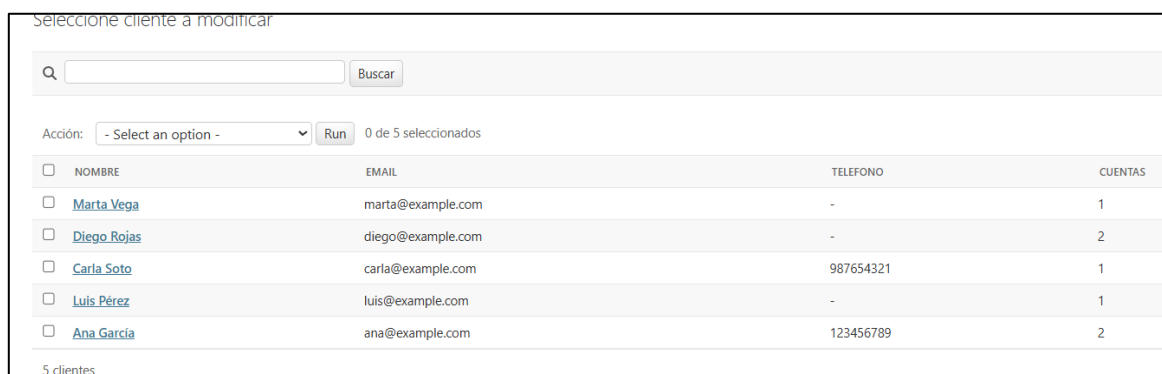


Figura 48. Listado de clientes con su cantidad de cuentas

Modificar cliente

Ana García

Usuario:

ana

Nombre:

Ana García

Email:

ana@example.com

Telefono:

123456789

Fecha registro:

29 de septiembre de 2026 a las 13:08

CUENTAS

NUMERO

0001 (CLP)

Modificar

0001

CLP

0003 (USD)

Modificar

0003

USD

+ Agregar Cuenta adicional.

CONTACTOS

AGENDADO

Ana García -> Luis Pérez

Luis Pérez

Ana García -> Carla Soto

Carla Soto

+ Agregar Contacto adicional.

APODO

Lucho

Carli

GUARDAR

Guardar y añadir otro

Guardar y continuar editando

Figura 49. Ficha de un cliente con sus tablas anidadas

Seleccione cuenta a modificar

Buscar

Acción:

- Select an option -

Run

 0 de 7 seleccionados

☐	NUMERO	CLIENTE	MONEDA	SALDO	ACTIVA	FECHA CREACION
☐	0001	Ana García	CLP	115000,00		29 de septiembre de 2026 a las 13:08
☐	0002	Luis Pérez	CLP	60000,00		29 de septiembre de 2026 a las 13:08
☐	0003	Ana García	USD	400,00		30 de septiembre de 2026 a las 20:28
☐	0004	Carla Soto	CLP	140000,00		30 de septiembre de 2026 a las 20:28
☐	0005	Diego Rojas	CLP	125000,00		30 de septiembre de 2026 a las 20:28
☐	0006	Diego Rojas	USD	400,00		30 de septiembre de 2026 a las 20:28
☐	0007	Marta Vega	CLP	95000,00		30 de septiembre de 2026 a las 20:28

7 cuentas

Figura 50. Listado de cuentas con saldos y filtros

Alke Wallet · Documentación técnica

Página 51

Filter by fecha [← Todas las fechas](#) [Agosto de 2026](#) [Septiembre de 2026](#)

Acción: - Select an option - Run 0 de 18 seleccionados

☐	ID	TIPO	CUENTA ORIGEN	CUENTA DESTINO	MONTO	DESCRIPCION	FECHA
☐	19	Depósito	-	0002 (CLP)	30000,00	Depósito Luis	29 de septiembre de 2026 a las 20:28
☐	2	Transferencia	0001 (CLP)	0002 (CLP)	30000,00	-	29 de septiembre de 2026 a las 13:08
☐	1	Depósito	-	0001 (CLP)	100000,00	-	29 de septiembre de 2026 a las 13:08
☐	18	Retiro	0007 (CLP)	-	5000,00	Retiro cajero Marta 2	28 de septiembre de 2026 a las 20:28
☐	17	Transferencia	0005 (CLP)	0004 (CLP)	10000,00	Ajuste entre amigos	27 de septiembre de 2026 a las 20:28
☐	16	Depósito	-	0001 (CLP)	50000,00	Sueldo Ana	25 de septiembre de 2026 a las 20:28
☐	15	Transferencia	0006 (USD)	0003 (USD)	100,00	Envío USD a Ana	23 de septiembre de 2026 a las 20:28
☐	14	Retiro	0007 (CLP)	-	20000,00	Retiro cajero Marta	20 de septiembre de 2026 a las 20:28
☐	13	Transferencia	0004 (CLP)	0001 (CLP)	10000,00	Pago de cuenta	15 de septiembre de 2026 a las 20:28

Figura 51. Listado de transacciones con filtro y navegación por fecha

Añadir transacción

Por favor, corrija el siguiente error.

Las cuentas deben tener la misma moneda.

Tipo:
Transferencia

Cuenta origen:
0009 (CLP)    

Cuenta destino:
0003 (USD)    

Monto:
24999,69

Figura 52. Validación en un formulario del panel

Eliminar

Are you sure you want to delete the usuario "tomas"? All of the following related items will be deleted:

Resumen

- Usuarios: 1
- Cientes: 1
- Cuentas: 1

Objetos

- Usuario: [tomas](#)
 - Ciente: [Tomas Prueba](#)
 - Cuenta: [0009 \(CLP\)](#)

Si, estoy seguro No, llévame atrás

Figura 53. Borrado protegido de una cuenta con movimientos

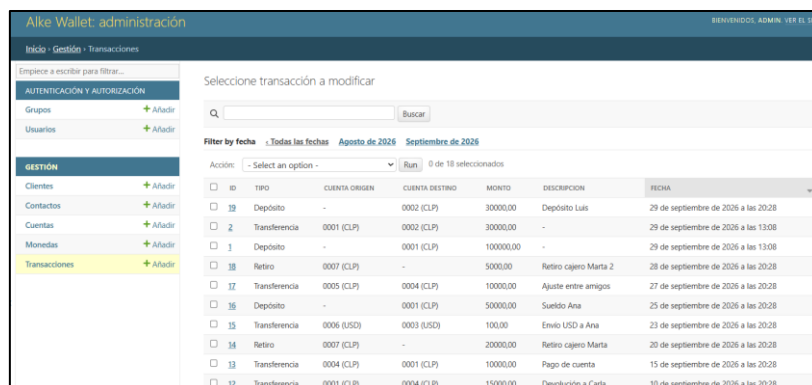


Figura 54. Panel funcionando sobre PostgreSQL

Comandos

5.4 Superusuario y recorrido del panel

```
# Crea la cuenta de administración (pide usuario, correo y contraseña)
python manage.py createsuperuser
```

Decisiones y alternativas descartadas

Tema	Elegida	Alternativa descartada	Motivo
Idioma y zona horaria	es-cl y America/Santiago	Inglés y UTC	El panel y los mensajes de validación salen en español, y las fechas en hora local
Nombres de los modelos	verbose_name en cada Meta, con la migración 0002	Dejar los nombres por defecto	Sin esto, el panel mostraría "Transacciones" y "Gestion"
Saldo en el panel	cuentas_con_saldo() de la etapa 4	La propiedad Cuenta.saldo	Una sola consulta para todo el listado, en vez de dos por cuenta
Superusuario	Crear en cada base de datos	Crear solo en una	Cada base guarda sus propios usuarios

Limitaciones conocidas

- **No revisaba el saldo disponible** (resuelto en la etapa 6.1): un retiro o una transferencia mayor que el saldo se guardaba y dejaba la cuenta en negativo. Ahora la regla vive en el modelo y se aplica también en el panel.
- **El superusuario no tiene un Cliente asociado.** Las pantallas de usuario que dependan de `request.user.cliente` deberán contemplar ese caso. (Resuelto en la etapa 7: el personal ve todo y un usuario sin cliente recibe un aviso.)

Reflexiones

- **Mucho resultado con poco código:** el panel completo, con búsqueda, filtros, tablas anidadas y validaciones, salió de unas 140 líneas en `admin.py`, reutilizando los modelos y las consultas.
- **Las validaciones del modelo se aprovechan solas:** `clean()` y las restricciones definidas en la etapa 2 actúan en el panel sin escribir nada más.
- **PROTECT y CASCADE se ven en acción:** el panel explica por qué no se puede borrar una cuenta con historial y qué se borraría en cascada, antes de confirmar.
- **Reutilizar consultas:** `cuentas_con_saldo()` sirve en el panel igual que en el comando de demostración, lo que justifica haberla dejado como función.

Etapa 6: pantallas con vistas basadas en clases

Objetivo

Construir las pantallas de la aplicación: CRUD con vistas basadas en clases, rutas dinámicas, formularios protegidos con CSRF y templates que heredan de una plantilla común.

Rama de trabajo: `feature/crud`. Esta etapa no cambia los modelos, por lo que no genera migraciones.

Qué se hizo

Cómo funciona cada pantalla:

Pieza	Para qué sirve	Archivo
Ruta	La dirección que escribe la persona	<code>gestion/urls.py</code>
Vista	Decide qué datos mostrar o guardar	<code>gestion/views.py</code>
Formulario	Define los campos y valida lo que se escribe	<code>gestion/forms.py</code>
Template	El HTML que ve la persona, con <code>{% csrf_token %}</code> en cada formulario	<code>templates/</code>

Rutas (todas con nombre, usados como `{% url 'gestion:cliente_detalle' pk %}`):

Sección	Rutas	Vistas
Inicio	<code>/</code>	<code>TemplateView</code>
Cientes	<code>/clientes/</code> , <code>/clientes/nuevo/</code> , <code>/clientes/<pk>/</code> , <code>/clientes/<pk>/editar/</code> , <code>/clientes/<pk>/eliminar/</code>	<code>ListView</code> , <code>CreateView</code> , <code>DetailView</code> , <code>UpdateView</code> , <code>DeleteView</code>
Cuentas	<code>/cuentas/</code> , <code>/cuentas/nueva/</code> , <code>/cuentas/<pk>/</code> , <code>/cuentas/<pk>/editar/</code> , <code>/cuentas/<pk>/eliminar/</code>	<code>ListView</code> , <code>CreateView</code> , <code>DetailView</code> , <code>UpdateView</code> , <code>DeleteView</code>
Transacciones	<code>/transacciones/</code> , <code>/transacciones/nueva/</code> , <code>/transacciones/<pk>/</code>	<code>ListView</code> , <code>FormView</code> , <code>DetailView</code>
Contactos	<code>/clientes/<pk>/contactos/nuevo/</code> , <code>/contactos/<pk>/eliminar/</code>	<code>CreateView</code> , <code>DeleteView</code>
Reporte	<code>/reporte/</code>	<code>TemplateView</code>

`<pk>` es una parte variable de la dirección: Django la convierte en un número y se la entrega a la vista para identificar el registro.

6.1 Reglas de saldo y cuentas activas

Las reglas viven en el modelo, por lo que valen para el panel y para las pantallas. Solo se revisan al crear un movimiento: `self._state.adding` es `True` mientras el objeto no se ha guardado.

```
# Reglas que solo se revisan al crear un movimiento nuevo, no al editar uno existente
if self._state.adding:
    # Una cuenta desactivada no puede enviar ni recibir movimientos nuevos
    if self.cuenta_origen is not None and not self.cuenta_origen.activa:
        raise ValidationError('La cuenta origen está inactiva.')
    if self.cuenta_destino is not None and not self.cuenta_destino.activa:
        raise ValidationError('La cuenta destino está inactiva.')
```

```
# El monto puede estar vacío si falló su propia validación; en ese caso no se
compara
# Si sale dinero de una cuenta (retiro o transferencia), debe haber saldo
suficiente
if self.cuenta_origen is not None and self.monto is not None:
    disponible = self.cuenta_origen.saldo
    if disponible < self.monto:
        raise ValidationError(f'Saldo insuficiente: la cuenta origen tiene
{disponible:.2f}.')
```

En el panel, `TransaccionAdmin` rechaza editar y borrar movimientos existentes (`has_change_permission` y `has_delete_permission`), por lo que se abren solo en modo lectura. Crear movimientos nuevos sigue funcionando, con las mismas reglas.

Prueba con `full_clean()` desde la shell, sin guardar nada (cuenta `0004` con saldo `140000` y `0005` con `125000`):

Caso	Resultado
Retiro de 100000 de la <code>0004</code>	Válido
Retiro de 150000 de la <code>0004</code>	Saldo insuficiente: la cuenta origen tiene 140000.00.
Transferencia de 200000 de la <code>0005</code> a la <code>0004</code>	Saldo insuficiente: la cuenta origen tiene 125000.00.
Transferencia de 1000 de la <code>0005</code> a la <code>0004</code>	Válido
Depósito de 50000 a la <code>0004</code>	Válido (un depósito no revisa saldo)
Depósito a una cuenta inactiva	La cuenta destino está inactiva.
Retiro desde una cuenta inactiva	La cuenta origen está inactiva.
Retiro sin monto	Este campo no puede ser nulo. (no un error de Python)
Retiro exacto del saldo (140000)	Válido
Retiro del saldo más un centavo	Saldo insuficiente: la cuenta origen tiene 140000.00.

```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Importa el error de validación y los modelos
>>> from django.core.exceptions import ValidationError
>>> from gestion.models import Cuenta, Transaccion
>>>
>>> # Cuentas de la demostración: 0004 (Carla, saldo 140000), 0005 (Diego, 125000), 0002 (Luis)
>>> c4 = Cuenta.objects.get(numero='0004')
>>> c5 = Cuenta.objects.get(numero='0005')
>>> c2 = Cuenta.objects.get(numero='0002')
>>>
>>> # Función auxiliar: valida un movimiento sin guardarlo y muestra el resultado
>>> def probar(descripcion, movimiento):
...     try:
...         movimiento.full_clean()
...         print(descripcion, '-> válido')
...     except ValidationError as e:
...         print(descripcion, '->', e.messages)
...
>>> # Casos con saldo
>>> probar('1 retiro 100000 de 0004', Transaccion(tipo='retiro', cuenta_origen=c4, monto=100000))
1 retiro 100000 de 0004 -> válido
>>> probar('2 retiro 150000 de 0004', Transaccion(tipo='retiro', cuenta_origen=c4, monto=150000))
2 retiro 150000 de 0004 -> ['Saldo insuficiente: la cuenta origen tiene 140000.00.']
>>> probar('3 transferencia 200000 de 0005 a 0004', Transaccion(tipo='transferencia', cuenta_origen=c5, cuenta_destino=c4, monto=200000))
3 transferencia 200000 de 0005 a 0004 -> ['Saldo insuficiente: la cuenta origen tiene 125000.00.']
>>> probar('4 transferencia 1000 de 0005 a 0004', Transaccion(tipo='transferencia', cuenta_origen=c5, cuenta_destino=c4, monto=1000))
4 transferencia 1000 de 0005 a 0004 -> válido
>>> probar('5 deposito 50000 a 0004', Transaccion(tipo='deposito', cuenta_destino=c4, monto=50000))
5 deposito 50000 a 0004 -> válido
>>>
>>> # Casos con cuenta inactiva: se desactiva solo en memoria (no se guarda en la base)
>>> c2.activa = False
>>> probar('6 deposito a cuenta inactiva', Transaccion(tipo='deposito', cuenta_destino=c2, monto=1000))
6 deposito a cuenta inactiva -> ['La cuenta destino está inactiva.']
>>> probar('7 retiro de cuenta inactiva', Transaccion(tipo='retiro', cuenta_origen=c2, monto=10))
7 retiro de cuenta inactiva -> ['La cuenta origen está inactiva.']
>>>
>>> # Monto vacío: debe dar un mensaje de campo, no un error de Python
>>> probar('8 retiro sin monto', Transaccion(tipo='retiro', cuenta_origen=c4))
8 retiro sin monto -> ['Este campo no puede ser nulo.']
>>>
>>> # Bordes del saldo
>>> probar('9 retiro exacto del saldo', Transaccion(tipo='retiro', cuenta_origen=c4, monto=140000))
9 retiro exacto del saldo -> válido
>>> probar('10 retiro del saldo más un centavo', Transaccion(tipo='retiro', cuenta_origen=c4, monto='140000.01'))
10 retiro del saldo más un centavo -> ['Saldo insuficiente: la cuenta origen tiene 140000.00.']
>>>
>>> # Sale de la shell
>>> exit()

```

Figura 55. Reglas del saldo probadas desde la shell

Vista transacción

Depósito de 30000.00

Tipo:

Depósito

Cuenta origen:

-

Cuenta destino:

[0002 \(CLP\)](#)

Monto:

30000,00

Descripción:

Depósito Luis

Fecha:

29 de septiembre de 2026 a las 20:27

Cerrar

Figura 56. Movimiento en modo lectura en el panel

Figura 57. Saldo insuficiente en el panel

6.2 Estructura base: templates/, rutas y página de inicio

Django busca los templates en la carpeta `templates/` de la raíz, gracias a esta línea de `core/settings.py`:

```
# Carpeta templates/ de la raíz del proyecto, donde viven las plantillas
compartidas
'DIRS': [BASE_DIR / 'templates'],
```

`templates/base.html` es la plantilla común (encabezado, menú, mensajes y pie). Cada página la hereda con `{% extends 'base.html' %}` y rellena solo sus bloques.

Una petición recorre este camino:

Paso	Qué pasa	Archivo
1	La persona escribe una dirección	
2	Django busca la ruta en las del proyecto	<code>core/urls.py</code>
3	Esa ruta lo deriva a las de la app (<code>include</code>)	<code>gestion/urls.py</code>
4	La ruta apunta a una vista	<code>gestion/views.py</code>
5	La vista prepara los datos y elige un template	<code>templates/*.html</code>
6	El template hereda <code>base.html</code> y se convierte en HTML	<code>templates/base.html</code>

En `core/urls.py`, el panel se declara antes de `include('gestion.urls')`, para que `/admin/` no lo capture la app. El menú creció junto con las pantallas: un enlace a una ruta que todavía no existe habría provocado un `NoReverseMatch`.

```

1  {% alke_wallet/templates/base.html %}
2
3  {% Plantilla base: el diseño común que heredan todas las páginas %}
4  <!DOCTYPE html>
5  <html lang="es">
6  <head>
7      {% Codificación de caracteres, para que se vean bien las tildes y la ñ %}
8      <meta charset="UTF-8">
9      {% Hace que la página se adapte al ancho del celular %}
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11     {% Título de la pestaña: cada página puede reemplazar este bloque %}
12     <title>{% block titulo %}Alke Wallet{% endblock %}</title>
13     {% Estilo mínimo temporal; en la etapa 7 se reemplaza por un archivo CSS estático %}
14     <style>
15         body { font-family: sans-serif; max-width: 960px; margin: 0 auto; padding: 1rem; }
16         nav a { margin-right: 1rem; }
17         table { border-collapse: collapse; width: 100%; }
18         th, td { border: 1px solid #ccc; padding: 0.4rem 0.6rem; text-align: left; }
19         .mensajes { list-style: none; padding: 0; }
20         .mensajes li { padding: 0.6rem; margin-bottom: 0.4rem; border-radius: 4px; background: #eee; }
21         .mensajes .success { background: #d4edda; }
22         .mensajes .error { background: #f8d7da; }
23         .errorlist { color: #b00020; }
24     </style>
25 </head>
26 <body>
27     {% Encabezado con el nombre del sitio y el menú %}
28     <header>
29         <h1>Alke Wallet</h1>
30         <nav>
31             {% url 'gestion:inicio' busca la dirección por el nombre de la ruta %}
32             <a href="{% url 'gestion:inicio' %}">Inicio</a>
33             {% Enlace al panel de administración de Django %}
34             <a href="{% url 'admin:index' %}">Panel de administración</a>
35         </nav>
36     </header>
37     <hr>
38     <main>
39         {% Muestra los mensajes de aviso (éxito o error) que dejan las vistas %}

```

Figura 58. Estructura de templates en VS Code

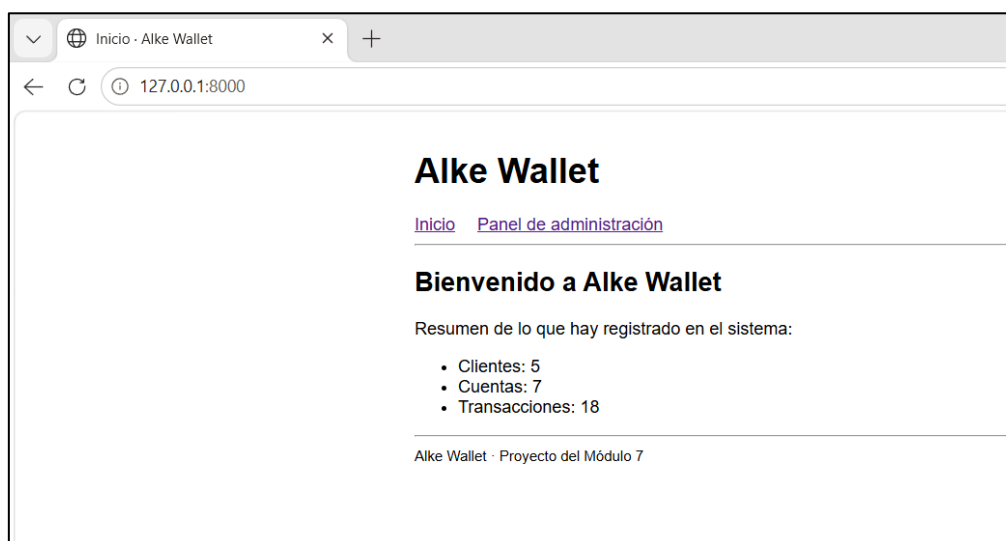


Figura 59. Página de inicio

6.3 CRUD de clientes

El formulario de creación solo ofrece usuarios que todavía no tienen cliente:

```
# Solo muestra los usuarios que todavía no tienen un cliente asociado
self.fields['usuario'].queryset =
User.objects.filter(cliente__isnull=True).order_by('username')
```

El borrado captura la protección de las cuentas con historial:

```
try:
    # Intenta el borrado normal de Django
    respuesta = super().form_valid(form)
except ProtectedError:
    # Si alguna cuenta del cliente tiene movimientos, Django lo impide
    messages.error(self.request, f'No se puede eliminar a {nombre}: sus cuentas tienen movimientos registrados.')
    return redirect('gestion:cliente_detalle', pk=self.object.pk)
```

Prueba	Resultado
Listado	5 clientes por nombre, con su cantidad de cuentas y un guion donde no hay teléfono
Ficha de Ana García	Sus datos, cuentas 0001 (\$115000) y 0003 (US\$400) con saldo, y sus contactos
Dirección inexistente (/clientes/9999/)	Error 404
Crear sin teléfono	Cliente «Tomás Prueba» creado correctamente. El teléfono se guarda como NULL, porque Django guarda NULL cuando se deja vacío un campo de texto que lo admite
Correo repetido	Ya existe Cliente con este Email.
Editar	El campo de usuario no aparece; aviso de actualización
Eliminar un cliente con cuentas con movimientos (Ana)	Aviso rojo No se puede eliminar a Ana García: sus cuentas tienen movimientos registrados. El cliente sigue existiendo
Eliminar un cliente sin cuentas	Cliente «...» eliminado correctamente.
CSRF: formulario, envío sin token y borrado sin token	200, 403 y 403

Alke Wallet

[Inicio](#) [Clientes](#) [Panel de administración](#)

Clientes

[Nuevo cliente](#)

Nombre	Correo electrónico	Teléfono	Cuentas	Acciones
Ana García	ana@example.com	123456789	2	Editar Eliminar
Carla Soto	carla@example.com	987654321	1	Editar Eliminar
Diego Rojas	diego@example.com	-	2	Editar Eliminar
Luis Pérez	luis@example.com	-	1	Editar Eliminar
Marta Vega	marta@example.com	-	1	Editar Eliminar
Tomas Prueba	tomas@example.com	555123456	0	Editar Eliminar

Figura 60. Listado de clients

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

Ana García

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Correo electrónico: ana@example.com
- Teléfono: 123456789
- Usuario de acceso: ana
- Registrado el: 29/09/2026 13:05

Cuentas

Número	Moneda	Saldo	Estado
0001	CLP	\$115000	Activa
0003	USD	US\$400	Activa

Contactos

Apodo	Nombre
Lucho	Luis Pérez
Carli	Carla Soto

Alke Wallet · Proyecto del Módulo 7

Figura 61. Ficha de un cliente

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

Nuevo cliente

Usuario de acceso: Solo aparecen usuarios sin cliente. Se crean en el panel de administración.

Nombre:

Correo electrónico:

Teléfono:

[Cancelar](#)

Alke Wallet · Proyecto del Módulo 7

Figura 62. Formulario de nuevo cliente

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

Nuevo cliente

Usuario de acceso: Solo aparecen usuarios sin cliente. Se crean en el panel de administración.

Nombre:

• Ya existe Cliente con este Email.

Correo electrónico:

Teléfono:

[Cancelar](#)

Alke Wallet · Proyecto del Módulo 7

Figura 63. Error de correo duplicado

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

Cliente «Pedro» creado correctamente.

Clientes

[Nuevo cliente](#)

Nombre	Correo electrónico	Teléfono	Cuentas	Acciones
Ana García	ana@example.com	123456789	2	Editar Eliminar
Carla Soto	carla@example.com	987654321	1	Editar Eliminar
Diego Rojas	diego@example.com	-	2	Editar Eliminar
Luis Pérez	luis@example.com	-	1	Editar Eliminar
Marta Vega	marta@example.com	-	1	Editar Eliminar
Pedro	Pedro@example.com	66556677	0	Editar Eliminar
Tomas Prueba	tomas@example.com	555123456	0	Editar Eliminar

Alke Wallet · Proyecto del Módulo 7

Figura 64. Cliente creado

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

Eliminar cliente

¿Seguro que quieres eliminar a **Tomas Prueba**?

También se eliminarán sus 0 cuenta(s) y 0 contacto(s) agendado(s).

Si alguna de sus cuentas tiene movimientos, no se podrá eliminar.

Alke Wallet · Proyecto del Módulo 7

Figura 65. Confirmación para eliminar un cliente

Alke Wallet

[Inicio](#)
[Clientes](#)
[Panel de administración](#)

No se puede eliminar a Ana García: sus cuentas tienen movimientos registrados.

Ana García

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Correo electrónico: ana@example.com
- Teléfono: 123456789
- Usuario de acceso: ana
- Registrado el: 29/09/2026 13:05

Cuentas

Número	Moneda	Saldo	Estado
0001	CLP	\$115000	Activa
0003	USD	US\$400	Activa

Contactos

Apodo	Nombre
Lucho	Luis Pérez
Carli	Carla Soto

Alke Wallet · Proyecto del Módulo 7

Figura 66. Eliminación protegida de un cliente con historial

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python .\manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> # Cliente de pruebas que SÍ exige el token CSRF, como un navegador real
>>> from django.test import Client
>>> c = Client(enforce_csrf_checks=True, HTTP_HOST='localhost')
>>>
>>> # Pedir el formulario funciona
>>> print('GET formulario ->', c.get('/clientes/nuevo/').status_code)
GET formulario -> 200
>>>
>>> # Enviar el formulario sin el token es rechazado
>>> print('POST sin token ->', c.post('/clientes/nuevo/', {'nombre': 'x'}).status_code)
Forbidden (CSRF token missing.): /clientes/nuevo/
POST sin token -> 403
>>>
>>> # Lo mismo para borrar: sin token, no se borra
>>> print('POST borrar sin token ->', c.post('/clientes/1/eliminar/').status_code)
Forbidden (CSRF token missing.): /clientes/1/eliminar/
POST borrar sin token -> 403
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...
```

Figura 67. Protección CSRF en clientes

6.4 CRUD de cuentas

Tema	Decisión
Qué se puede editar	Solo el número y si está activa. Cambiar el cliente o la moneda dejaría los movimientos históricos sin sentido
Eliminar	Se puede si no tiene movimientos. Si los tiene, se ofrece desactivarla
Saldo	El listado y la ficha usan <code>cuentas_con_saldo()</code> de la etapa 4
Número al crear	El formulario propone el siguiente (0008)
Crear desde un cliente	?cliente=3 en la dirección deja ese cliente elegido

Prueba	Resultado
Listado	7 cuentas con sus saldos: \$115000, \$60000, US\$400, \$140000, \$125000, US\$400, \$95000
Ficha de la 0001	Cliente Ana García y movimientos con signo: + los que entran y - los que salen
Crear la cuenta 0008	Cuenta 0008 creada correctamente. y saldo \$0
Número repetido	Ya existe una cuenta con ese número. (mensaje propio: el de Django sale sin tilde)
Editar	Cliente y moneda aparecen solo como texto; al desmarcar Cuenta activa , la ficha muestra Inactiva
Eliminar una cuenta con movimientos (0001)	Aviso rojo con la sugerencia de desactivarla
Eliminar una cuenta sin movimientos	Cuenta 0008 eliminada correctamente.
CSRF	200, 403 y 403

Alke Wallet					
Inicio Clientes Cuentas Panel de administración					
Cuentas					
Nueva cuenta					
Número	Cliente	Moneda	Saldo	Estado	Acciones
0001	Ana García	CLP	\$115000	Activa	Editar Eliminar
0002	Luis Pérez	CLP	\$60000	Activa	Editar Eliminar
0003	Ana García	USD	US\$400	Activa	Editar Eliminar
0004	Carla Soto	CLP	\$140000	Activa	Editar Eliminar
0005	Diego Rojas	CLP	\$125000	Activa	Editar Eliminar
0006	Diego Rojas	USD	US\$400	Activa	Editar Eliminar
0007	Marta Vega	CLP	\$95000	Activa	Editar Eliminar

Figura 68. Listado de cuentas

Alke Wallet			
Inicio Clientes Cuentas Panel de administración			
Cuenta 0001			
Editar Eliminar Volver al listado			
• Cliente: Ana García • Moneda: CLP (Peso chileno) • Saldo: \$115000 • Estado: Activa • Creada el: 29/09/2026 13:05			
Movimientos			
Fecha	Tipo	Monto	Descripción
29/09/2026 13:05	Transferencia	-30000,00	-
29/09/2026 13:05	Depósito	+100000,00	-
25/09/2026 20:27	Depósito	+50000,00	Sueldo Ana
15/09/2026 20:27	Transferencia	+10000,00	Pago de cuenta
10/09/2026 20:27	Transferencia	-15000,00	Devolución a Carla

Alke Wallet · Proyecto del Módulo 7

Figura 69. Ficha de una cuenta

Alke Wallet	
Inicio Clientes Cuentas Panel de administración	
Nueva cuenta	
Cliente:	- Select an option - v
Moneda:	- Select an option - v
Número de cuenta:	0008
Cuenta activa:	☒
Guardar	Cancelar
Alke Wallet · Proyecto del Módulo 7	

Figura 70. Formulario de nueva cuenta

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Panel de administración](#)

Cuenta 0008 creada correctamente.

Cuenta 0008

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Cliente: [Ana García](#)
- Moneda: CLP (Peso chileno)
- Saldo: \$0
- Estado: Activa
- Creada el: 05/10/2026 18:05

Movimientos

Esta cuenta todavía no tiene movimientos.

Alke Wallet · Proyecto del Módulo 7

Figura 71. Cuenta creada

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Panel de administración](#)

Cuenta 0008 actualizada correctamente.

Cuenta 0008

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Cliente: [Ana García](#)
- Moneda: CLP (Peso chileno)
- Saldo: \$0
- Estado: Inactiva
- Creada el: 05/10/2026 18:05

Movimientos

Esta cuenta todavía no tiene movimientos.

Alke Wallet · Proyecto del Módulo 7

Figura 72. Cuenta desactivada tras editarla

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Panel de administración](#)

No se puede eliminar la cuenta 0001: tiene movimientos registrados. Puedes desactivarla desde Editar.

Cuenta 0001

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Cliente: [Ana García](#)
- Moneda: CLP (Peso chileno)
- Saldo: \$115000
- Estado: Activa
- Creada el: 29/09/2026 13:05

Movimientos

Fecha	Tipo	Monto	Descripción
29/09/2026 13:05	Transferencia	-30000,00	-
29/09/2026 13:05	Depósito	+100000,00	-
25/09/2026 20:27	Depósito	+50000,00	Sueldo Ana
15/09/2026 20:27	Transferencia	+10000,00	Pago de cuenta
10/09/2026 20:27	Transferencia	-15000,00	Devolución a Carla

Alke Wallet · Proyecto del Módulo 7

Figura 73. Eliminación protegida de una cuenta con movimientos

```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> # Cliente de pruebas que SÍ exige el token CSRF, como un navegador real
>>> from django.test import Client
>>> c = Client(enforce_csrf_checks=True, HTTP_HOST='localhost')
>>>
>>> # Pedir el formulario funciona; enviarlo o borrar sin token es rechazado
>>> print('GET formulario ->', c.get('/cuentas/nueva/').status_code)
GET formulario -> 200
>>> print('POST sin token ->', c.post('/cuentas/nueva/', {'numero': 'x'}).status_code)
Forbidden (CSRF token missing.): /cuentas/nueva/
POST sin token -> 403
>>> print('POST borrar sin token ->', c.post('/cuentas/1/eliminar/').status_code)
Forbidden (CSRF token missing.): /cuentas/1/eliminar/
POST borrar sin token -> 403
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...

```

Figura 74. Protección CSRF en cuentas

6.5 Transacciones y registro atómico

Todo movimiento creado desde las pantallas pasa por `gestion/servicios.py`:

```

def registrar_transaccion(tipo, cuenta_origen=None, cuenta_destino=None,
monto=None, descripcion=''):
    # atomic: todo lo que ocurre dentro se guarda junto o se deshace junto
    with transaction.atomic():
        # Si el dinero sale de una cuenta, se bloquea esa cuenta hasta terminar y
        # se vuelve a leer
        if cuenta_origen is not None:
            cuenta_origen =
Cuenta.objects.select_for_update().get(pk=cuenta_origen.pk)
        # La cuenta destino se vuelve a leer sin bloqueo, para conocer su estado
        # actual
        if cuenta_destino is not None:
            cuenta_destino = Cuenta.objects.get(pk=cuenta_destino.pk)
        # Arma el movimiento, repite todas las validaciones con la cuenta
        # bloqueada y lo guarda
        movimiento = Transaccion(tipo=tipo, cuenta_origen=cuenta_origen,
cuenta_destino=cuenta_destino,
                                monto=monto, descripcion=descripcion)
        movimiento.full_clean()
        movimiento.save()
    return movimiento

```

El formulario valida al enviarse; el servicio **repite las validaciones con la cuenta ya bloqueada**, porque entre una y otra el saldo pudo cambiar. Solo se bloquea la cuenta de origen: una cuenta destino solo recibe dinero y no corre riesgo de quedar en negativo.

`select_for_update()` bloquea la fila en PostgreSQL, el motor de producción. SQLite no tiene bloqueos por fila y lo ignora, lo cual no afecta el desarrollo.

El listado usa un formulario de filtros enviado por **GET**: los filtros viajan en la dirección (`?tipo=retiro&cuenta=0004`) y por eso no necesitan token CSRF, mientras que el formulario que escribe datos sí lo lleva. La paginación (10 por página) conserva los filtros en sus enlaces.

Prueba	Resultado
Listado	18 movimiento(s) encontrado(s), 10 filas y Página 1 de 2; la página 2 tiene 8

Prueba	Resultado
Tipo Retiro / Depósito / Transferencia	3 / 8 / 7 movimientos
Cuenta 0004	6 movimientos
Cuenta 0004 y tipo Transferencia	4 movimientos
Cuenta 9999	0 movimientos y el aviso No hay movimientos con esos filtros
Fecha mal escrita en la dirección	Aviso de error en el filtro y listado completo
Ficha de un movimiento	Tipo, fecha, monto, descripción y cuentas con enlace
/transacciones/<pk>/editar/	Error 404: el historial no se edita ni se elimina
Selector de cuentas	Solo cuentas activas, escritas como 0004 · Carla Soto (CLP)
Depósito válido	Movimiento registrado: Depósito de 25000.
Retiro mayor que el saldo	Saldo insuficiente: la cuenta origen tiene 165000.00. No se guarda
Transferencia entre monedas distintas	Las cuentas deben tener la misma moneda.
Depósito con cuenta origen	Un depósito no debe tener cuenta origen.
Transferencia a la misma cuenta	La cuenta origen y la destino deben ser distintas.
Monto cero	Asegúrese de que este valor es mayor o igual a 0.01.

Validación repetida. Para simular un formulario enviado con datos que ya quedaron viejos, se registran dos retiros de 100000 desde una cuenta con 125000, dentro de una operación que se deshace al final:

Paso	Resultado
Saldo inicial de la 0005	125000
Primer retiro de 100000	Registrado; saldo 25000
Segundo retiro con los mismos datos	Saldo insuficiente: la cuenta origen tiene 25000.00.
Al final	Saldo 125000 y 18 movimientos: no queda nada

CSRF: GET del formulario 200, GET del listado con filtros 200, y POST sin token 403.

Alke Wallet						
Inicio Clientes Cuentas Transacciones Panel de administración						
Transacciones						
Nuevo movimiento						
Tipo: Todos						
Número de cuenta:						
Desde:						
Hasta:						
Filtrar Limpiar filtros						
18 movimiento(s) encontrado(s).						
Fecha	Tipo	Origen	Destino	Monto	Descripción	
29/09/2026 20:27	Depósito	-	0002	\$30000,00	Depósito Luis	Ver
29/09/2026 13:05	Transferencia	0001	0002	\$30000,00	-	Ver
29/09/2026 13:05	Depósito	-	0001	\$100000,00	-	Ver
28/09/2026 20:27	Retiro	0007	-	\$5000,00	Retiro cajero Marta 2	Ver
27/09/2026 20:27	Transferencia	0005	0004	\$10000,00	Ajuste entre amigos	Ver
25/09/2026 20:27	Depósito	-	0001	\$50000,00	Sueldo Ana	Ver
23/09/2026 20:27	Transferencia	0006	0003	US\$100,00	Envío USD a Ana	Ver
20/09/2026 20:27	Retiro	0007	-	\$20000,00	Retiro cajero Marta	Ver

Figura 75. Listado de transacciones

Alke Wallet
Inicio Clientes Cuentas Transacciones Panel de administración

Transacciones
Nuevo movimiento

Tipo: Retiro
Número de cuenta:
Desde: dd-mm-aaaa
Hasta: dd-mm-aaaa
Filtrar Limpiar filtros

3 movimiento(s) encontrado(s).

Fecha	Tipo	Origen	Destino	Monto	Descripción	
28/09/2026 20:27	Retiro	0007	-	\$5000,00	Retiro cajero Marta 2	Ver
20/09/2026 20:27	Retiro	0007	-	\$20000,00	Retiro cajero Marta	Ver
05/09/2026 19:27	Retiro	0004	-	\$50000,00	Retiro cajero Carla	Ver

Alke Wallet · Proyecto del Módulo 7

Figura 76. Listado filtrado

Alke Wallet
Inicio Clientes Cuentas Transacciones Panel de administración

Transacciones
Nuevo movimiento

Tipo: Todos
Número de cuenta:
Desde: dd-mm-aaaa
Hasta: dd-mm-aaaa
Filtrar Limpiar filtros

18 movimiento(s) encontrado(s).

Fecha	Tipo	Origen	Destino	Monto	Descripción	
05/09/2026 19:27	Retiro	0004	-	\$50000,00	Retiro cajero Carla	Ver
31/08/2026 19:27	Transferencia	0005	0007	\$40000,00	Arriendo compartido	Ver
26/08/2026 19:27	Transferencia	0004	0005	\$25000,00	Pago almuerzo	Ver
21/08/2026 19:27	Depósito	-	0003	US\$300,00	Depósito USD Ana	Ver
16/08/2026 19:27	Depósito	-	0006	US\$500,00	Depósito USD Diego	Ver
11/08/2026 19:27	Depósito	-	0007	\$80000,00	Depósito inicial Marta	Ver
06/08/2026 19:27	Depósito	-	0005	\$150000,00	Depósito inicial Diego	Ver
01/08/2026 19:27	Depósito	-	0004	\$200000,00	Depósito inicial Carla	Ver

Anterior Página 2 de 2

Alke Wallet · Proyecto del Módulo 7

Figura 77. Página 2 del listado

Alke Wallet
Inicio Clientes Cuentas Transacciones Panel de administración

Movimiento registrado: Transferencia de 25000.

Movimiento 22
Volver al listado

- Tipo: Transferencia
- Fecha: 05/10/2026 18:44
- Monto: **\$25000,00**
- Descripción: Pago almuerzo
- Cuenta origen: **0004** (Carla Soto)
- Cuenta destino: **0005** (Diego Rojas)

Los movimientos forman parte del historial: no se editan ni se eliminan.

Alke Wallet · Proyecto del Módulo 7

Figura 78. Ficha de un movimiento

Alke Wallet

[Inicio](#) [Clientes](#) [Cuentas](#) [Transacciones](#) [Panel de administración](#)

Nuevo movimiento

En un retiro o una transferencia, el monto no puede superar el saldo de la cuenta origen.

Tipo de movimiento: Depósito: solo cuenta destino. Retiro: solo cuenta origen. Transferencia: las dos.

Cuenta origen:

Cuenta destino:

Monto:

Descripción:

[Cancelar](#)

Alke Wallet · Proyecto del Módulo 7

Figura 79. Formulario de nuevo movimiento

Alke Wallet

[Inicio](#) [Clientes](#) [Cuentas](#) [Transacciones](#) [Panel de administración](#)

Movimiento registrado: Depósito de 25000.

Movimiento 23

[Volver al listado](#)

- Tipo: Depósito
- Fecha: 05/10/2026 18:49
- Monto: **\$25000,00**
- Descripción: Prueba 6.5 depósito
- Cuenta origen: -
- Cuenta destino: **0004** (Carla Soto)

Los movimientos forman parte del historial: no se editan ni se eliminan.

Alke Wallet · Proyecto del Módulo 7

Figura 80. Movimiento registrado

Alke Wallet

[Inicio](#) [Clientes](#) [Cuentas](#) [Transacciones](#) [Panel de administración](#)

Nuevo movimiento

En un retiro o una transferencia, el monto no puede superar el saldo de la cuenta origen.

- **Saldo insuficiente: la cuenta origen tiene 140000.00.**

Tipo de movimiento: Depósito: solo cuenta destino. Retiro: solo cuenta origen. Transferencia: las dos.

Cuenta origen:

Cuenta destino:

Monto:

Descripción:

[Cancelar](#)

Alke Wallet · Proyecto del Módulo 7

Figura 81. Saldo insuficiente

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Nuevo movimiento

En un retiro o una transferencia, el monto no puede superar el saldo de la cuenta origen.

- Las cuentas deben tener la misma moneda.

Tipo de movimiento: Transferencia Depósito: solo cuenta destino. Retiro: solo cuenta origen. Transferencia: las dos.

Cuenta origen: 0004 - Carla Soto (CLP)

Cuenta destino: 0003 - Ana García (USD)

Monto:

Descripción:

Alke Wallet · Proyecto del Módulo 7

Figura 82. Error de validación en el formulario

```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Herramientas de la prueba
>>> from django.db import transaction
>>> from django.core.exceptions import ValidationError
>>> from gestion.models import Cuenta, Transaccion
>>> from gestion.servicios import registrar_transaccion
>>>
>>> # Excepción creada solo para deshacer toda la prueba al final
>>> class Deshacer(Exception):
...     pass
...
>>> # Cuenta de Diego, con saldo 125000
>>> c5 = Cuenta.objects.get(numero='0005')
>>> print('saldo inicial 0005:', c5.saldo)
saldo inicial 0005: 150000
>>>
>>> try:
...     # Todo lo de este bloque se deshacerá al final
...     with transaction.atomic():
...         # Los mismos datos de un formulario enviado dos veces
...         datos = dict(tipo='retiro', cuenta_origen=c5, monto='100000')
...         # El primer retiro es válido
...         registrar_transaccion(**datos)
...         print('primer retiro: ok, saldo', Cuenta.objects.get(pk=c5.pk).saldo)
...         try:
...             # El segundo, con los mismos datos "viejos", debe ser rechazado
...             registrar_transaccion(**datos)
...         except ValidationError as e:
...             print('segundo retiro:', e.messages)
...             # Fuerza el deshacer de todo lo anterior
...             raise Deshacer
... except Deshacer:
...     pass
...
<Transaccion: Retiro de 100000>
primer retiro: ok, saldo 50000
segundo retiro: ['Saldo insuficiente: la cuenta origen tiene 50000.00.']
>>> # Comprueba que no quedó nada
>>> print('saldo final 0005:', Cuenta.objects.get(pk=c5.pk).saldo)
saldo final 0005: 150000
>>> print('movimientos:', Transaccion.objects.count())
movimientos: 19
>>>
>>> # Sale de la shell
>>> exit()

```

Figura 83. Validación repetida dentro de la operación atómica

```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> # Cliente de pruebas que SÍ exige el token CSRF, como un navegador real
>>> from django.test import Client
>>> c = Client(enforce_csrf_checks=True, HTTP_HOST='localhost')
>>>
>>> # Pedir el formulario funciona, y el listado con filtros (GET) tampoco necesita token
>>> print('GET formulario ->', c.get('/transacciones/nueva/').status_code)
GET formulario -> 200
>>> print('GET listado con filtros ->', c.get('/transacciones/?tipo=retiro').status_code)
GET listado con filtros -> 200
>>>
>>> # Enviar el formulario sin token es rechazado
>>> print('POST sin token ->', c.post('/transacciones/nueva/', {'tipo': 'deposito'}).status_code)
Forbidden (CSRF token missing.): /transacciones/nueva/
POST sin token -> 403
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...

```

Figura 84. Protección CSRF en transacciones

6.6 Contactos

Los contactos se agregan y se quitan desde la ficha del cliente. El dueño de la agenda sale de la dirección (/clientes/3/contactos/nuevo/), y el formulario solo pide a quién agendar y el apodo. El selector excluye al dueño y a quienes ya están en su agenda; la restricción de la base de datos queda como respaldo.

El atajo **Transferir** de cada contacto abre el formulario de movimiento con la transferencia y la primera cuenta activa del contacto ya elegidas, usando una dirección con parámetros (?tipo=transferencia&cuenta_destino=2). No registra nada por sí solo.

Prueba	Resultado
Ficha de Ana García	Contactos Lucho (Luis Pérez) y Carli (Carla Soto), cada uno con Transferir y Quitar ; en Aparece en la agenda de : Luis Pérez
Formulario de agregar contacto en Ana	Solo ofrece a Diego Rojas y Marta Vega
Agregar a Diego con apodo	Contacto «Diego Rojas» agregado correctamente. y deja de aparecer en el selector
Agregar sin apodo	Se guarda; el apodo se ve como un guion
Quitar un contacto	Contacto «Diego Rojas» quitado de la agenda. El cliente y sus cuentas siguen existiendo
Cliente inexistente (/clientes/9999/contactos/nuevo/)	Error 404
Atajo Transferir	El formulario se abre con Transferencia y la cuenta destino elegidas
CSRF	200, 403 y 403

La agenda no es simétrica: que Ana tenga agendado a Luis no hace que Luis tenga agendada a Ana. La sección **Aparece en la agenda de** usa la relación inversa `agendado_por`, definida en la etapa 2.

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Ana García

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Correo electrónico: ana@example.com
- Teléfono: 123456789
- Usuario de acceso: ana
- Registrado el: 29/09/2026 13:05

Cuentas

[Nueva cuenta para este cliente](#)

Número	Moneda	Saldo	Estado
0001	CLP	\$115000	Activa
0003	USD	US\$400	Activa
0008	CLP	\$0	Inactiva

Contactos

[Agregar contacto](#)

Apodo	Nombre	Acciones
Lucho	Luis Pérez	Transferir Quitar
Carli	Carla Soto	Transferir Quitar

Aparece en la agenda de

- [Luis Pérez](#)

Figura 85. Contactos en la ficha de un cliente

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Agregar contacto

Agenda de **Ana García**.

Cliente a agendar:

Apodo (opcional):

Alke Wallet · Proyecto del Módulo 7

Figura 86. Formulario de agregar contacto

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Contacto «Diego Rojas» agregado correctamente.

Ana García

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Correo electrónico: ana@example.com
- Teléfono: 123456789
- Usuario de acceso: ana
- Registrado el: 29/09/2026 13:05

Figura 87. Contacto agregado

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Quitar contacto

¿Seguro que quieres quitar a **Diego Rojas** de la agenda de **Ana García**?

Solo se quita de la agenda: el cliente y sus cuentas no se eliminan.

Alke Wallet · Proyecto del Módulo 7

Figura 88. Confirmación para quitar un contacto

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Contacto «Diego Rojas» quitado de la agenda.

Ana García

[Editar](#)
[Eliminar](#)
[Volver al listado](#)

- Correo electrónico: ana@example.com
- Teléfono: 123456789
- Usuario de acceso: ana
- Registrado el: 29/09/2026 13:05

Cuentas

[Nueva cuenta para este cliente](#)

Número	Moneda	Saldo	Estado
0001	CLP	\$115000	Activa
0003	USD	US\$400	Activa
0008	CLP	\$0	Inactiva

Figura 89. Contacto quitado

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Panel de administración](#)

Nuevo movimiento

En un retiro o una transferencia, el monto no puede superar el saldo de la cuenta origen.

Tipo de movimiento: Depósito: solo cuenta destino. Retiro: solo cuenta origen. Transferencia: las dos.

Cuenta origen:

Cuenta destino:

Monto:

Descripción:

Alke Wallet · Proyecto del Módulo 7

Figura 90. Atajo Transferir: formulario con la transferencia y el destino elegidos

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python .\manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>> # Cliente de pruebas que SÍ exige el token CSRF, como un navegador real
>>> from django.test import Client
>>> from gestion.models import Cliente, Contacto
>>>
>>> # Un cliente con contactos y una de sus fichas
>>> ana = Cliente.objects.get(email='ana@example.com')
>>> ficha = Contacto.objects.filter(proprietario=ana).first()
>>> c = Client(enforce_csrf_checks=True, HTTP_HOST='localhost')
>>>
>>> # Pedir el formulario funciona; agregar o quitar sin token es rechazado
>>> print('GET formulario ->', c.get(f'/clientes/{ana.pk}/contactos/nuevo/').status_code)
GET formulario -> 200
>>> print('POST agregar sin token ->', c.post(f'/clientes/{ana.pk}/contactos/nuevo/', {'agendado': 4}).status_code)
Forbidden (CSRF token missing.): /clientes/1/contactos/nuevo/
POST agregar sin token -> 403
>>> print('POST quitar sin token ->', c.post(f'/contactos/{ficha.pk}/eliminar/').status_code)
Forbidden (CSRF token missing.): /contactos/1/eliminar/
POST quitar sin token -> 403
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...
```

Figura 91. Protección CSRF en contactos

6.7 Inicio y reporte

La página de inicio muestra los totales (con enlace a cada listado), los 5 movimientos más recientes y enlaces rápidos. El reporte reutiliza las consultas de `gestion/consultas.py`, que ahora tiene 12 funciones:

Dato del reporte	Consulta
Movimientos por tipo, por moneda	<code>resumen_por_tipo()</code> (etapa 4)
Saldo total por cliente, por moneda	<code>saldo_por_cliente(): values + annotate(Sum('saldo_calculado'))</code> sobre la consulta de saldos
Movimientos por mes	<code>movimientos_por_mes(): TruncMonth('fecha') + Count</code>

El reporte tiene una sección por moneda, porque los montos de monedas distintas no se suman entre sí. Agrupa por el `id` del cliente y no por su nombre, para que dos clientes con el mismo nombre no se mezclen.

Sección	Resultado
Inicio	Cientes 5, Cuentas 7, Transacciones 18 y los 5 últimos movimientos
CLP, movimientos por tipo	Depósito 6 (\$610000), Retiro 3 (\$75000), Transferencia 6 (\$130000)
CLP, saldo por cliente	Carla Soto \$140000, Diego Rojas \$125000, Ana García \$115000, Marta Vega \$95000, Luis Pérez \$60000; total \$535000
USD, movimientos por tipo	Depósito 2 (US\$800), Transferencia 1 (US\$100)
USD, saldo por cliente	Ana García US\$400 y Diego Rojas US\$400; total US\$800
Movimientos por mes	Una fila por mes; la columna Cantidad suma 18, igual que el total

Las cifras coinciden con las del comando `demo_consultas` de la etapa 4.

Alke Wallet

[Inicio](#)
[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Reporte](#)
[Panel de administración](#)

Bienvenido a Alke Wallet

Resumen de lo que hay registrado en el sistema:

- [Clientes](#): 7
- [Cuentas](#): 8
- [Transacciones](#): 19

Últimos movimientos

Fecha	Tipo	Origen	Destino	Monto
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00
29/09/2026 20:27	Depósito	-	0002	\$30000,00
29/09/2026 13:05	Transferencia	0001	0002	\$30000,00
29/09/2026 13:05	Depósito	-	0001	\$100000,00
28/09/2026 20:27	Retiro	0007	-	\$5000,00

[Registrar un movimiento](#) ·
 [Ver todos los movimientos](#) ·
 [Ver el reporte](#)

Figura 92. Inicio con totales y últimos movimientos

Reporte		
Los montos de monedas distintas no se suman entre sí: cada moneda tiene su propia sección.		
CLP · Peso chileno		
Movimientos por tipo		
Tipo	Cantidad	Monto total
Depósito	6	\$610000
Retiro	3	\$75000
Transferencia	7	\$155000
Saldo por cliente		
Cliente	Saldo	
Diego Rojas	\$150000	
Ana García	\$115000	
Carla Soto	\$115000	
Marta Vega	\$95000	
Luis Pérez	\$60000	
Total	\$535000	

Figura 93. Reporte, sección CLP

USD · Dólar estadounidense

Movimientos por tipo

Tipo	Cantidad	Monto total
Depósito	2	US\$800
Transferencia	1	US\$100

Saldo por cliente

Cliente	Saldo
Ana García	US\$400
Diego Rojas	US\$400
Total	US\$800

Movimientos por mes

Mes	Cantidad
agosto 2026	7
septiembre 2026	11
octubre 2026	1
Total	19

Alke Wallet · Proyecto del Módulo 7

Figura 94. Reporte, sección USD y movimientos por mes

Decisiones y alternativas descartadas

Tema	Elegida	Motivo
Qué lleva CRUD completo	Cientes y cuentas. Las transacciones solo se listan, se ven y se crean	Los movimientos son el historial y el saldo se calcula con ellos
Regla del saldo	En <code>Transaccion.clean()</code> , solo al crear	Una sola regla para el panel y para las pantallas
Registro de movimientos	A través de <code>registrar_transaccion()</code> , con <code>transaction.atomic()</code> y bloqueo de la cuenta de origen	Evita que dos retiros simultáneos dejen una cuenta en negativo
Cuentas inactivas	No pueden enviar ni recibir movimientos nuevos	El campo <code>activa</code> ya existía y no tenía efecto

Tema	Elegida	Motivo
Crear un cliente	Selector con los usuarios que aún no tienen cliente	Más simple que crear usuario y cliente a la vez
Borrar con historial	Se captura ProtectedError y se muestra un mensaje claro	Evita una página de error 500
Templates	Carpeta <code>templates/</code> en la raíz, con <code>base.html</code> compartido	Coincide con la consigna y comparte el diseño
Coherencia con el panel	Los movimientos son de solo lectura en el panel	Las mismas reglas en todos los accesos

Incidencias y solución

Origen: 6.1 Reglas de saldo y cuentas activas

Incidencia resuelta: al pegar el método `clean()` nuevo, quedó con 4 espacios de más, dentro de `class Meta`. Django lo rechazó al arrancar con `'class Meta' got invalid attribute(s): clean`. `Meta` solo acepta opciones de configuración; `clean()` es un método del modelo y se alinea con `class Meta`, no dentro de ella.

Origen: 6.5 Transacciones y registro atómico

Incidencia resuelta: al compactar un `import` a una sola línea quedó una coma al final antes del comentario: `from .forms import ..., FiltroTransaccionForm, # ....`. Python rechaza con `trailing comma not allowed without surrounding parentheses`, porque esa coma solo es válida si la lista va entre paréntesis.

Limitaciones conocidas

- **Todavía no hay login.** Cualquier persona que llegue a la dirección ve todas las pantallas, incluido el enlace al panel. *(Resuelto en la etapa 7.)*
- **El superusuario no tiene un Cliente asociado.** Las pantallas de usuario que dependan de `request.user.cliente` deberán contemplar ese caso. *(Resuelto en la etapa 7: el personal ve todo y un usuario sin cliente recibe un aviso.)*
- **El diseño es mínimo.** Hay un bloque de estilos temporal dentro de `base.html`, que la etapa 7 reemplaza por archivos estáticos. *(Resuelto en la etapa 7.)*

Reflexiones

- **Las vistas genéricas ahorran mucho código:** `ListView`, `DetailView`, `CreateView`, `UpdateView` y `DeleteView` resuelven lo repetitivo, y cada vista solo declara lo que la distingue.
- **Una sola regla, todos los accesos:** poner el saldo y las cuentas inactivas en `Transaccion.clean()` hace que el panel y las pantallas las apliquen sin duplicar código.
- **Validar dos veces tiene sentido:** el formulario da una respuesta rápida y el servicio, con la cuenta bloqueada, da la definitiva.
- **El historial no se edita:** no hay pantallas ni permisos para modificar movimientos; un error se corrige con un movimiento inverso.
- **Reutilizar consultas paga:** el reporte, el panel y el comando de demostración comparten `cuentas_con_saldo()`, y sus cifras coinciden.

Etapa 7: autenticación y archivos estáticos

Objetivo

Que solo entren personas con sesión iniciada, que cada rol vea lo que le corresponde, que cada cliente vea únicamente sus propios datos, y reemplazar los estilos provisionales por archivos estáticos.

Rama de trabajo: `feature/auth`. Esta etapa no cambia los modelos, por lo que no genera migraciones.

Qué se hizo

Subetapas:

Subetapa	Qué se hizo
7.1	Login y logout con las vistas de <code>django.contrib.auth</code>
7.2	Archivos estáticos: CSS y logo
7.3	Roles: personal, cliente y usuario sin cliente
7.4	Alcance por usuario: cada cliente ve solo lo suyo
7.5	Registro público de clientes con su primera cuenta
7.6	Mi perfil y cambio de contraseña
7.7	Verificación de accesos y de la protección CSRF

7.1 Login y logout

Django 5.1 agregó `LoginRequiredMiddleware`: con él, **todas** las pantallas piden sesión por defecto y solo se publican las que se marcan con `login_not_required`. Es más seguro que proteger cada vista por separado, porque una vista nueva queda protegida aunque se olvide.

Pieza	Dónde	Qué hace
<code>LoginRequiredMiddleware</code>	MIDDLEWARE, después de <code>AuthenticationMiddleware</code>	Redirige al login a quien no tiene sesión
<code>LOGIN_URL = 'login'</code>	<code>settings.py</code>	A dónde se envía a quien no ha entrado
<code>LOGIN_REDIRECT_URL = 'gestion:inicio'</code>	<code>settings.py</code>	A dónde se va después de entrar
<code>LOGOUT_REDIRECT_URL = 'login'</code>	<code>settings.py</code>	A dónde se va después de salir
<code>/acceso/login/</code>	<code>core/urls.py</code> , <code>LoginView</code>	Pantalla de ingreso; si ya hay sesión, va al inicio
<code>/acceso/logout/</code>	<code>core/urls.py</code> , <code>LogoutView</code>	Cierra la sesión; solo acepta POST

Cuando alguien intenta entrar a una pantalla sin sesión, el login recibe `?next=/ruta/original/` y, tras entrar, lo devuelve a esa pantalla. El botón «Cerrar sesión» es un formulario POST con token CSRF y no un enlace: así un sitio externo no puede cerrar la sesión de otra persona con solo enlazar una dirección.

Alke Wallet

[Iniciar sesión](#)

Iniciar sesión

Ingresa con tu usuario y tu contraseña para ver tus cuentas y movimientos.

Nombre de usuario:

Contraseña:

Alke Wallet · Proyecto del Módulo 7

Figura 95. Formulario de ingreso

Alke Wallet

[Iniciar sesión](#)

Iniciar sesión

Ingresa con tu usuario y tu contraseña para ver tus cuentas y movimientos.

- Por favor, introduzca un nombre de usuario y clave correctos. Observe que ambos campos pueden ser sensibles a mayúsculas.

Nombre de usuario:

Contraseña:

Alke Wallet · Proyecto del Módulo 7

Figura 96. Error de credenciales

Alke Wallet

[Inicio](#) [Clientes](#) [Cuentas](#) [Transacciones](#) [Reporte](#) [Panel de administración](#) Sesión: **admin** [Cerrar sesión](#)

Clientes

[Nuevo cliente](#)

Nombre	Correo electrónico	Teléfono	Cuentas	Acciones
Ana García	ana@example.com	123456789	3	Editar Eliminar
Carla Soto	carla@example.com	987654321	1	Editar Eliminar
Diego Rojas	diego@example.com	-	2	Editar Eliminar
Luis Pérez	luis@example.com	-	1	Editar Eliminar
María Vega	maria@example.com	-	1	Editar Eliminar
Pedro	Pedro@example.com	66556677	0	Editar Eliminar
Tomas Prueba	tomas@example.com	555123456	0	Editar Eliminar

Alke Wallet · Proyecto del Módulo 7

Figura 97. Inicio con la sesión iniciada

Alke Wallet

[Iniciar sesión](#)

Iniciar sesión

Ingresa con tu usuario y tu contraseña para ver tus cuentas y movimientos.

Nombre de usuario:

Contraseña:

Alke Wallet · Proyecto del Módulo 7

Figura 98. Redirección al login con ?next=

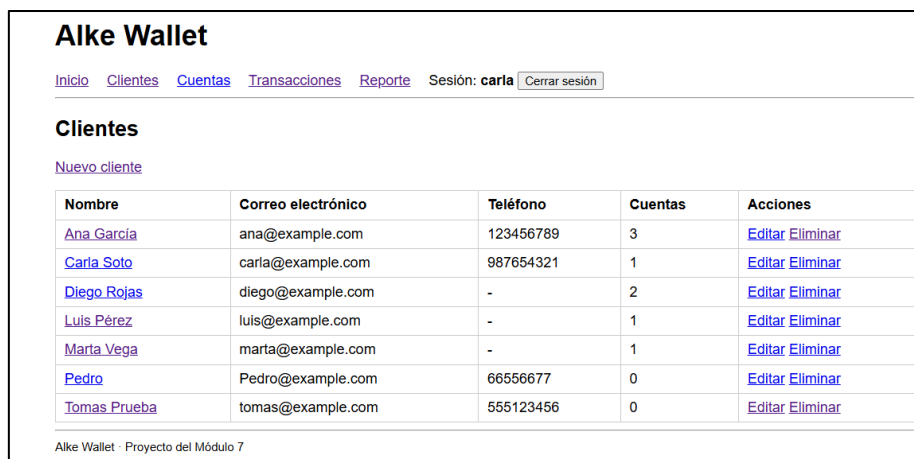


Figura 99. Menú de un cliente, sin enlace al panel

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> # Cliente de pruebas que SÍ exige el token CSRF, como un navegador real
>>> from django.test import Client
>>> c = Client(enforce_csrf_checks=True, HTTP_HOST='localhost')
>>>
>>> # Pedir el formulario de login funciona; enviarlo o cerrar sesión sin token es rechazado
>>> print('GET login ->', c.get('/acceso/login/').status_code)
GET login -> 200
>>> print('POST login sin token ->', c.post('/acceso/login/', {'username': 'x', 'password': 'x'}).status_code)
Forbidden (CSRF token missing.): /acceso/login/
POST login sin token -> 403
>>> print('POST logout sin token ->', c.post('/acceso/logout/').status_code)
Forbidden (CSRF token missing.): /acceso/logout/
POST logout sin token -> 403
>>>
>>> # Sale de la shell
>>> exit()
now exiting InteractiveConsole...
```

Figura 100. El login sin token CSRF es rechazado con 403

7.2 Archivos estáticos

Los estilos pasaron del bloque provisorio de `base.html` a `static/css/estilos.css`, y se agregó `static/img/logo.svg`.

Configuración	Para qué sirve
<code>STATIC_URL = 'static/'</code>	Prefijo de las direcciones de los archivos estáticos
<code>STATICFILES_DIRS = [BASE_DIR / 'static']</code>	Carpeta del proyecto donde están los archivos propios
<code>STATIC_ROOT = BASE_DIR / 'staticfiles'</code>	Carpeta donde <code>collectstatic</code> junta todo para producción

En los templates se usa `{% load static %}` y `{% static 'css/estilos.css' %}`, de modo que la dirección real la calcula Django. Dos comandos permiten comprobar la configuración:

Comandos: ver «Comandos», apartado «7.2 Archivos estáticos».

`staticfiles/` se genera con `collectstatic` y no se sube al repositorio.

Nota: los estilos de esta etapa se reemplazaron después, en la etapa 9, por una capa de marca sobre Bootstrap. Las capturas siguientes muestran el diseño original.

```

/* =====
Alke Wallet: estilos generales de todas las pantallas
===== */

/* Variables: colores y medidas que se definen una vez y se usan en todo el archivo */
:root {
  --color-principal: #1d4e89;
  --color-principal-oscuro: #153a68;
  --color-fondo: #eef2f7;
  --color-texto: #1f2933;
  --color-gris: #667085;
  --color-borde: #d5dbe3;
  --color-fila: #f6f8fb;
  --color-exito: #2f9e5a;
  --color-exito-fondo: #e1f5e8;
  --color-error: #c0392b;
  --color-error-fondo: #fde8e6;
  --color-aviso: #d9a400;
  --color-aviso-fondo: #fff6d9;
  --color-info-fondo: #e6eefa;
  --radio: 6px;
}

/* El fondo de toda la página es un gris azulado suave */
html {
  background: var(--color-fondo);
}

```

Figura 101. Estructura de la carpeta static

```

(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Comprueba que Django encuentre los dos archivos estáticos
>> python manage.py findstatic css/estilos.css img/logo.svg
Found 'css/estilos.css' here:
  D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet\static\css\estilos.css
Found 'img/logo.svg' here:
  D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet\static\img\logo.svg
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py collectstatic
c - -noinput

0 static files copied to 'D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet\staticfiles',
132 unmodified.

```

Figura 102. findstatic y collectstatic

Alke Wallet

[Iniciar sesión](#)

Iniciar sesión

Ingresa con tu usuario y tu contraseña para ver tus cuentas y movimientos.

Nombre de usuario:

Contraseña:

Entrar

Alke Wallet - Proyecto del Módulo 7

Figura 103. Login con estilos

Alke Wallet

Inicio Clientes Cuentas Transacciones Reporte Panel de administración Sesión: **admin** [Cerrar sesión](#)

Bienvenido a Alke Wallet

Resumen de lo que hay registrado en el sistema:

- [Clientes](#): 7
- [Cuentas](#): 8
- [Transacciones](#): 19

Últimos movimientos

Fecha	Tipo	Origen	Destino	Monto
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00
29/09/2026 20:27	Depósito	-	0002	\$30000,00
29/09/2026 13:05	Transferencia	0001	0002	\$30000,00
29/09/2026 13:05	Depósito	-	0001	\$100000,00
28/09/2026 20:27	Retiro	0007	-	\$5000,00

[Registrar un movimiento](#) · [Ver todos los movimientos](#) · [Ver el reporte](#)

Alke Wallet - Proyecto del Módulo 7

Figura 104. Inicio con estilos

Alke Wallet

Inicio Clientes Cuentas Transacciones Reporte Panel de administración Sesión: **admin** [Cerrar sesión](#)

Cuentas

[Nueva cuenta](#)

Número	Cliente	Moneda	Saldo	Estado	Acciones
0001	Ana García	CLP	\$115000	Activa	Editar Eliminar
0002	Luis Pérez	CLP	\$60000	Activa	Editar Eliminar
0003	Ana García	USD	US\$400	Activa	Editar Eliminar
0004	Carla Soto	CLP	\$115000	Activa	Editar Eliminar
0005	Diego Rojas	CLP	\$150000	Activa	Editar Eliminar
0006	Diego Rojas	USD	US\$400	Activa	Editar Eliminar
0007	Marta Vega	CLP	\$95000	Activa	Editar Eliminar
0008	Ana García	CLP	\$0	Inactiva	Editar Eliminar

Alke Wallet - Proyecto del Módulo 7

Figura 105. Listado con estilos

7.3 Roles

Hay tres tipos de persona con sesión. Se distinguen sin crear tablas nuevas: el personal tiene `is_staff` y un cliente tiene un `Cliente` asociado (relación 1:1).

Rol	Cómo se detecta	Qué puede hacer
Personal	<code>user.is_staff</code>	Todo: clientes, cuentas, movimientos, reporte y panel
Cliente	<code>hasattr(user, 'cliente')</code>	Ver su ficha, sus cuentas y sus movimientos; transferir; gestionar su agenda
Sin cliente	Ninguno de los anteriores	Solo ver su perfil y un aviso en el inicio

gestion/mixins.py define dos mixins basados en UserPassesTestMixin, que se ponen **primero** en las clases base de cada vista. Si el test falla, la persona recibe una página 403 propia (templates/403.html).

Mixin	Quién pasa	Vistas
SoloPersonalMixin	Solo el personal	Listado, alta y baja de clientes; alta, edición y baja de cuentas; reporte
PersonalOClienteMixin	Personal y clientes	Ficha y edición de cliente; listado y ficha de cuentas; movimientos; contactos

El menú de base.html también cambia según el rol, pero ocultar un enlace no protege nada: la protección real está en las vistas, y el menú solo evita ofrecer lo que daría 403. hasattr(user, 'cliente') se usa en lugar de user.cliente porque este último lanza una excepción cuando no hay ficha.

```
#gestion/mixins.py
# Mezclas (mixins) para las vistas de la app gestion

# UserPassesTestMixin ejecuta un test antes de mostrar la pantalla; si falla, responde 403
from django.contrib.auth.mixins import UserPassesTestMixin

# Pantallas reservadas al personal (usuarios con is_staff)
class SoloPersonalMixin(UserPassesTestMixin):
    # Django ejecuta este método primero; si devuelve False, la persona recibe un 403
    def test_func(self):
        return self.request.user.is_staff # Solo pasa quien pertenece al personal

# Pantallas para el personal y para los clientes (usuarios que tienen un Cliente asociado)
class PersonalOClienteMixin(UserPassesTestMixin):
    # Django ejecuta este método primero; si devuelve False, la persona recibe un 403
    def test_func(self):
        usuario = self.request.user # Usuario que hace la petición
        return usuario.is_staff or hasattr(usuario, 'cliente') # hasattr devuelve False si el usuario
```

Figura 106. Mixins de roles en mixins.py

```

inicio                302      200      200      200
clientes lista        302      200      403      403
cliente nuevo         302      200      403      403
cliente detalle       302      200      200      403
cliente editar        302      200      200      403
cliente eliminar      302      200      403      403
cuentas lista         302      200      200      403
cuenta detalle       302      200      200      403
cuenta nueva         302      200      403      403
cuenta editar        302      200      403      403
cuenta eliminar      302      200      403      403
transacciones lista   302      200      200      403
transaccion detalle   302      200      200      403
transaccion nueva     302      200      200      403
contacto nuevo        302      200      200      403
contacto eliminar     302      200      200      403
reporte               302      200      403      403
>>> # Sale de la shell
```

Figura 107. Matriz de permisos

Alke Wallet

Inicio Clientes Cuentas Transacciones Reporte Panel de administración Sesión: **admin** (personal) [Cerrar sesión](#)

Bienvenido a Alke Wallet

Resumen de lo que hay registrado en el sistema:

- [Clientes](#): 7
- [Cuentas](#): 8
- [Transacciones](#): 19

Últimos movimientos

Fecha	Tipo	Origen	Destino	Monto
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00
29/09/2026 20:27	Depósito	-	0002	\$30000,00
29/09/2026 13:05	Transferencia	0001	0002	\$30000,00
29/09/2026 13:05	Depósito	-	0001	\$100000,00
28/09/2026 20:27	Retiro	0007	-	\$5000,00

[Registrar un movimiento](#) · [Ver todos los movimientos](#) · [Ver el reporte](#)

Alke Wallet · Proyecto del Módulo 7

Figura 108. Menú del personal

Alke Wallet

Inicio Mi ficha Cuentas Transacciones Sesión: **carla** (cliente) [Cerrar sesión](#)

Bienvenido a Alke Wallet

Resumen de lo que hay registrado en el sistema:

- [Clientes](#): 7
- [Cuentas](#): 8
- [Transacciones](#): 19

Últimos movimientos

Fecha	Tipo	Origen	Destino	Monto
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00
29/09/2026 20:27	Depósito	-	0002	\$30000,00
29/09/2026 13:05	Transferencia	0001	0002	\$30000,00
29/09/2026 13:05	Depósito	-	0001	\$100000,00
28/09/2026 20:27	Retiro	0007	-	\$5000,00

[Registrar un movimiento](#) · [Ver todos los movimientos](#) · [Ver el reporte](#)

Alke Wallet · Proyecto del Módulo 7

Figura 109. Menú de un cliente

Alke Wallet

Inicio Mi ficha Cuentas Transacciones Sesión: **carla** (cliente) [Cerrar sesión](#)

No tienes permiso para ver esta página

Esta sección está reservada para otro tipo de usuario.

[Volver al inicio](#)

Alke Wallet · Proyecto del Módulo 7

Figura 110. Página 403 para un cliente

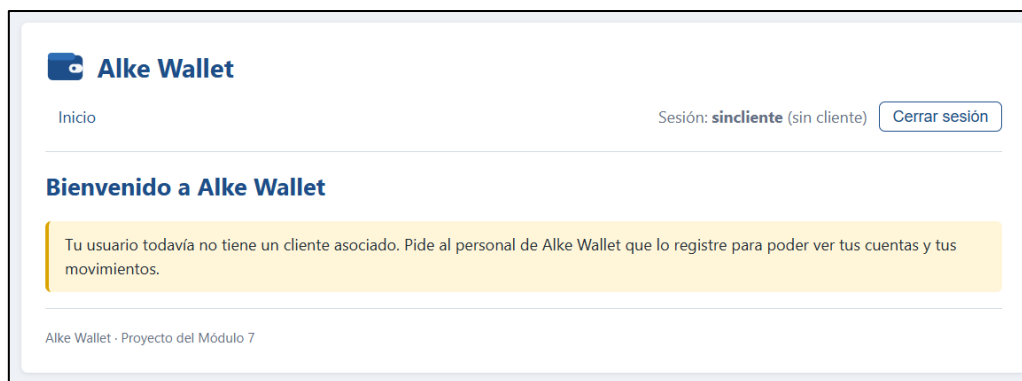


Figura 111. Inicio de un usuario sin cliente

7.4 Alcance por usuario

Los roles dicen **qué pantallas** puede abrir cada persona; el alcance dice **qué registros**. Un cliente solo ve los suyos, y si escribe en la dirección el número de un registro ajeno recibe **404** y no 403, para no confirmar que ese registro existe.

`gestion/alcance.py` concentra las reglas:

Función	Qué devuelve
<code>limitar_a_cliente(queryset, usuario, campo)</code>	Deja solo los registros cuyo campo apunta al cliente de la sesión; el personal ve todo
<code>movimientos_visibles(usuario)</code>	Movimientos donde alguna cuenta del cliente es origen o destino
<code>cuentas_de_origen(usuario)</code>	Cuentas activas desde las que se puede sacar dinero: solo las propias
<code>cuentas_de_destino(usuario)</code>	Cuentas activas que se pueden recibir: las propias y las de sus contactos

Las vistas usan estas funciones en `get_queryset()`, de modo que el listado, la ficha, la edición y el borrado aplican el mismo filtro. Además:

- **Movimientos:** el formulario recibe al usuario (`TransaccionForm(usuario=...)`) y ofrece solo las cuentas permitidas. Un retiro solo sale de cuentas propias, una transferencia puede ir a un contacto, y un depósito debe ir a una cuenta propia (regla en `clean()`).
- **Agenda:** `ContactoCreateView` busca al dueño de la agenda con `cached_property` y no en `dispatch`, para que el rol se revise antes. Con `dispatch`, un usuario sin cliente habría provocado un error 500 en lugar de un 403.
- **Inicio:** el personal ve los totales del sistema; un cliente ve sus cuentas con saldo y sus últimos movimientos.

Caso (cliente <code>carla</code>)	Resultado
<code>/cuentas/</code>	Solo <code>0004</code>
Ficha, cuenta o movimiento de otro cliente	404
Destinos de una transferencia	Sus cuentas y las de sus contactos
Depósito a la cuenta de un contacto	Error «Un depósito solo puede ir a una de tus cuentas.»
Retiro desde una cuenta ajena	Rechazado: la opción no está entre las disponibles


Alke Wallet

[Inicio](#)
[Mi ficha](#)
[Cuentas](#)
[Transacciones](#)

Sesión: **carla** (cliente) [Cerrar sesión](#)

Bienvenido a Alke Wallet

Estas son tus cuentas:

- 0004: \$115000 (CLP)

Últimos movimientos

Fecha	Tipo	Origen	Destino	Monto
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00
27/09/2026 20:27	Transferencia	0005	0004	\$10000,00
15/09/2026 20:27	Transferencia	0004	0001	\$10000,00
10/09/2026 20:27	Transferencia	0001	0004	\$15000,00
05/09/2026 19:27	Retiro	0004	-	\$50000,00

[Ver todos los movimientos](#)

Alke Wallet - Proyecto del Módulo 7

Figura 112. Inicio de un cliente


Alke Wallet

[Inicio](#)
[Mi ficha](#)
[Cuentas](#)
[Transacciones](#)

Sesión: **carla** (cliente) [Cerrar sesión](#)

Cuentas

[Nueva cuenta](#)

Número	Cliente	Moneda	Saldo	Estado	Acciones
0004	Carla Soto	CLP	\$115000	Activa	Editar Eliminar

Alke Wallet - Proyecto del Módulo 7

Figura 113. Cuentas: solo las propias

7 movimiento(s) encontrado(s).

Fecha	Tipo	Origen	Destino	Monto	Descripción	
05/10/2026 18:44	Transferencia	0004	0005	\$25000,00	Pago almuerzo	Ver
27/09/2026 20:27	Transferencia	0005	0004	\$10000,00	Ajuste entre amigos	Ver
15/09/2026 20:27	Transferencia	0004	0001	\$10000,00	Pago de cuenta	Ver
10/09/2026 20:27	Transferencia	0001	0004	\$15000,00	Devolución a Carla	Ver
05/09/2026 19:27	Retiro	0004	-	\$50000,00	Retiro cajero Carla	Ver
26/08/2026 19:27	Transferencia	0004	0005	\$25000,00	Pago almuerzo	Ver
01/08/2026 19:27	Depósito	-	0004	\$200000,00	Depósito inicial Carla	Ver

Alke Wallet - Proyecto del Módulo 7

Figura 114. Movimientos: solo los propios

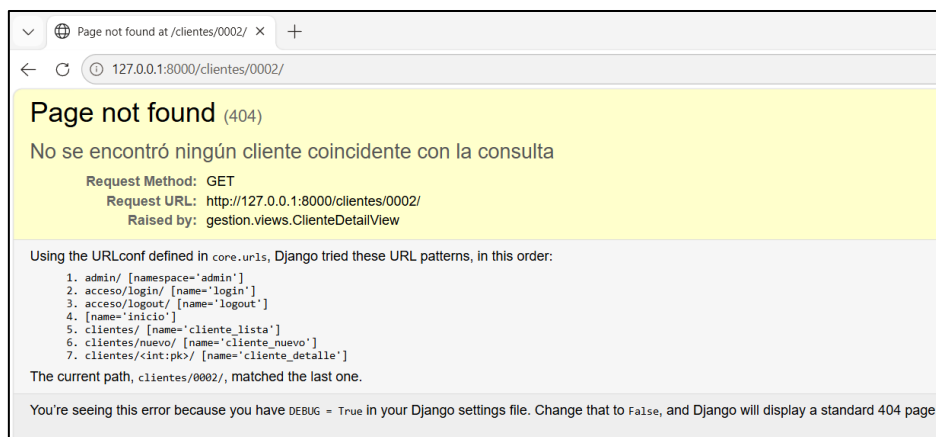


Figura 115. 404 al abrir la ficha de otro cliente

Figura 116. Destinos permitidos en una transferencia

Figura 117. Depósito a una cuenta ajena rechazado

7.5 Registro de clientes

Cualquier persona puede crear su usuario desde `/registro/`. Es, junto con el login, la única pantalla pública: se marca con `login_not_required` en `core/urls.py`.

Pieza	Qué hace
RegistroForm (forms.py)	Extiende UserCreationForm: pide usuario, nombre, correo, teléfono (opcional), moneda y contraseña dos veces
Validaciones	Contraseña no débil ni numérica, claves iguales, correo no repetido (sin distinguir mayúsculas) y moneda elegida
crear_cliente_con_cuenta() (servicios.py)	Crea el cliente y su primera cuenta con saldo cero y el siguiente número libre (0008, 0009...)
RegistroView (views.py)	Guarda todo, inicia la sesión y lleva al inicio; quien ya tiene sesión es enviado al inicio

El usuario, el cliente y la cuenta se crean dentro de `transaction.atomic()`: o se crean los tres o no se crea ninguno, y nunca queda un usuario sin ficha. El usuario creado nunca pertenece al personal.

Figura 118. Enlace de registro en el login

Figura 119. Formulario de registro

Crear mi cuenta

Completa tus datos. Al terminar se abre tu primera cuenta, con saldo cero, y entras al sistema.

Ya existe un usuario con este nombre.

Nombre de usuario:

Requerido. 150 caracteres como máximo. Únicamente letras, dígitos y @/./+/_/

Nombre completo:

Ya existe un cliente con ese correo.

Correo electrónico:

Figura 120. Errores de validación en el registro

Alke Wallet

Inicio Mi ficha Cuentas Transacciones Sesión: **Pablo** (cliente) [Cerrar sesión](#)

¡Bienvenido a Alke Wallet! Tu cuenta quedó creada.

Bienvenido a Alke Wallet

Estas son tus cuentas:

- 0009: \$0 (CLP)

Últimos movimientos

Todavía no hay movimientos.

[Ver todos los movimientos](#)

Alke Wallet - Proyecto del Módulo 7

Figura 121. Registro exitoso: inicio con la cuenta nueva

Alke Wallet

Inicio Clientes Cuentas Transacciones Reporte Panel de administración Sesión: **admin** (personal) [Cerrar sesión](#)

Clientes

[Nuevo cliente](#)

Nombre	Correo electrónico	Teléfono	Cuentas	Acciones
Ana García	ana@example.com	123456789	3	Editar Eliminar
Carla Soto	carla@example.com	987654321	1	Editar Eliminar
Diego Rojas	diego@example.com	-	2	Editar Eliminar
Luis Pérez	luis@example.com	-	1	Editar Eliminar
Marta Vega	marta@example.com	-	1	Editar Eliminar
Pablo Almonacid	pabloalmonacid@example.com	93355111	1	Editar Eliminar
Pedro	Pedro@example.com	66556677	0	Editar Eliminar
Tomas Prueba	tomas@example.com	555123456	0	Editar Eliminar

Alke Wallet - Proyecto del Módulo 7

Figura 122. Cliente y cuenta creados, vistos en el panel

7.6 Perfil y cambio de contraseña

Ruta	Vista	Qué muestra o hace
/perfil/	PerfilView	Usuario, rol y último ingreso; si es cliente, también sus datos de cliente con los enlaces «Editar mis datos» y «Ver mi ficha»
/perfil/clave/	CambiarClaveView (PasswordChangeView)	Pide la contraseña actual y la nueva dos veces, con las validaciones de Django

El perfil sirve para cualquier rol. «Editar mis datos» reutiliza la edición de cliente de la subetapa 7.4, por lo que un cliente solo puede editar su propia ficha. Al cambiar la contraseña la sesión sigue abierta y se muestra un aviso de éxito.

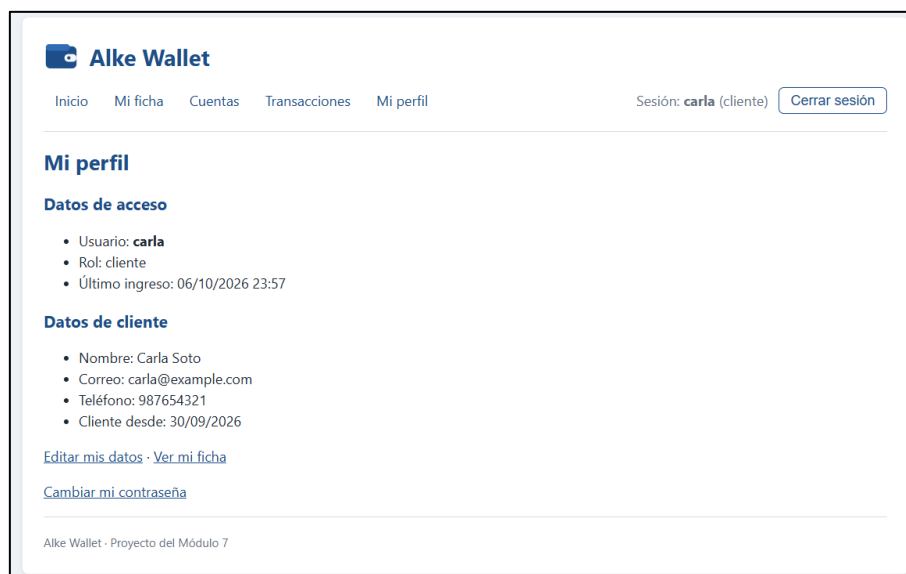


Figura 123. Perfil de un cliente

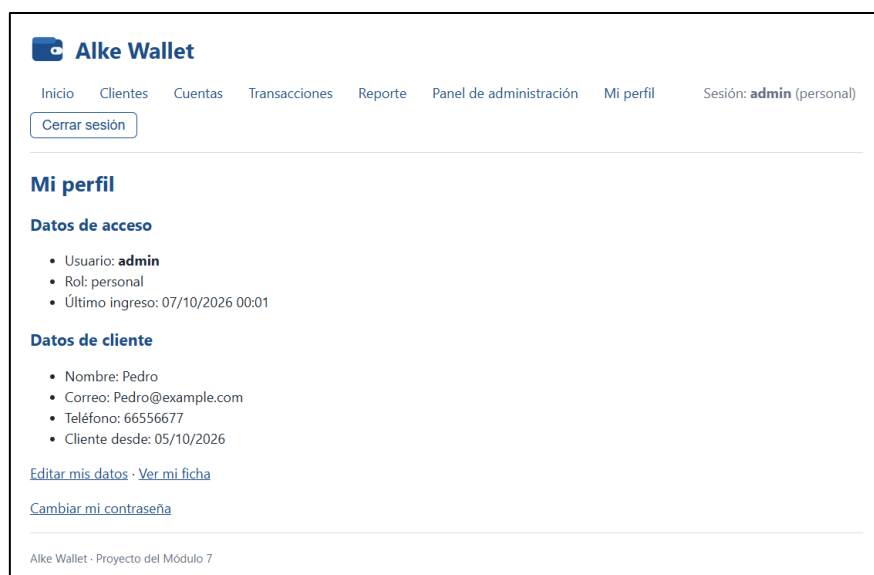
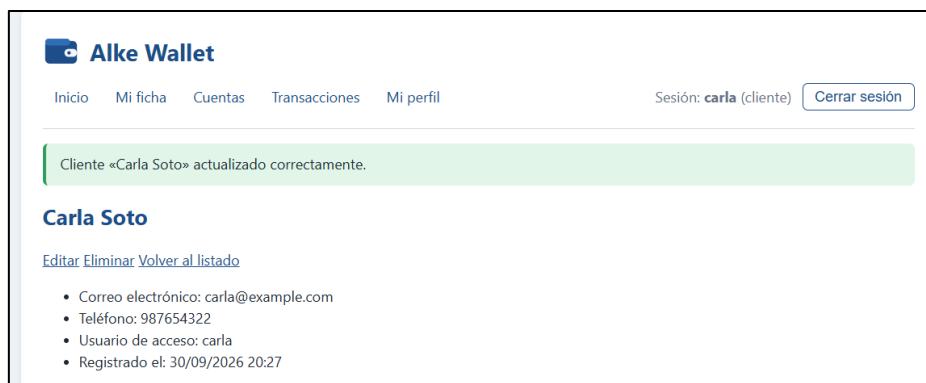


Figura 124. Perfil del personal



Alke Wallet

Inicio Mi ficha Cuentas Transacciones Mi perfil Sesión: **carla** (cliente) [Cerrar sesión](#)

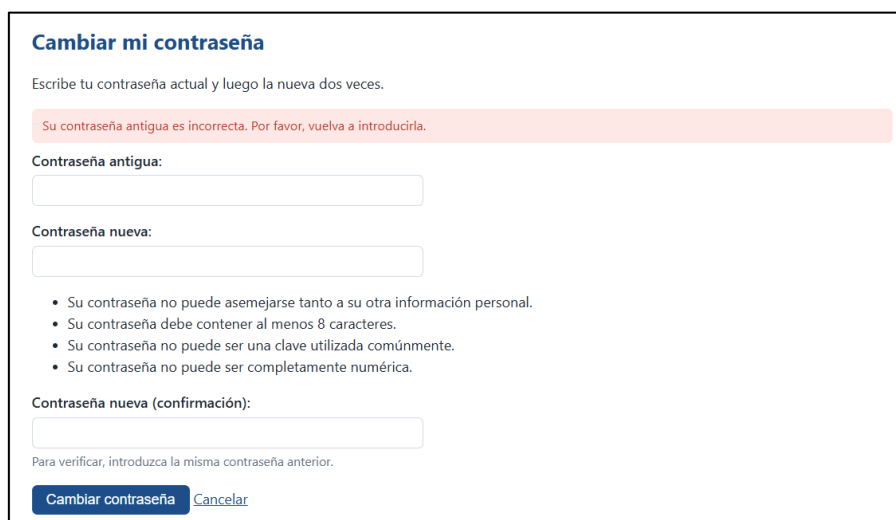
Cliente «Carla Soto» actualizado correctamente.

Carla Soto

[Editar](#) [Eliminar](#) [Volver al listado](#)

- Correo electrónico: carla@example.com
- Teléfono: 987654322
- Usuario de acceso: carla
- Registrado el: 30/09/2026 20:27

Figura 125. Edición de los datos propios



Cambiar mi contraseña

Escribe tu contraseña actual y luego la nueva dos veces.

Su contraseña antigua es incorrecta. Por favor, vuelva a introducirla.

Contraseña antigua:

Contraseña nueva:

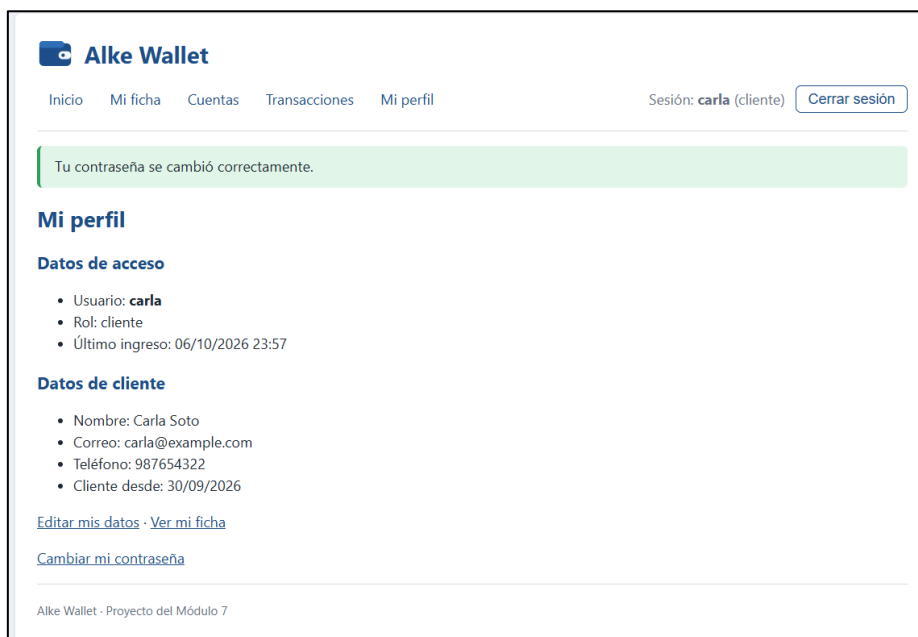
- Su contraseña no puede asemejarse tanto a su otra información personal.
- Su contraseña debe contener al menos 8 caracteres.
- Su contraseña no puede ser una clave utilizada comúnmente.
- Su contraseña no puede ser completamente numérica.

Contraseña nueva (confirmación):

Para verificar, introduzca la misma contraseña anterior.

[Cambiar contraseña](#) [Cancelar](#)

Figura 126. Error al escribir mal la contraseña actual



Alke Wallet

Inicio Mi ficha Cuentas Transacciones Mi perfil Sesión: **carla** (cliente) [Cerrar sesión](#)

Tu contraseña se cambió correctamente.

Mi perfil

Datos de acceso

- Usuario: **carla**
- Rol: cliente
- Último ingreso: 06/10/2026 23:57

Datos de cliente

- Nombre: Carla Soto
- Correo: carla@example.com
- Teléfono: 987654322
- Cliente desde: 30/09/2026

[Editar mis datos](#) · [Ver mi ficha](#)

[Cambiar mi contraseña](#)

Alke Wallet · Proyecto del Módulo 7

Figura 127. Contraseña cambiada con aviso de éxito

7.7 Verificación de accesos y CSRF

El comando `verificar_accesos` prueba todo lo anterior de una vez. Crea datos temporales (personal, dos clientes con cuenta, un movimiento y una ficha de agenda ajenos, y un usuario sin cliente) dentro de `transaction.atomic()` y los deshace al terminar, así que no deja nada en la base de datos. Usa `Client` de Django, que simula un navegador, con `enforce_csrf_checks=True` para exigir el token como en el navegador real.

Comandos: ver «Comandos», apartado «7.7 Verificación de accesos y CSRF».

Grupo	Qué se prueba	Esperado
Sin sesión	Las pantallas llevan al login con <code>?next=</code> ; login y registro son públicos	302 / 200
Personal	Todas las pantallas, incluidas las de otros clientes	200
Cliente	Sus pantallas propias	200
Cliente	Pantallas del personal	403
Cliente	Ficha, cuenta, movimiento y agenda de otro cliente	404
Sin cliente	Perfil e inicio	200
Sin cliente	Cuentas, transacciones y clientes	403
CSRF	Login, registro, logout, cambio de contraseña, nueva transacción y edición, sin token	403
Logout	Cerrar sesión con GET	405
CSRF	Login con token y clave incorrecta	200 (se procesa)

Resultado: **55 de 55 pruebas correctas**. Las pruebas unitarias formales y el informe de pruebas se abordan en la etapa 8.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\W7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py verificar_accesos
OK sin sesión: / lleva al login 302
OK sin sesión: /clientes/ lleva al login 302
OK sin sesión: /cuentas/ lleva al login 302
OK sin sesión: /transacciones/ lleva al login 302
OK sin sesión: /reporte/ lleva al login 302
OK sin sesión: /perfil/ lleva al login 302
OK sin sesión: el destino incluye ?next= 302
OK sin sesión: /acceso/login/ es pública 200
OK sin sesión: /registro/ es pública 200
OK personal: / 200
OK personal: /clientes/ 200
OK personal: /cuentas/ 200
OK personal: /transacciones/ 200
OK personal: /reporte/ 200
OK personal: /perfil/ 200
OK personal: /clientes/nuevo/ 200
OK personal: /cuentas/nueva/ 200
OK personal: /clientes/13/ 200
OK personal: /cuentas/14/ 200
OK personal: /transacciones/24/ 200
OK cliente: / (propio) 200
OK cliente: /cuentas/ (propio) 200
OK cliente: /transacciones/ (propio) 200
OK cliente: /perfil/ (propio) 200
OK cliente: /transacciones/nueva/ (propio) 200
OK cliente: /clientes/12/ (propio) 200
OK cliente: /clientes/12/editar/ (propio) 200
OK cliente: /cuentas/13/ (propio) 200
OK cliente: /clientes/ (solo personal) 403
OK cliente: /clientes/nuevo/ (solo personal) 403
OK cliente: /cuentas/nueva/ (solo personal) 403
OK cliente: /reporte/ (solo personal) 403
OK cliente: /cuentas/13/editar/ (solo personal) 403
OK cliente: /cuentas/13/eliminar/ (solo personal) 403
OK cliente: /clientes/12/eliminar/ (solo personal) 403
OK cliente: /clientes/13/ (ajeno) 404
OK cliente: /clientes/13/editar/ (ajeno) 404
```

Figura 128. Resultado de `verificar_accesos`

Comandos

7.2 Archivos estáticos

```
# Muestra en qué carpeta encuentra Django un archivo estático
python manage.py findstatic css/estilos.css
```

```
# Junta los estáticos propios y los del panel de administración en STATIC_ROOT
(se usa al desplegar)
python manage.py collectstatic
```

7.7 Verificación de accesos y CSRF

```
python manage.py verificar_accesos
```

Decisiones y alternativas descartadas

Decisión	Alternativa descartada	Motivo
LoginRequiredMiddleware	Un mixin de login en cada vista	Todo queda protegido por defecto; las pantallas públicas son la excepción explícita
Roles con <code>is_staff</code> y la relación 1:1 con Cliente	Grupos y permisos de Django o un campo <code>rol</code>	No agrega tablas ni migraciones y se entiende de un vistazo
Un solo módulo <code>alcance.py</code> con las reglas	Filtros repetidos en cada vista	Una regla en un lugar; el listado, la ficha, la edición y el borrado no pueden contradecirse
404 para registros ajenos	403	No confirma que el registro exista
El rol se revisa antes de buscar el registro	Buscar el registro en <code>dispatch</code>	Evita un error 500 para un usuario sin cliente
Registro con <code>atomic()</code>	Crear usuario, cliente y cuenta por separado	Nunca queda un usuario sin ficha

Limitaciones conocidas

- **No hay recuperación de contraseña por correo** ni verificación del correo al registrarse; requeriría configurar un servidor de correo.
- **El número de cuenta es «el mayor más uno».** Con muchos registros simultáneos podría repetirse un número y fallar un registro; para este proyecto es suficiente.
- **Solo el personal abre cuentas adicionales** y gestiona clientes; un cliente tiene la cuenta inicial del registro.
- **Las claves de los usuarios de ejemplo** son aleatorias o salen de `CLAVE_DEMO`; los datos de ejemplo no deben usarse en un sistema publicado.
- **No hay bloqueo por intentos fallidos de ingreso.**

Reflexiones

- **Proteger por defecto es más seguro:** con el middleware, olvidar marcar una vista deja un efecto visible (pide login), no un agujero.
- **Un rol no basta:** poder abrir `/cuentas/` no significa poder ver todas las cuentas. Separar roles (qué pantalla) de alcance (qué registro) evitó mezclar ambas reglas.
- **Ocultar un enlace no es seguridad:** el menú por rol es comodidad; las reglas viven en las vistas y se comprueban con `verificar_accesos`.
- **El orden importa:** el mixin de rol debe ejecutarse antes de cualquier código que use `request.user.cliente`.
- **Comprobar con datos temporales** permite repetir las pruebas sin ensuciar la base de datos.

Etapa 8: pruebas automatizadas

Objetivo

Comprobar con pruebas automáticas que las reglas de negocio, los formularios, las pantallas y la seguridad funcionan, y que las pruebas fallan cuando una regla se rompe.

Rama de trabajo: `feature/tests`. Esta etapa no cambia los modelos, por lo que no genera migraciones. El informe formal está en `docs/informe_pruebas.docx`.

Qué se hizo

Cómo se organizan: la carpeta `gestion/tests/` reemplaza al archivo `tests.py` y tiene un archivo por bloque. `utilidades.py` reúne funciones que crean datos de prueba (moneda, cliente, cuenta, depósito y usuario del personal) para no repetir código.

Subetapas:

Subetapa	Archivo	Pruebas	Qué revisa
8.0	<code>test_humo.py</code>	5	Que el entorno de pruebas funciona: base vacía, aislamiento y login
8.1	<code>test_modelos.py</code>	42	Restricciones, validaciones, saldo y protección al borrar
8.2	<code>test_servicios.py</code>	21	Movimientos atómicos, bloqueo de la cuenta origen y alta de cliente con cuenta
8.3	<code>test_consultas.py</code>	31	Filtros, agregaciones, saldos y consultas SQL propias
8.4	<code>test_formularios.py</code>	41	Opciones, validaciones y mensajes de cada formulario
8.4	<code>test_vistas.py</code>	49	CRUD, filtros, paginación, avisos y protecciones, como personal
8.5	<code>test_seguridad.py</code>	50	Roles, alcance por cliente, reglas de operación, CSRF y sesión
8.6	<code>docs/informe_pruebas.docx</code>		Informe con tabla de casos, resultados y evidencias

Cómo ejecutarlas (con `DB_ENGINE=sqlite` en `.env`):

Comandos: ver «Comandos», apartado «Cómo ejecutarlas».

Comandos

Cómo ejecutarlas

```
# Todas las pruebas
python manage.py test gestion

# Un bloque con detalle
python manage.py test gestion.tests.test_seguridad -v 2

# Comprobación manual de accesos (no deja datos)
python manage.py verificar_accesos
```

Decisiones y alternativas descartadas

- **Las pruebas usan una base temporal.** Django crea una base vacía, deshace cada prueba y la elimina al final; los datos reales no se tocan.
- **force_login y usuarios sin contraseña** evitan cifrar una clave en cada prueba, que era lo más lento (de 23 s a menos de 1 s en los modelos). Las pocas pruebas que necesitan una clave real usan `secrets`, así no hay contraseñas escritas en el código.
- **CSRF solo se prueba con Client(enforce_csrf_checks=True).** El cliente normal de las pruebas se salta esa revisión.
- **Una prueba vale si falla cuando se rompe la regla.** Se rompió a propósito cada regla de seguridad y se comprobó que al menos una prueba fallaba; los resultados están en el informe.
- **En una prueba que falló al principio** la causa era la prueba y no el código: pedir la página 2 sin tener 11 movimientos devuelve 404.

Resultado

Ran 239 tests
OK

```
-----
Ran 239 tests in 13.094s
OK
Destroying test database for alias 'default'...
```

Figura 129. Suite completa: 239 pruebas correctas

```
OK cliente: /contactos/7/eliminar/ (ajeno) 404
OK sin cliente: /perfil/ 200
OK sin cliente: / (inicio con aviso) 200
OK sin cliente: /cuentas/ 403
OK sin cliente: /transacciones/ 403
OK sin cliente: /clientes/ 403
OK sin cliente: /transacciones/nueva/ 403
OK CSRF: login sin token 403
OK CSRF: registro sin token 403
OK CSRF: cerrar sesión sin token 403
OK CSRF: cambiar contraseña sin token 403
OK CSRF: nueva transacción sin token 403
OK CSRF: editar mis datos sin token 403
OK logout con GET no permitido 405
OK CSRF: login con token (clave mala) se procesa 200
55 de 55 pruebas correctas. Los datos de prueba se deshicieron.
```

Figura 130. verificar_accesos: 55 de 55 comprobaciones correctas

Nota: el total posterior es de 245 pruebas. Las 6 pruebas adicionales (`test_salud.py` y `test_comandos.py`) se agregaron en la etapa 10.

Limitaciones conocidas

- Las pruebas se ejecutaron en SQLite. En PostgreSQL funcionan igual, pero el usuario de la base necesita permiso `CREATEDB` para crear la base de pruebas.
- En SQLite, `LIKE` ya ignora mayúsculas, así que quitar `LOWER()` de la búsqueda SQL propia no se detecta con esta base.
- `select_for_update` solo tiene efecto real en PostgreSQL; la concurrencia no se prueba de forma automática.
- No se mide la cobertura de código con `coverage.py`.

Reflexiones

- Un formulario se prueba con `is_valid()` y `errors`; una vista, con `self.client`, `reverse` y `assertRedirects`.
- Sin sesión redirige al login, con rol incorrecto da 403 y con un registro ajeno da 404.

Etapa 9: mejora visual con Bootstrap

Objetivo

Dar a la aplicación una interfaz profesional, moderna y adaptable a escritorio, tablet y celular, sin cambiar la lógica del sistema (modelos, vistas, reglas y rutas).

Rama de trabajo: `feature/bootstrap`. Esta etapa no cambia los modelos, por lo que no genera migraciones. Se integró a `main` con 7 commits por tema (fast-forward).

Qué se hizo

Subetapas:

Subetapa	Qué se hizo
9.1	Base: Bootstrap 5.3.3, Bootstrap Icons y la tipografía Inter por CDN; <code>base.html</code> con menú y pie de página; <code>estilos.css</code> como capa de marca
9.2	Componentes de plantilla reutilizables y filtros propios para dibujar los formularios
9.3	Rediseño de inicio, perfil, reporte, clientes, cuentas, transacciones y contactos
9.4	Portada pública con el inicio de sesión y el registro en la misma pantalla
9.5	Adaptación a tablet y celular
9.6	Comportamientos en el navegador (<code>static/js/app.js</code>)
9.7	Páginas de error 403, 404 y 500 con el mismo diseño
9.8	Montos con formato chileno

9.1 Base: Bootstrap, menú y pie de página

Bootstrap se carga desde un CDN en `base.html`, junto con Bootstrap Icons y la fuente Inter. `static/css/estilos.css` se reescribió: ya no define el diseño completo, sino una **capa de marca** sobre Bootstrap, con variables de color (azul `#1D4E89`, turquesa `#17B8A6`), tarjetas, tablas, formularios y animaciones.

Pieza de <code>base.html</code>	Qué hace
Menú superior	Degradado azul con línea turquesa; se convierte en botón de hamburguesa bajo 992 px
Enlace activo	Se marca según <code>request.resolver_match.url_name</code> , el nombre de la ruta actual
Desplegable de usuario	Muestra la inicial, el nombre y el rol; contiene "Mi perfil" y "Cerrar sesión" (que sigue siendo un formulario POST con CSRF)
Avisos	<code>componentes/_mensajes.html</code> ; Django llama error al tipo que Bootstrap llama <code>danger</code> , y se traduce
Pie de página	Marca, enlaces según el rol, tecnologías y año actual con <code>{% now "Y" %}</code>
Bloque cuerpo	Permite que la portada reemplace el contenedor centrado y use todo el ancho

9.2 Componentes y filtros de plantilla

Pieza	Dónde	Para qué
Filtro bootstrap	<code>gestion/templatetags/ui.py</code>	Agrega <code>form-control</code> , <code>form-select</code> o <code>form-check-input</code> a un campo y <code>is-invalid</code> si tiene errores

Pieza	Dónde	Para qué
Filtros <code>es_casilla</code> y <code>es_lista</code>	<code>gestion/templatetags/ui.py</code>	Detectan el tipo de campo para dibujarlo distinto
<code>_campos.html</code>	<code>templates/componentes/</code>	Dibuja todos los campos de un formulario (etiqueta, campo, ayuda y errores); con <code>dos_columnas=True</code> los acomoda de dos en dos
<code>_tipo.html</code>	<code>templates/componentes/</code>	Etiqueta de color según el tipo de movimiento: verde, rojo o azul
<code>_estado.html</code>	<code>templates/componentes/</code>	Etiqueta Activa o Inactiva de una cuenta
<code>_mensajes.html</code>	<code>templates/componentes/</code>	Avisos de las vistas
<code>_movimiento_item.html</code>	<code>templates/componentes/</code>	Fila de un movimiento pensada para celular

La alternativa de editar `forms.py` para agregar las clases a cada widget se descartó: habría mezclado la presentación con la lógica de los formularios y obligado a tocar las pruebas.

9.3 Pantallas de gestión

Las listas pasaron a tarjetas con tablas dentro, las fichas a tarjetas con datos y avatares con la inicial, y las confirmaciones de borrado a una tarjeta con franja roja. En el inicio, el personal ve tres tarjetas con los totales y el cliente ve sus cuentas como tarjetas con aspecto de tarjeta bancaria. El reporte muestra las proporciones con barras hechas con CSS puro (`{% widthratio %}` calcula el porcentaje).

Un cambio de comportamiento: los botones que solo puede usar el personal (nueva cuenta, editar, eliminar, volver al listado de clientes) **ya no se muestran a los clientes**. Antes se veían y, al pulsarlos, la persona recibía un 403.

9.4 Portada con inicio de sesión y registro

`templates/registration/landing.html` es la portada pública. De ella heredan `login.html` y `registro.html`, que solo aportan su formulario. Incluye un mensaje de presentación, tres ventajas, una tarjeta decorativa (no muestra datos reales), las características, "Empieza en tres pasos" y un llamado final.

- **Inicio de sesión y registro en la misma pantalla:** las pestañas "Iniciar sesión" y "Crear cuenta" son enlaces a `/acceso/login/#acceso` y `/registro/#acceso`. Cada formulario sigue atendido por su propia vista, por lo que no cambió ninguna ruta ni lógica; para la persona se siente como una sola pantalla.
- **Ancho distinto por página:** la portada define bloques (`columna_texto`, `columna_acceso`, `clase_tarjeta_acceso`) para que el registro, que tiene siete campos, use una tarjeta más ancha.
- **Registro en dos columnas:** los campos se acomodan de dos en dos; la lista de monedas ocupa todo el ancho. Las ayudas largas (más de 120 caracteres), como los requisitos de la contraseña, quedan plegadas detrás de "Ver requisitos". El formulario pasó de unos 900 px de alto a unos 600 px en escritorio.

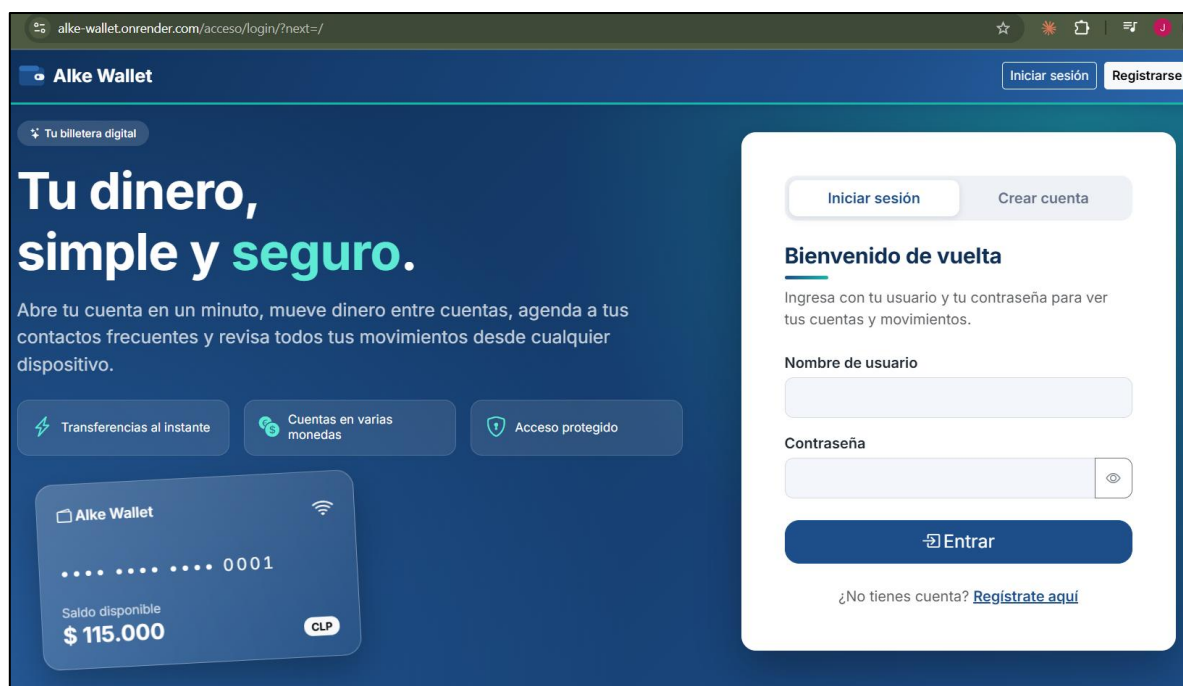


Figura 131. Portada en escritorio

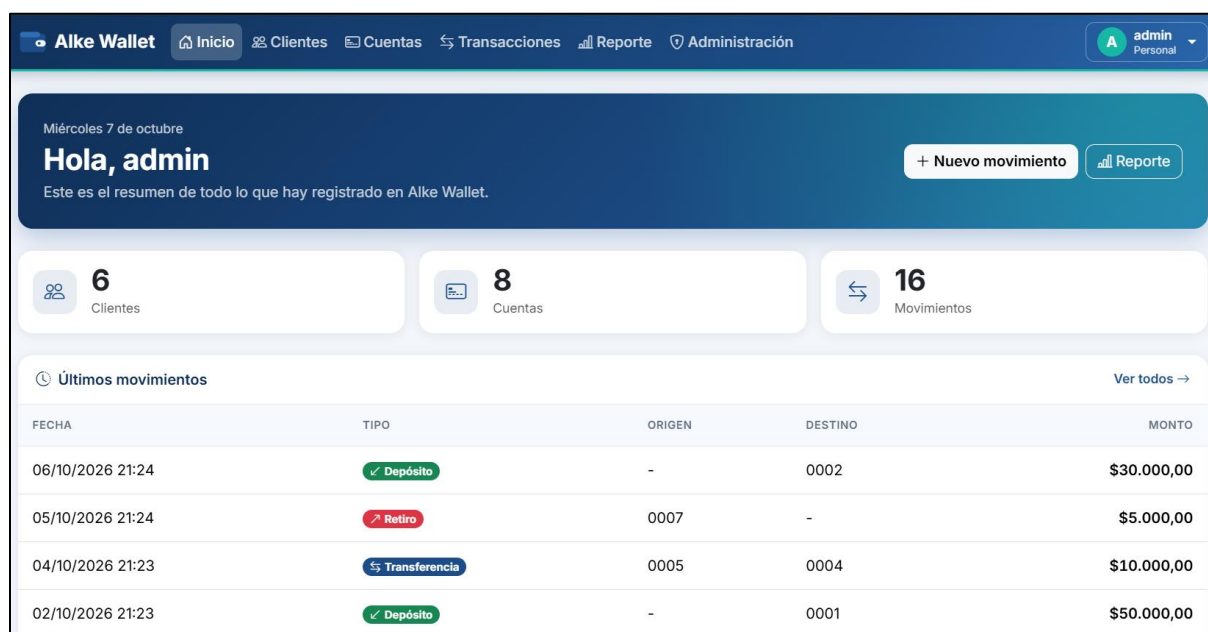


Figura 132. Inicio del personal en escritorio

9.5 Adaptación a tablet y celular

Elemento	Celular	Tablet (desde 768 px)	Escritorio (desde 992 px)
Menú	Botón de hamburguesa	Hamburguesa	Menú horizontal
Movimientos (inicio y listado)	Lista compacta con ícono, tipo, cuentas y monto	Tabla completa	Tabla completa con descripción
Cuentas y clientes	Columnas secundarias ocultas; el dato va bajo el principal	Más columnas	Todas las columnas

Elemento	Celular	Tablet (desde 768 px)	Escritorio (desde 992 px)
Filtros de transacciones	Plegados detrás de un botón	Siempre visibles	Siempre visibles
Totales del inicio	Tres tarjetas en una fila	Tres tarjetas	Tres tarjetas
Paginación	Botones anterior y siguiente con el número al centro	Igual, con texto	Igual, con texto
Portada	Texto centrado, tarjeta de acceso debajo	Tarjeta centrada con ancho máximo	Texto a la izquierda y tarjeta a la derecha
Pie de página	Marca centrada y dos columnas ordenadas a la izquierda	Tres columnas	Tres columnas

Se revisó con capturas de Chrome a 390 px (celular), 768 a 820 px (tablet) y 1100 a 1280 px (escritorio). No se probó en un teléfono real.

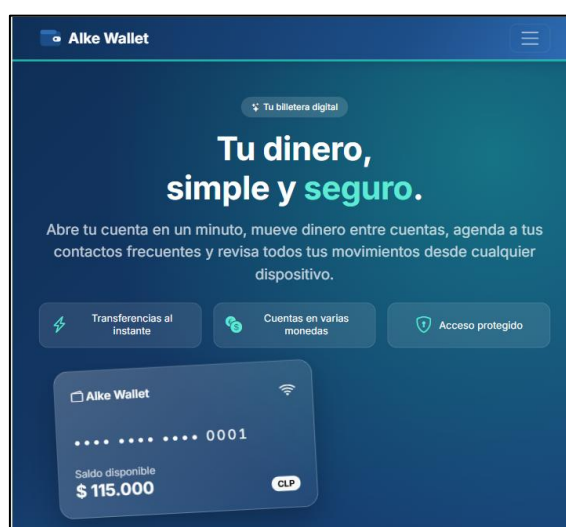


Figura 133. Portada en tablet

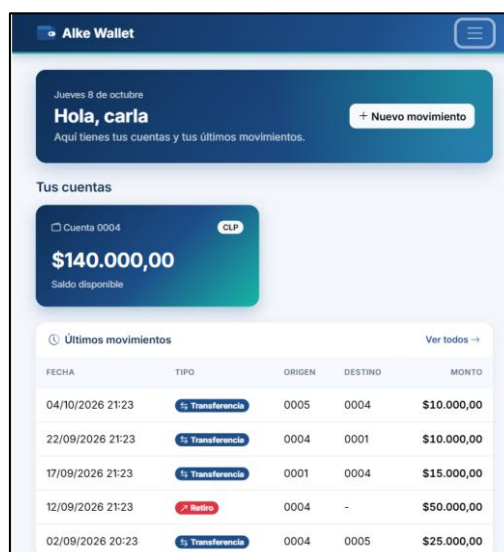


Figura 134. Inicio en tablet

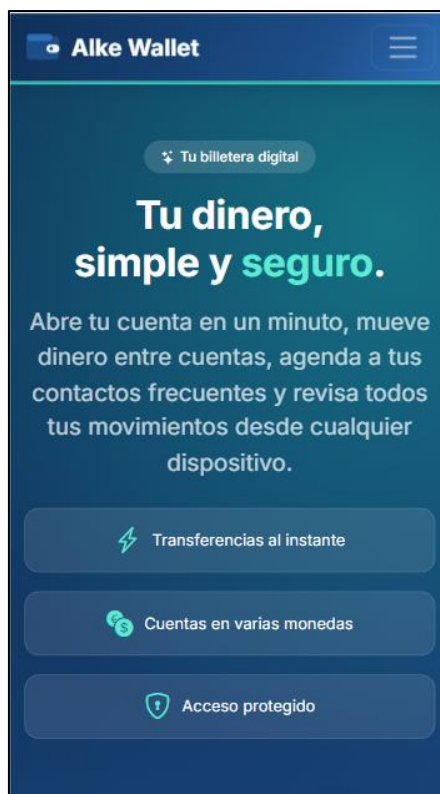


Figura 135. Portada en celular

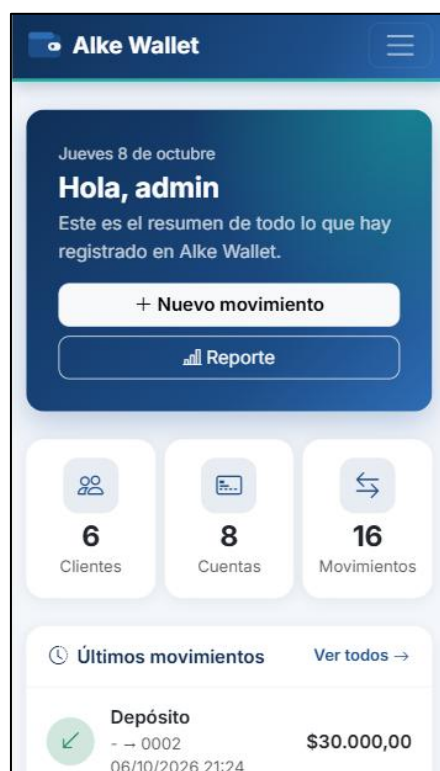


Figura 136. Inicio del personal en celular

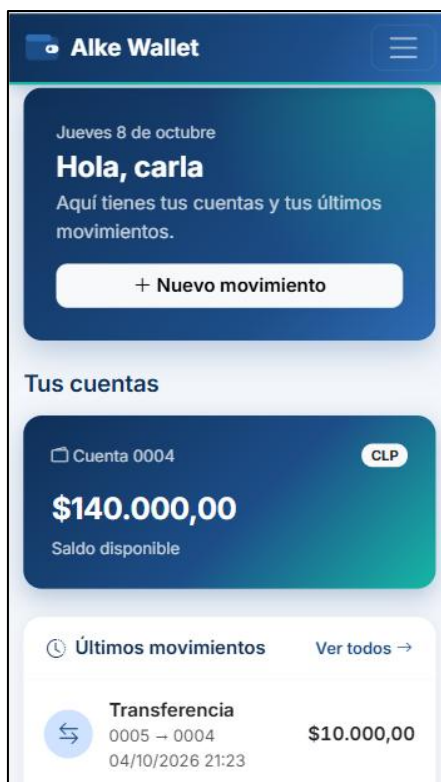


Figura 137. Inicio del cliente en celular

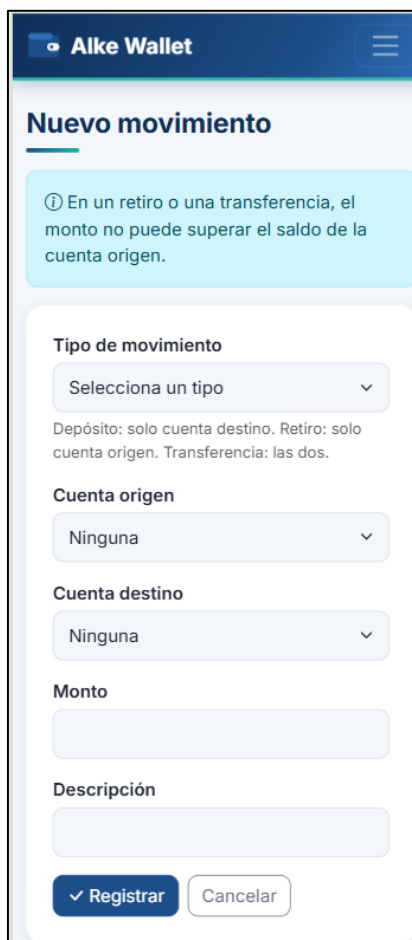


Figura 138. Movimientos en celular

9.6 Comportamientos en el navegador

`static/js/app.js` mejora la experiencia, pero la aplicación funciona igual sin él:

Comportamiento	Qué hace
Ver u ocultar la contraseña	Agrega un botón con un ojo a cada campo de contraseña
Evitar el doble envío	Al enviar un formulario POST, el botón se desactiva y muestra un indicador de carga, para que un doble clic no registre dos veces el mismo movimiento
Avisos que se cierran	Los avisos de éxito desaparecen a los 6 segundos; los de error y advertencia se quedan hasta que se cierran

Cuando se vuelve a una página con el botón Atrás, los botones desactivados se reactivan (evento `pageshow`).

9.7 Páginas de error

Página	Cuándo aparece	Botones
403.html	Una persona entra a una sección que no le corresponde	"Volver a mi panel" (con sesión) o "Iniciar sesión"; "Volver atrás"
404.html	La dirección o el registro no existen	"Volver a mi panel" (con sesión) o "Ir al inicio"; "Volver atrás"
500.html	Error interno del servidor	"Reintentar" e "Ir al inicio"

La 403 y la 404 heredan de `base.html`. La **500 es un documento independiente**: si el error viene de la base de datos o de la sesión, una página que dependa del menú o de `user` fallaría también. Por eso no usa `{% url %}`, `user` ni `request`, solo archivos estáticos y el enlace fijo `/`.

Con `DEBUG=True`, Django muestra su propia página técnica en lugar de `404.html` y `500.html`. Para ver los diseños en desarrollo existen las rutas `/vista-403/`, `/vista-404/` y `/vista-500/`, que solo se registran cuando `DEBUG` es `True`. Los tres errores se comprobaron de forma real con `DEBUG=False`: un 403 (un cliente entrando al listado de clientes), un 404 (dirección inexistente) y un 500 (excepción forzada en la vista de inicio).

9.8 Montos con formato chileno

Los montos pasaron de `$115000,00` a `$115.000,00`. Django solo trae el formato de España para el español, que usa un espacio como separador de miles. En Chile se usa el punto, así que se definió un formato propio:

Configuración	Para qué sirve
<code>USE_THOUSAND_SEPARATOR = True</code>	Activa el separador de miles en toda la aplicación
<code>FORMAT_MODULE_PATH = ['core.formats']</code>	Indica dónde buscar formatos propios
<code>core/formats/es_CL/formats.py</code>	Define punto como separador de miles y coma como separador decimal

Decisiones y alternativas descartadas

Decisión	Alternativa	Por qué
Bootstrap 5	Tailwind o CSS propio	Es el estándar para tablas, formularios y menú responsive, y no requiere compilar nada

Decisión	Alternativa	Por qué
Bootstrap por CDN	Descargarlo al proyecto	Evita subir archivos de terceros; la aplicación necesita internet para verse completa
Filtros de plantilla propios	<code>django-crispy-forms</code>	Evita una dependencia nueva para algo que se resuelve con un filtro de pocas líneas
Ocultar columnas y mostrar una lista en celular	Dejar la tabla con desplazamiento horizontal	Una tabla de siete columnas obligaba a desplazarse y cortaba el monto
Pestañas como enlaces entre dos rutas	Un solo formulario con JavaScript	No cambia las vistas ni las rutas; funciona sin JavaScript
Barras de proporción con CSS	<code>Chart.js</code>	Cubre lo esencial sin otra librería
Modo oscuro descartado	Activarlo con <code>data-bs-theme</code>	La paleta de marca está fija; habría que rediseñarla

Incidencias y solución

Problema	Causa	Solución
Las tarjetas se veían grises en lugar de blancas	Bootstrap toma el color de <code>var(--bs-body-bg)</code> , que se había cambiado a gris	Fijar <code>--bs-card-bg</code> en blanco y hacer transparentes las tablas
La tarjeta decorativa perdió su inclinación	La animación de entrada terminaba en <code>transform: translateY(0)</code> y anulaba el giro	Crear una animación propia (aparecer-inclinada) que conserva la rotación
El desplegable de usuario se salía de la pantalla en celular	Estaba fuera de un <code>navbar-nav</code> , así que Bootstrap lo posicionaba con Popper hacia afuera	Colocarlo dentro de un <code><ul class="navbar-nav"></code> : se abre dentro del menú
Los títulos de la portada salían oscuros sobre fondo azul	Una regla global les asignaba un color fijo	Usar la variable <code>--bs-heading-color</code> y cambiarla dentro de las secciones oscuras
Los montos se cortaban en celular	Tabla de siete columnas	Lista compacta en celular y columnas secundarias ocultas
Montos con un espacio como separador	El formato de Django para español usa un espacio	Formato propio <code>es_CL</code>

Resultado

Verificación: las 239 pruebas existentes siguieron pasando sin cambios después de cada rediseño, y se renderizaron las pantallas como personal, como cliente y sin sesión, todas con código 200.

Reflexiones

- Bootstrap pone la estructura; `estilos.css` solo agrega la marca.
- Un componente reutilizable (`{% include %}`) evita repetir el mismo HTML en cada pantalla.
- La página de error 500 no puede depender de nada que pueda fallar: ni sesión, ni menú, ni base de datos.

Etapa 10: preparación para el despliegue en Render

Objetivo

Dejar el proyecto listo para publicarse en Render (aplicación) con PostgreSQL en Neon (base de datos), sin escribir claves en el código y sin romper el funcionamiento local.

Qué se hizo

Ramas de trabajo: `feature/espera` (endpoint de salud y ventana de espera) y `feature/render` (configuración de producción), ambas ya fusionadas con `main`. Esta etapa no cambia los modelos, por lo que no genera migraciones.

Subetapas:

Subetapa	Qué se hizo
10.1	Variables de entorno para <code>DEBUG</code> , <code>SECRET_KEY</code> , hosts y orígenes de confianza
10.2	Dependencias nuevas en <code>requirements.txt</code>
10.3	Base de datos por <code>DATABASE_URL</code> y archivos estáticos con <code>WhiteNoise</code>
10.4	Seguridad en producción y registros
10.5	Comando <code>crear_superuser</code>
10.6	<code>build.sh</code> , <code>render.yaml</code> y <code>.gitattributes</code>
10.7	Endpoint <code>/salud/</code> , ventana de espera y página puente en GitHub Pages
10.8	Verificación simulando la producción en el equipo
10.9	Base de datos en Neon
10.10	Despliegue en Render y verificación en línea

10.1 Variables de entorno

Los valores que cambian entre el equipo y el servidor se leen del entorno. Si la variable no existe se usa el valor de desarrollo, así el proyecto sigue funcionando en local sin configurar nada.

Variable	Para qué sirve	Valor en local	Valor en Render
<code>DEBUG</code>	Activa o no el modo de desarrollo	<code>True</code> (por defecto)	<code>False</code>
<code>SECRET_KEY</code>	Clave secreta de Django	Clave de desarrollo (pública)	Aleatoria, la genera Render
<code>ALLOWED_HOSTS</code>	Direcciones desde las que se puede entrar	Vacía	<code>.onrender.com</code>
<code>CSRF_TRUSTED_ORIGINS</code>	Orígenes desde los que se aceptan formularios	Vacía	<code>https://*.onrender.com</code>
<code>DATABASE_URL</code>	Conexión completa a la base de datos	No existe	Dirección de Neon
<code>DJANGO_SUPERUSER_USERNAME</code> , <code>_EMAIL</code> , <code>_PASSWORD</code>	Datos del administrador	No existen	Privados, en el panel de Render
<code>CLAVE_DEMO</code>	Clave de los clientes de demostración	Opcional	Privada, en el panel de Render

Si `DEBUG=False` y falta `SECRET_KEY` (o es la de desarrollo), la aplicación se niega a arrancar con un error `ImproperlyConfigured`. Así es imposible publicarla por descuido con la clave pública del repositorio. El punto inicial de `.onrender.com` incluye todos los subdominios.

10.2 Dependencias

Cambio en <code>requirements.txt</code>	Para qué
<code>gunicorn</code>	Servidor que ejecuta la aplicación en Render
<code>whitenoise</code>	Sirve CSS, JS e imágenes; en Render no hay otro servidor que lo haga
<code>dj-database-url</code>	Convierte <code>DATABASE_URL</code> en la configuración de Django
<code>psycopg</code> → <code>psycopg[binary]</code>	Instala directo en Render, sin compilar

Además, el archivo se reescribió en UTF-8 sin BOM y con saltos de línea LF; tenía el formato de Windows.

10.3 Base de datos y archivos estáticos

- **Base de datos:** si existe `DATABASE_URL`, tiene prioridad sobre todo lo demás (`dj_database_url.config(conn_max_age=600, conn_health_checks=True)`). Si no, se mantiene la lógica anterior: `DB_ENGINE=postgres` para PostgreSQL local o SQLite por defecto.
- **Archivos estáticos:** el middleware `whitenoise.middleware.WhiteNoiseMiddleware` va justo después de `SecurityMiddleware`. En producción (`DEBUG=False`) se usa `CompressedManifestStaticFilesStorage`, que comprime los archivos y les pone un código en el nombre (`estilos.9ea9b391c386.css`) para que el navegador descargue la versión nueva cuando cambian. Esa versión necesita haber ejecutado `collectstatic`, por eso en desarrollo se usa el almacenamiento normal.

10.4 Seguridad y registros en producción

Cuando `DEBUG` es `False`:

Configuración	Qué hace
<code>SECURE_PROXY_SSL_HEADER</code>	Render atiende el HTTPS por fuera y le pasa a Django la petición como HTTP; este encabezado le indica que el original era HTTPS
<code>SECURE_SSL_REDIRECT = True</code>	Redirige toda visita HTTP a HTTPS
<code>SECURE_REDIRECT_EXEMPT</code>	Exime a <code>/salud/</code> , que Render consulta por HTTP interno
<code>SESSION_COOKIE_SECURE</code> y <code>CSRF_COOKIE_SECURE</code>	Las cookies de sesión y de CSRF solo viajan por HTTPS
<code>SECURE_HSTS_SECONDS = 3600</code>	El navegador recordará usar solo HTTPS durante 1 hora (se puede subir cuando todo esté estable)

Registros: con `DEBUG=False`, Django no muestra los errores en ninguna parte por defecto. Se configuró `LOGGING` para enviarlos a la consola, que es lo que Render muestra en la pestaña de registros.

10.5 Comando `crear_superuser`

Render gratuito no ofrece una terminal, así que el administrador no se puede crear con `createsuperuser`. El comando `python manage.py crear_superuser` lo crea desde las variables `DJANGO_SUPERUSER_*`:

- Si faltan las variables, no hace nada (así no falla en local).
- Si el usuario ya existe, no lo toca (repetir el despliegue no cambia una clave que se haya modificado después).
- Nunca imprime la contraseña.

Tiene 4 pruebas en `gestion/tests/test_comandos.py`: crea el superusuario, no hace nada sin variables, repetirlo no cambia la clave y no muestra la clave en pantalla.

10.6 `build.sh`, `render.yaml` y `.gitattributes`

`build.sh` se ejecuta en cada despliegue y se detiene si cualquier paso falla (`set -o errexit`):

Paso	Comando
1	<code>pip install -r requirements.txt</code>
2	<code>python manage.py collectstatic --no-input</code>
3	<code>python manage.py migrate --no-input</code>
4	<code>python manage.py crear_superusuario</code>
5	<code>python manage.py poblar_datos</code> (no duplica y no cambia las claves de usuarios que ya existen)

`render.yaml` es el plano del servicio: tipo web, plan gratuito, `buildCommand`: `bash build.sh`, `startCommand`: `gunicorn core.wsgi:application --workers 2 --timeout 120` y `healthCheckPath`: `/salud/`. Los valores privados (`DATABASE_URL`, `DJANGO_SUPERUSER_*` y `CLAVE_DEMO`) están marcados con `sync: false`: se escriben a mano en el panel y no se guardan en el repositorio.

`.gitattributes` fuerza saltos de línea LF en `*.sh`, `render.yaml` y `requirements.txt`. Con los saltos de Windows (CRLF), Linux no puede ejecutar `build.sh`.

10.7 Endpoint `/salud/`, ventana de espera y página puente

`/salud/` responde `{"estado": "ok"}` sin sesión ni consulta a la base de datos, y tiene dos pruebas. La usan dos cosas: Render, para saber que la aplicación arrancó, y la página puente (más abajo). Incluye el encabezado `Access-Control-Allow-Origin: *`, que permite que una página de otro dominio lea la respuesta; no expone ningún dato.

Ventana de espera: el plan gratuito de Render duerme el servicio tras 15 minutos sin visitas y la primera respuesta puede tardar cerca de un minuto; la base de Neon también se suspende por inactividad. Al enviar el formulario de inicio de sesión o el de registro, la aplicación muestra una ventana con un indicador de carga cuyo mensaje cambia con el tiempo:

Tiempo	Mensaje
Al enviar	"Estamos procesando tu solicitud"
4 segundos	"Esto está tardando un poco más de lo normal"
20 segundos	"Despertando el servidor...", con la explicación de que la demostración usa un servidor gratuito
75 segundos	Aparece el botón "Reintentar"

Desaparece sola: cuando el servidor responde, el navegador carga la página siguiente. Se activa con el atributo `data-espera` en el formulario y se implementa en `static/js/app.js`. La primera carga con el servicio dormido, en cambio, la muestra Render con su propia pantalla de arranque, y eso no se puede personalizar en el plan gratuito. Para que quien llega desde el README o el portafolio no vea esa pantalla, existe una página puente:

Pieza	Que hace
docs/index.html	Página estática, publicada con GitHub Pages, con el mismo diseño de la aplicación: mensaje "Preparando la demostración...", barra de progreso, contador de segundos y consejos que rotan
Sondeo de /salud/	Cada 3 segundos pregunta a la aplicación si ya despertó. Mientras Render arranca, su pantalla de espera no incluye el permiso (CORS), la consulta falla y se sigue esperando; cuando la aplicación responde {"estado": "ok"}, redirige sola
Limite de espera	A los 150 segundos ofrece un botón "Reintentar"
Enlace directo	Siempre muestra "Ir directo a la aplicación"

Se publica en <https://javiermll.github.io/alke-wallet-django/> (GitHub: Settings → Pages, rama main, carpeta /docs; el archivo docs/.nojekyll evita que GitHub lo procese con Jekyll). Es el enlace que se usa en el README y en el portafolio. Quien escriba directamente la dirección de Render seguirá viendo la pantalla de Render. La ventana de espera de la aplicación sigue cubriendo la demora al iniciar sesión o registrarse. Una primera versión de esta idea se descartó por ser más llamativa que la ventana; se retomó al comprobar que la alternativa era la pantalla de Render. La ventana cubre lo que ocurre dentro de la aplicación, ya cargada.

10.8 Verificación simulando la producción

Con `DEBUG=False`, `SECRET_KEY` y `DATABASE_URL` definidos, una base temporal y el mismo orden de pasos de `build.sh`, se comprobó:

Comprobación	Resultado
collectstatic	399 archivos procesados con nombre con código
migrate	Todas las migraciones aplicadas
crear_superusuario (dos veces)	La primera crea el usuario; la segunda informa que ya existe
poblar_datos	Datos de demostración cargados
/salud/ por HTTP	200 {"estado": "ok"}
Otra página por HTTP	301 hacia HTTPS
Login por HTTPS	200
CSS con código en el nombre	200, text/css, caché de largo plazo
Dirección inexistente	404 con la página propia
Inicio de sesión de un cliente	302 al inicio; cookies de sesión y CSRF con secure
Arrancar sin SECRET_KEY	ImproperlyConfigured

Las 245 pruebas pasan. `check --deploy` informa un error por el backend de correo de consola (la aplicación no envía correos) y avisos de HSTS que se dejaron sin cambiar a propósito. Unicorn no corre en Windows, así que la verificación usa el cliente de pruebas de Django, que recorre las mismas capas (incluido WhiteNoise); el arranque con unicorn solo se podrá comprobar en Render.

10.9 Base de datos en Neon

Se creó un proyecto en Neon y se usó la **cadena de conexión directa** (sin la opción "pooled"), para evitar problemas con los cursores del servidor de Django. La conexión se verificó desde el equipo con `DATABASE_URL` definida solo para esa sesión de PowerShell:

Comandos: ver «Comandos», apartado «10.9 Base de datos en Neon».

Las dos migraciones aparecieron sin aplicar ([]), lo que confirma que se conecta a la base nueva y vacía de Neon, y no al `db.sqlite3` local. La cadena de conexión nunca se escribe en un archivo del repositorio.

10.10 Despliegue en Render y verificación en línea

Paso	Estado
Código y configuración de producción	Listo
Base de datos en Neon y conexión verificada	Listo
main publicada en GitHub (15 commits y 3 ramas), tras revisar que no se filtraran claves	Listo
Servicio creado en Render como Blueprint desde <code>render.yaml</code> , con los valores privados cargados en el panel	Listo
Verificación en línea	Listo
Página Puente (<code>docs/index.html</code>) publicada y probada con GitHub Pages	Listo

Servicio: `alke-wallet`, plan gratuito, rama `main`, dirección <https://alke-wallet.onrender.com>. Se creó desde **New + → Blueprint**, que lee `render.yaml`. Los valores privados (`DATABASE_URL`, `DJANGO_SUPERUSER_USERNAME`, `DJANGO_SUPERUSER_EMAIL`, `DJANGO_SUPERUSER_PASSWORD` y `CLAVE_DEMO`) se escribieron a mano en el panel y no existen en el repositorio. En cada construcción se ejecuta `build.sh` y luego arranca `unicorn`. Por defecto, Render vuelve a desplegar cada vez que se publica un cambio en `main`.

Verificación externa (peticiones de solo lectura sin sesión):

Dirección	Resultado
<code>/salud/</code>	200, {"estado": "ok"}
<code>/acceso/login/</code> y <code>/registro/</code>	200
<code>/clientes/</code> sin sesión	302 hacia el login
Una dirección inexistente	404 con la página propia
<code>/static/css/estilos.css</code>	200, text/css (WhiteNoise)
Cookies	Con el atributo Secure
Encabezados	Strict-Transport-Security (HSTS), X-Frame-Options: DENY, X-Content-Type-Options: nosniff y Referrer-Policy: same-origin
https://javiermll.github.io/alke-wallet-django/	200, con el título "Despertando la demostración"
<code>/salud/</code> con el encabezado Origin de GitHub Pages	200, {"estado": "ok"} y <code>access-control-allow-origin: *</code>

Verificación manual: se comprobaron en línea el inicio de sesión de un cliente de demostración, el CRUD, el reporte, los estilos y las páginas de error.

Observaciones:

- **Versión de Python:** el despliegue se completó con `PYTHON_VERSION=3.13.5`, así que el supuesto quedó confirmado. En local se sigue usando Python 3.14.6.
- **Región:** `render.yaml` no fija una región, por lo que el servicio usa la que Render asigna por defecto. Si se notara latencia entre la aplicación y la base de Neon, es lo primero que habría que revisar.

Evidencias:



Figura 139. Servicio alke-wallet en estado Live en Render, desplegado desde la rama main

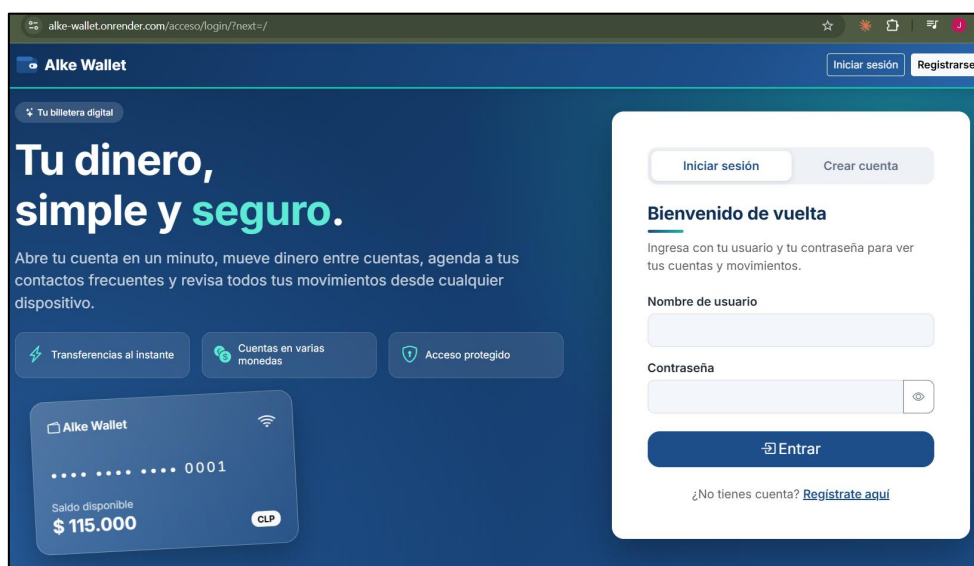


Figura 140. Portada en línea, en alke-wallet.onrender.com

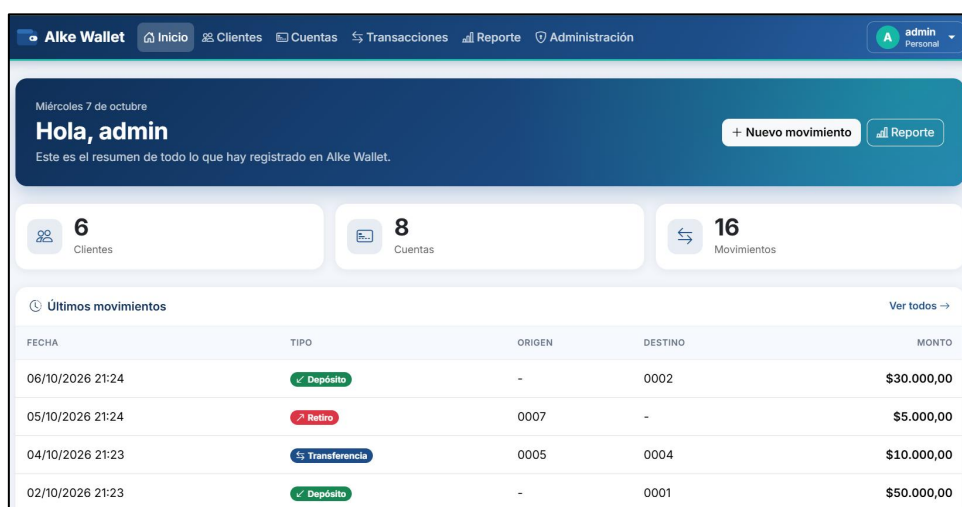


Figura 141. Inicio del personal en línea, con los totales y los últimos movimientos

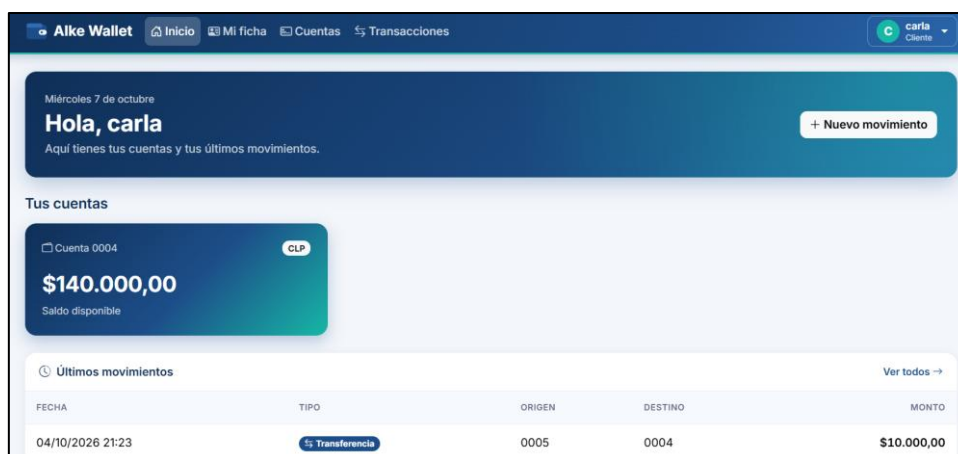


Figura 142. Inicio de un cliente en línea, con su cuenta y su saldo

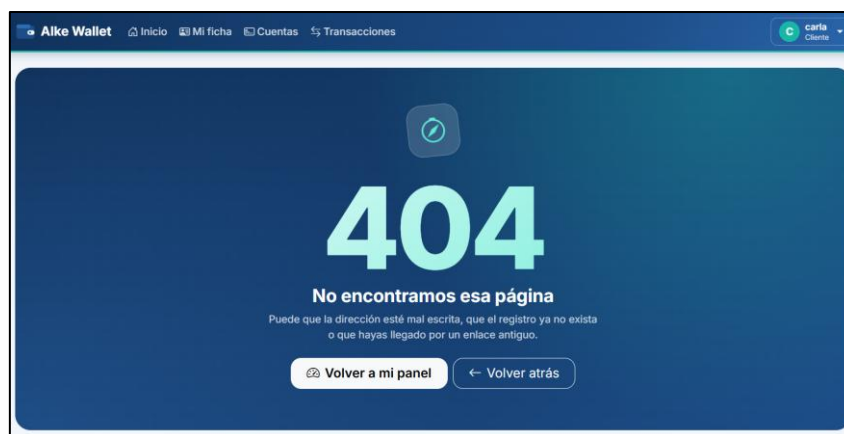


Figura 143. Página 404 propia en línea, con el botón que vuelve al panel

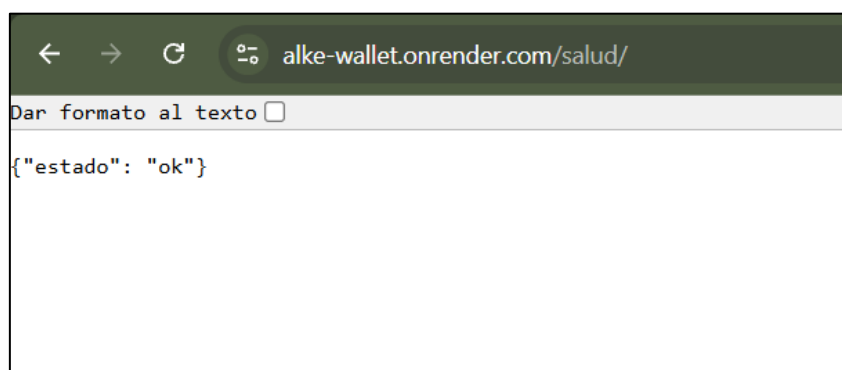


Figura 144. Comprobación de salud en línea: {"estado": "ok"}

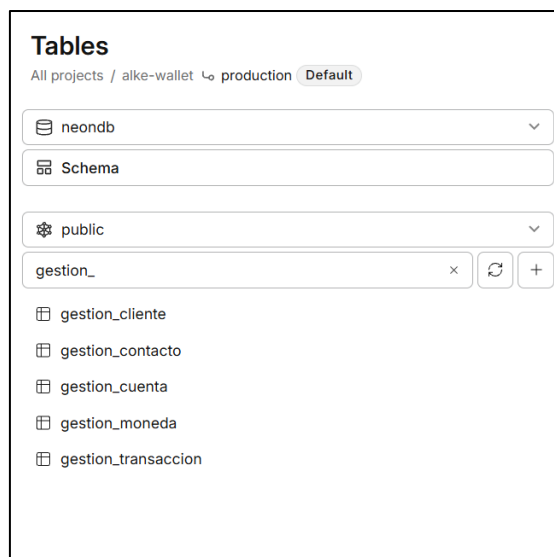


Figura 145. Tablas `gestion_*` creadas por las migraciones en la base de Neon

Comandos

10.9 Base de datos en Neon

```
$env:DATABASE_URL = "postgresql://usuario:clave@host/base?sslmode=require"
.\venv\Scripts\python.exe manage.py showmigrations gestion
```

Decisiones y alternativas descartadas

Decisión	Alternativa	Por qué
Render	Vercel	Vercel es serverless: no sirve archivos estáticos de Django ni mantiene SQLite. Render ejecuta la aplicación como un servicio normal
Neon para la base de datos	PostgreSQL de Render	La base gratuita de Render expira; la de Neon no
DATABASE_URL con prioridad	Variables <code>DB_*</code> separadas	Una sola cadena es lo que entregan Neon y Render
Archivos estáticos con código en el nombre solo con <code>DEBUG=False</code>	Siempre	Requieren <code>collectstatic</code> ; en desarrollo y en las pruebas fallarían
Comando <code>crear_superuser</code>	Crear el administrador desde una terminal	Render gratuito no tiene terminal
<code>poblar_datos</code> en cada construcción	Cargarlo una vez a mano	Es idempotente y deja la demostración siempre disponible
Clave de demostración solo para clientes, definida con <code>CLAVE_DEMO</code> en el panel de Render y fuera del repositorio	Publicar también la del administrador, o escribir la clave en el README	El personal puede borrar todos los datos, y una clave escrita en el repositorio la detectan los escáneres de secretos
Cadena directa de Neon	Cadena con pooler	El pooler puede dar problemas con los cursores de Django
Página Puente en Github Pages como enlace público	Mantener el servicio despierto con un monitor, o pasar a un plan de pago	Es gratis y no consume las 750 horas gratuitas mensuales de Render, que se reparten entre todos los servicios gratuitos de la cuenta

Incidencias y solución

Problema	Causa	Solución
<code>.env.example</code> tenía <code>B_ENGINE</code>	Error de tipeo (faltaba la <code>D</code> de <code>DB_ENGINE</code>)	Corregido y documentadas las variables de producción
Un script con saltos de línea de Windows falla en Linux	Git en Windows convierte LF en CRLF	<code>.gitattributes</code> con <code>eo1=lf</code>
Los errores no aparecerían en los registros de Render	Con <code>DEBUG=False</code> Django no los muestra por defecto	Configuración <code>LOGGING</code> hacia la consola
<code>requirements.txt</code> con BOM y saltos de Windows	Se generó con PowerShell	Reescrito en UTF-8 sin BOM y con LF

Limitaciones conocidas

- El plan gratuito de Render duerme el servicio tras 15 minutos sin visitas y tarda cerca de un minuto en despertar; Neon también se suspende por inactividad.
- La demostración pública permite que cualquiera se registre y cree datos de prueba.

Reflexiones

- Nada privado va al repositorio: lo que cambia entre el equipo y el servidor se lee de variables de entorno.
- `DEBUG=False` cambia varias cosas a la vez: estáticos con código, HTTPS obligatorio, cookies seguras y la exigencia de una `SECRET_KEY` propia.
- Sin terminal, todo lo que se hace una vez (administrador, datos de ejemplo) debe poder ejecutarse sin intervención en la construcción.

Etapa 11: revisión final y publicación

Objetivo

Dejar el proyecto listo para entregarlo y mostrarlo.

Qué se hizo

Subetapas:

Subetapa	Qué se hizo
11.1	Revisión del repositorio: sin <code>.env</code> , base de datos, entornos virtuales ni claves escritas en el código; <code>.gitignore</code> y <code>.env.example</code> al día; migraciones sin cambios pendientes; dependencias con versiones fijas; instalación desde cero comprobada (239 pruebas en <code>OK</code> en ese momento; hoy son 245)
11.2	README en versión resumida (README.md). El detalle de cada etapa, con las capturas, está en este documento técnico, disponible en docs/Alke_Wallet_Documentacion_Tecnica.pdf.
11.3	Las capturas sueltas se retiraron del repositorio; siguen en el historial de Git

Resultado

Instalación desde cero comprobada: se clonó el repositorio en otra carpeta, se creó un entorno virtual nuevo, se instalaron las dependencias, se copió `.env.example` a `.env`, se aplicaron las migraciones, se cargaron los datos de ejemplo y las pruebas (239 en ese momento) terminaron en `OK`.

```

• (venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_prueba> python manage.py test gestion
Found 239 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 239 tests in 13.868s

OK
Destroying test database for alias 'default'...
```

Figura 146. Instalación desde cero y pruebas en una copia limpia

5. Pruebas

El proyecto tiene **245 pruebas automatizadas** en `gestion/tests/`, un archivo por bloque: modelos, servicios, consultas, formularios, vistas, seguridad, la comprobación de salud (`test_salud.py`) y el comando `crear_superusuario` (`test_comandos.py`). Se ejecutan con SQLite (deja `DB_ENGINE=sqlite` en `.env`):

```
# Ejecuta todas las pruebas; Django usa una base temporal y no toca tus datos
python manage.py test gestion

# Ejecuta un bloque con detalle
python manage.py test gestion.tests.test_seguridad -v 2
```

Resultado esperado: `Ran 245 tests` y OK. Las primeras 239 son las de la etapa 8 y están documentadas, junto con la verificación de que detectan fallos, en `docs/informe_pruebas.docx`. Las 6 restantes se agregaron en la etapa 10: 2 del endpoint `/salud/` y 4 del comando `crear_superusuario`.

Subetapas de la etapa de pruebas

Subetapa	Archivo	Pruebas	Qué revisa
8.0	<code>test_humo.py</code>	5	Que el entorno de pruebas funciona: base vacía, aislamiento y login
8.1	<code>test_modelos.py</code>	42	Restricciones, validaciones, saldo y protección al borrar
8.2	<code>test_servicios.py</code>	21	Movimientos atómicos, bloqueo de la cuenta origen y alta de cliente con cuenta
8.3	<code>test_consultas.py</code>	31	Filtros, agregaciones, saldos y consultas SQL propias
8.4	<code>test_formularios.py</code>	41	Opciones, validaciones y mensajes de cada formulario
8.4	<code>test_vistas.py</code>	49	CRUD, filtros, paginación, avisos y protecciones, como personal
8.5	<code>test_seguridad.py</code>	50	Roles, alcance por cliente, reglas de operación, CSRF y sesión
8.6	<code>docs/informe_pruebas.docx</code>		Informe con tabla de casos, resultados y evidencias

El total actual es de 245 pruebas: las 239 de la etapa 8, más 2 del endpoint `/salud/` y 4 del comando `crear_superusuario`.

El informe formal de pruebas, con la tabla de casos, los resultados y las evidencias, está en `docs/informe_pruebas.docx`. Ese informe aún documenta las 239 pruebas de la etapa 8; su actualización a 245 está pendiente.

6. Reflexiones

Migraciones (Etapa 3)

- **Historial versionado:** la migración `0001_initial.py` se versiona en Git. Cualquier persona que clone el repositorio reconstruye el mismo esquema con `migrate`. El archivo `db.sqlite3` no se versiona porque contiene datos, no código.
- **Un plan, dos dialectos:** los mismos archivos de migración sirven para SQLite y PostgreSQL. El ORM traduce cada modelo al SQL de cada motor.
- **Reconstrucción de tablas en SQLite:** `sqlmigrate` muestra que SQLite no puede añadir restricciones a una tabla existente. Django crea una tabla temporal `new__`..., copia los datos, borra la original y renombra la nueva. Es una limitación del motor, que el ORM resuelve automáticamente.
- **Decimales:** SQLite no tiene un tipo decimal nativo, por eso el saldo de Ana se muestra como `70000` y no como `70000.00`. El valor es el mismo. PostgreSQL sí conserva los dos decimales.
- **Validaciones:** las creaciones desde la shell no ejecutan `clean()`; solo actúan las restricciones de la base de datos. Por eso el monto 0 se probó con `IntegrityError`. Las reglas de `clean()` se comprobarán en formularios y en el panel de administración.

ORM y consultas personalizadas (Etapa 4)

- **ORM frente a SQL propio:** el ORM resuelve casi todo y evita errores de sintaxis. El SQL propio ayuda a entender qué hay debajo y a escribir consultas complejas, como el saldo, que pueden hacerse igual con subconsultas del ORM.
- **NULL no es lo mismo que texto vacío:** pueden convivir en la misma columna (en los datos de demostración, Marta tiene `' '` y Luis tiene `NULL`). Un formulario de Django guarda `NULL` cuando se deja vacío un campo de texto que admite `NULL`, pero los datos que llegan por otra vía pueden traer `' '`. La consulta de la consigna (`IS NOT NULL`) cuenta como si tuviera teléfono a quien tiene `' '`. Hay que descartar ambos casos.
- **Las agregaciones pueden mentir sin avisar:** dos `Sum` sobre relaciones distintas dan montos inflados sin ningún error. Conviene contrastar siempre con una cuenta conocida.
- **Seguridad:** los parámetros `%s` son lo que separa una consulta segura de una vulnerable a inyección.
- **Las validaciones no se aplican siempre:** `create()`, `update()` masivo y el cursor no ejecutan `clean()`. Solo actúan las restricciones de la base de datos. Las reglas de Python se aplican con `full_clean()`, en formularios y en el panel de administración.
- **Sin dependencia del motor:** las diez consultas dan los mismos resultados con SQLite y con PostgreSQL, cada uno con su adaptador.

7. Conclusiones y próximos pasos

Conclusiones

- Se construyó una billetera digital en Django con cinco modelos (Moneda, Cliente, Contacto, Cuenta, Transaccion) y relaciones 1:1, N:1 y N:M.
- La lógica de negocio queda protegida en dos niveles: validaciones de Python y restricciones de la base de datos.
- La aplicación funciona con SQLite en desarrollo y con PostgreSQL en producción, cambiando solo una variable de entorno.
- El acceso está protegido por defecto: todas las pantallas piden sesión, cada rol ve lo que le corresponde y cada cliente ve únicamente sus propios datos.
- La etapa de pruebas dejó 239 pruebas automatizadas y un informe de pruebas; con las etapas posteriores el total es de 245.
- La interfaz se rediseñó con Bootstrap (etapa 9) y la aplicación se publicó en Render con PostgreSQL en Neon (etapa 10) y está disponible en <https://javiermll.github.io/alke-wallet-django/> (la aplicación corre en <https://alke-wallet.onrender.com>).

Próximos pasos

- Recuperación de contraseña por correo y verificación del correo al registrarse.
- Bloqueo temporal tras varios intentos fallidos de ingreso.
- Número de cuenta generado de forma segura ante registros simultáneos.
- Medición de cobertura de código con `coverage.py` y pruebas de interfaz con un navegador automatizado.
- Modo oscuro (Bootstrap lo admite, pero la paleta de la marca está fija y habría que rediseñarla).
- Gráficos interactivos en el reporte (hoy las proporciones son barras hechas con CSS).
- Limpieza automática de los usuarios de prueba que se registren en la demostración pública.
- Dominio propio y envío real de correos (hoy el proyecto no envía ninguno).

Autor

Javier · Full Stack Python Trainee (Talento Digital / Alkemy)

- GitHub: github.com/Javiermll
- LinkedIn: linkedin.com/in/jamunozll

Proyecto con fines educativos, desarrollado para el Módulo 7 del bootcamp.

8. Anexo: índice de capturas

Todas las capturas del documento, en orden de aparición. Las de origen «VP» son las vistas previas de docs/vista_previa.

Figura	Captura	Descripción	Etapas	Archivo
1	01	Servidor de Django funcionando	Etapas 0	01_servidor_cohete.png
2	02	Estructura del proyecto en VS Code	Etapas 0	02_estructura_proyecto.png
3	03	Bloque DATABASES en settings.py	Etapas 1	03_settings_databases.png
4	04	Verificación con SQLite	Etapas 1	04_verificacion_sqlite.png
5	05	Verificación con PostgreSQL	Etapas 1	05_verificacion_postgresql.png
6	06	INSTALLED_APPS con la app gestion	Etapas 2	06_installed_apps.png
7	07	Archivo models.py en VS Code	Etapas 2	07_models_py.png
8	08	Migración 0001_initial.py en VS Code	Etapas 3	08_migracion_inicial.png
9	09	Salida de sqlmigrate	Etapas 3	09_sqlmigrate.png
10	10	Migrate y showmigrations en SQLite	Etapas 3	10_migrate_sqlite.png
11	11	Tablas en SQLite	Etapas 3	11_tablas_sqlite.png
12	12	Migrate y showmigrations en PostgreSQL	Etapas 3	12_migrate_postgresql.png
13	13	Tablas en PostgreSQL con psql	Etapas 3	13_tablas_postgresql.png
14	14	Prueba en la shell con SQLite	Etapas 3	14_prueba_shell_sqlite.png
15	15	Prueba en la shell con PostgreSQL	Etapas 3	15_prueba_shell_postgresql.png
16	16	Primera ejecución de poblar_datos	Etapas 4	16_poblar_datos_sqlite.png
17	17	Segunda ejecución: sin registros nuevos	Etapas 4	17_poblar_datos_repetido.png
18	18	poblar_datos en PostgreSQL	Etapas 4	18_poblar_datos_postgresql.png
19	19	CRUD: crear	Etapas 4	19_crud_create.png
20	20	CRUD: leer	Etapas 4	20_crud_read.png
21	21	CRUD: actualizar	Etapas 4	21_crud_update.png
22	22	Validación con full_clean()	Etapas 4	22_crud_validar.png
23	23	CRUD: borrar, con PROTECT y CASCADE	Etapas 4	23_crud_delete.png
24	24	Operadores de campo	Etapas 4	24_filtros_operadores.png
25	25	exclude y Q	Etapas 4	25_filtros_exclude_q.png
26	26	Filtros a través de relaciones	Etapas 4	26_filtros_relaciones.png
27	27	NULL frente a texto vacío	Etapas 4	27_filtros_null_vacio.png
28	28	Orden, límite y SQL generado	Etapas 4	28_filtros_orden_sql.png
29	29	aggregate	Etapas 4	29_aggregate.png
30	30	Agrupación por tipo	Etapas 4	30_group_by_tipo.png
31	31	annotate sobre clientes	Etapas 4	31_annotate_clientes.png
32	32	La trampa del saldo	Etapas 4	32_saldo_trampa.png
33	33	Saldo correcto con subconsultas	Etapas 4	33_saldo_subqueries.png

Figura	Captura	Descripción	Etapas	Archivo
34	34	raw() básico	Etapas 4	34_raw_basico.png
35	35	Parámetros e inyección SQL	Etapas 4	35_raw_parametros.png
36	36	Columnas extra y clave primaria	Etapas 4	36_raw_columnas_extra.png
37	37	SalDOS con cursor	Etapas 4	37_cursor_salDOS.png
38	38	UPDATE con cursor	Etapas 4	38_cursor_update.png
39	39	demo_consultas en SQLite	Etapas 4	39_demo_consultas_sqlite.png
40	40	demo_consultas en PostgreSQL	Etapas 4	40_demo_consultas_postgresql.png
41	41	Idioma y zona horaria en settings.py	Etapas 5	41_settings_idioma_zona.png
42	42	Pantalla de inicio de sesión del panel, en español	Etapas 5	42_admin_login_es.png
43	43	Migración 0002 generada y su SQL vacío	Etapas 5	43_migracion_0002.png
44	44	Migración 0002 aplicada	Etapas 5	44_migrate_0002.png
45	45	Configuración de admin.py	Etapas 5	45_admin_py.png
46	46	Modelos registrados en el panel	Etapas 5	46_admin_registrados.png
47	47	Inicio del panel	Etapas 5	47_admin_inicio.png
48	48	Listado de clientes con su cantidad de cuentas	Etapas 5	48_admin_clientes.png
49	49	Ficha de un cliente con sus tablas anidadas	Etapas 5	49_admin_cliente_inlines.png
50	50	Listado de cuentas con saldos y filtros	Etapas 5	50_admin_cuentas_salDO.png
51	51	Listado de transacciones con filtro y navegación por fecha	Etapas 5	51_admin_transacciones.png
52	52	Validación en un formulario del panel	Etapas 5	52_admin_validaciones.png
53	53	Borrado protegido de una cuenta con movimientos	Etapas 5	53_admin_borrado_protegido.png
54	54	Panel funcionando sobre PostgreSQL	Etapas 5	54_admin_postgresql.png
55	55	Reglas del saldo probadas desde la shell	Etapas 6	55_regla_salDO_shell.png
56	56	Movimiento en modo lectura en el panel	Etapas 6	56_admin_movimiento_solo_lectura.png
57	57	SalDO insuficiente en el panel	Etapas 6	57_admin_salDO_insuficiente.png
58	58	Estructura de templates en VS Code	Etapas 6	58_estructura_templates.png
59	59	Página de inicio	Etapas 6	59_pagina_inicio.png
60	60	Listado de clientes	Etapas 6	60_clientes_lista.png
61	61	Ficha de un cliente	Etapas 6	61_cliente_detalle.png
62	62	Formulario de nuevo cliente	Etapas 6	62_cliente_formulario.png
63	63	Error de correo duplicado	Etapas 6	63_cliente_error_email.png
64	64	Cliente creado	Etapas 6	64_cliente_creado.png

Figura	Captura	Descripción	Etapas	Archivo
65	65	Confirmación para eliminar un cliente	Etapas 6	65_cliente_eliminar_confirmar.png
66	66	Eliminación protegida de un cliente con historial	Etapas 6	66_cliente_eliminar_protegido.png
67	67	Protección CSRF en clientes	Etapas 6	67_csrf_403.png
68	68	Listado de cuentas	Etapas 6	68_cuentas_lista.png
69	69	Ficha de una cuenta	Etapas 6	69_cuenta_detalle.png
70	70	Formulario de nueva cuenta	Etapas 6	70_cuenta_formulario.png
71	71	Cuenta creada	Etapas 6	71_cuenta_creada.png
72	72	Cuenta desactivada tras editarla	Etapas 6	72_cuenta_editar_inactiva.png
73	73	Eliminación protegida de una cuenta con movimientos	Etapas 6	73_cuenta_eliminar_protegida.png
74	74	Protección CSRF en cuentas	Etapas 6	74_csrf_cuentas_403.png
75	75	Listado de transacciones	Etapas 6	75_transacciones_lista.png
76	76	Listado filtrado	Etapas 6	76_transacciones_filtros.png
77	77	Página 2 del listado	Etapas 6	77_transacciones_pagina2.png
78	78	Ficha de un movimiento	Etapas 6	78_transaccion_detalle.png
79	79	Formulario de nuevo movimiento	Etapas 6	79_transaccion_formulario.png
80	80	Movimiento registrado	Etapas 6	80_transaccion_creada.png
81	81	Saldo insuficiente	Etapas 6	81_transaccion_saldo_insuficiente.png
82	82	Error de validación en el formulario	Etapas 6	82_transaccion_validaciones.png
83	83	Validación repetida dentro de la operación atómica	Etapas 6	83_servicio_validacion_repetida.png
84	84	Protección CSRF en transacciones	Etapas 6	84_csrf_transacciones_403.png
85	85	Contactos en la ficha de un cliente	Etapas 6	85_cliente_contactos.png
86	86	Formulario de agregar contacto	Etapas 6	86_contacto_formulario.png
87	87	Contacto agregado	Etapas 6	87_contacto_agregado.png
88	88	Confirmación para quitar un contacto	Etapas 6	88_contacto_quitar_confirmar.png
89	89	Contacto quitado	Etapas 6	89_contacto_quitado.png
90	90	Atajo Transferir: formulario con la transferencia y el destino elegidos	Etapas 6	90_atajo_transferir.png
91	91	Protección CSRF en contactos	Etapas 6	91_csrf_contactos_403.png
92	92	Inicio con totales y últimos movimientos	Etapas 6	92_inicio_ampliado.png
93	93	Reporte, sección CLP	Etapas 6	93_reporte_clp.png
94	94	Reporte, sección USD y movimientos por mes	Etapas 6	94_reporte_usd_meses.png
95	95	Formulario de ingreso	Etapas 7	95_login_formulario.png
96	96	Error de credenciales	Etapas 7	96_login_error.png
97	97	Inicio con la sesión iniciada	Etapas 7	97_inicio_con_sesion.png
98	98	Redirección al login con ?next=	Etapas 7	98_redireccion_login.png

Figura	Captura	Descripción	Etapas	Archivo
99	99	Menú de un cliente, sin enlace al panel	Etapas 7	99_menu_cliente_sin_panel.png
100	100	El login sin token CSRF es rechazado con 403	Etapas 7	100_csrf_login_403.png
101	101	Estructura de la carpeta static	Etapas 7	101_static_estructura.png
102	102	findstatic y collectstatic	Etapas 7	102_findstatic_collectstatic.png
103	103	Login con estilos	Etapas 7	103_login_con_estilos.png
104	104	Inicio con estilos	Etapas 7	104_inicio_con_estilos.png
105	105	Listado con estilos	Etapas 7	105_listado_con_estilos.png
106	106	Mixins de roles en mixins.py	Etapas 7	106_roles_mixins_py.png
107	107	Matriz de permisos	Etapas 7	107_matriz_permisos.png
108	108	Menú del personal	Etapas 7	108_menu_personal.png
109	109	Menú de un cliente	Etapas 7	109_menu_cliente.png
110	110	Página 403 para un cliente	Etapas 7	110_403_cliente.png
111	111	Inicio de un usuario sin cliente	Etapas 7	111_inicio_sin_cliente.png
112	112	Inicio de un cliente	Etapas 7	112_inicio_cliente.png
113	113	Cuentas: solo las propias	Etapas 7	113_cuentas_solo_propias.png
114	114	Movimientos: solo los propios	Etapas 7	114_movimientos_solo_propios.png
115	115	404 al abrir la ficha de otro cliente	Etapas 7	115_404_ficha ajena.png
116	116	Destinos permitidos en una transferencia	Etapas 7	116_destinos_permitidos.png
117	117	Depósito a una cuenta ajena rechazado	Etapas 7	117_deposito_ajeno_rechazado.png
118	118	Enlace de registro en el login	Etapas 7	118_login_con_registro.png
119	119	Formulario de registro	Etapas 7	119_registro_formulario.png
120	120	Errores de validación en el registro	Etapas 7	120_registro_errores.png
121	121	Registro exitoso: inicio con la cuenta nueva	Etapas 7	121_registro_exitoso.png
122	122	Cliente y cuenta creados, vistos en el panel	Etapas 7	122_admin_cliente_registrado.png
123	123	Perfil de un cliente	Etapas 7	123_perfil_cliente.png
124	124	Perfil del personal	Etapas 7	124_perfil_personal.png
125	125	Edición de los datos propios	Etapas 7	125_editar_mis_datos.png
126	126	Error al escribir mal la contraseña actual	Etapas 7	126_clave_error.png
127	127	Contraseña cambiada con aviso de éxito	Etapas 7	127_clave_cambiada.png
128	128	Resultado de verificar_accesos	Etapas 7	128_verificar_accesos.png
129	141	Suite completa: 239 pruebas correctas	Etapas 8	141_test_todas_239.png
130	142	verificar_accesos: 55 de 55 comprobaciones correctas	Etapas 8	142_pruebas_correctas.png
131	VP	Portada en escritorio	Etapas 9	escritorio_portada.png

Figura	Captura	Descripción	Etapas	Archivo
132	VP	Inicio del personal en escritorio	Etapas 9	escritorio_inicio_personal.png
133	VP	Portada en tablet	Etapas 9	tablet_portada.png
134	VP	Inicio en tablet	Etapas 9	tablet_inicio.png
135	VP	Portada en celular	Etapas 9	movil_portada.png
136	VP	Inicio del personal en celular	Etapas 9	movil_inicio_personal.png
137	VP	Inicio del cliente en celular	Etapas 9	movil_inicio_cliente.png
138	VP	Movimientos en celular	Etapas 9	movil_transacciones.png
139	144	Servicio alke-wallet en estado Live en Render, desplegado desde la rama main	Etapas 10	144_render_servicio_live.png
140	149	Portada en línea, en alke-wallet.onrender.com	Etapas 10	149_app_en_linea_portada.png
141	145	Inicio del personal en línea, con los totales y los últimos movimientos	Etapas 10	145_app_en_linea_inicio_personal.png
142	148	Inicio de un cliente en línea, con su cuenta y su saldo	Etapas 10	148_app_en_linea_inicio_cliente.png
143	146	Página 404 propia en línea, con el botón que vuelve al panel	Etapas 10	146_app_en_linea_404.png
144	147	Comprobación de salud en línea: {"estado": "ok"}	Etapas 10	147_app_en_linea_salud.png
145	150	Tablas gestion_* creadas por las migraciones en la base de Neon	Etapas 10	150_neon_tablas.png
146	143	Instalación desde cero y pruebas en una copia limpia	Etapas 11	143_instalacion_desde_cero.png